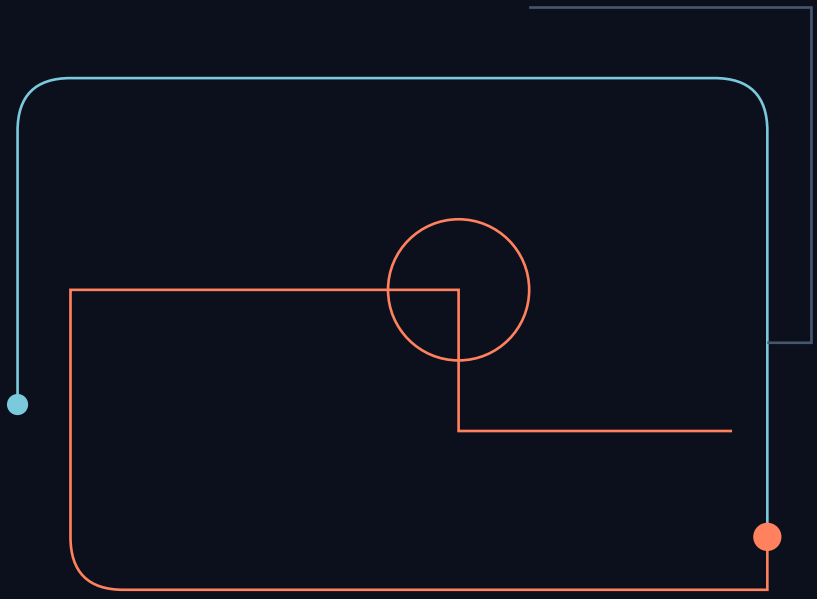
CODEX MASTERY LAB

驾驭 Codex

从基础使用到复杂任务掌控

把模糊想法转化为可验证成果的实战方法



FIELD GUIDE / FIRST EDITION

8 个部分 · 28 章 · 5 类实战 / 2026

OBSERVE. VERIFY. ADAPT.

驾驭 Codex：从基础使用到复杂任务掌控

第一版 · 离线阅读版 / 2026-10-26

Codex Mastery Lab 是独立开源中文教学项目。正文来自同一份 Markdown：导读、28 章与来源附录。网页交互在这里转换为明确的离线练习；示意、参考程序执行和读者实际验证仍分别标明。

案例源码与在线阅读

https://liamamilin.github.io/codex-mastery/
github.com/liamamilin/codex-mastery

运行命令必须在正文规定的工作目录执行。书中代码块为适应 A5 页面可能换行；复制命令优先使用网站或原始 Markdown，避免把版面换行当作命令换行。图片来自可编辑 SVG 与真实案例截图。中文字体为 Noto Sans SC，按 SIL Open Font License 嵌入。

目录

导读：把想法带到可验证的成果 5

第 1 章：Codex 不只是聊天窗口 9

第 2 章：建立可操作的工作环境 15

第 3 章：第一次从指令走到可见成果 21

第 4 章：当你还不知道自己真正想要什么 29

第 5 章：把期望转成可验证的完成状态 36

第 6 章：给 Codex 的任务说明怎么写 41

第 7 章：接手一份你不熟悉的代码 46

第 8 章：提供信息，不等于堆满信息 51

第 9 章：项目约定与会话接续 55

第 10 章：什么时候直接做，什么时候先研究 61

第 11 章：用最小实验处理关键不确定性 66

第 12 章：把较大任务分成有价值的里程碑 71

第 13 章：如何读懂进度并在途中介入 75

第 14 章：变化范围、版本和回退 80

第 15 章：从“程序启动”到“功能真的可用” 86

第 16 章：代码审查、解释和交付 90

第 17 章：识别错误发生在什么层次 94

第 18 章：停止无效修补，重新检查假设 99

第 19 章：换路线、降范围与及时止损 106

第 20 章：他说完成了，但你怎么确定? 111

第 21 章：让有价值的方法成为 Skill 116

第 22 章：MCP 与插件，让工具进入工作流 120

第 23 章：把重复操作变成脚本与自动化 125

第 24 章：模型、推理强度、时间和成本 129

第 25 章：跨会话、跨阶段推进一个项目 133

第 26 章：什么时候值得使用多个 Agent 139

第 27 章：把一次成功变成下一次更容易成功 143

第 28 章：毕业实战--在陌生任务上独立驾驭 Codex 148

附录：术语、操作与来源 152

导读：把想法带到可验证的成果

你可能已经见过 Codex 写代码、解释报错，甚至连续工作很久。真正困难的时刻往往出现在后面：它说完成了，你却不知道如何检查；它修改了很多文件，问题仍然存在；新会话开始后，之前已经确认的事情又被当成未知。

本书从这些处境出发。学完之后，你应当能够接住一个陌生任务，让 Codex 获取必要事实，选择合适的下一步，完成一份可使用的成果；遇到偏差时，知道怎样改变路线，最后保存足够的状态，让工作能够继续。你会逐渐读懂代码，但起步时不需要会编程。

这里把“驾驭”理解为一种判断能力。你无需逐条指定 Codex 的每个动作，却需要知道目标在哪里，什么证据足以支持继续，以及什么时候应当介入。这种能力比记住某个菜单的位置更耐用。

第一趟学习，只完成一个小闭环

建议第一次按顺序阅读导读与第 1-3 章。第 1 章训练观察：分清它说了什么、实际查了什么、文件有没有变化。第 2 章把工作位置和工具准备好。第 3 章让你亲手做出 LinkShelf，一个在浏览器中添加和保存链接的小工具，并检查刷新之后的数据。

这三章的终点很具体：你的电脑上有一个独立练习目录，里面有可打开的页面；你能新增一条链接，刷新，删除，再刷新；你能解释哪些行为亲自看过，哪些还没有检查。页面颜色与书中截图不同不影响达成这个终点。截图的作用是帮助你找到界面，不是要求模型生成同一张脸。

第一趟结束后，再沿着完整课程深入。全书由八个部分、二十八章组成：认识执行搭档；把想法变成工作；调查上下文；选择推进路线；验证与交付；诊断和纠偏；扩展与复用；长期任务与毕业实战。站点的课程目录列出每章的能力目标与阅读入口。先完成一条可操作的学习路径，再按自己的任务回到相应章节。

一本书，三处工作空间

阅读时，你会在三个地方来回切换。教材网站提供讲解、图解和交互判断练习；操作系统终端接受短命令，例如查看目录、启动本地网页；Codex 的输入框接受自然语言任务，让它调查、修改或解释。代码块前会写清楚应该在哪里输入。

终端（terminal）可以理解为用文字操作电脑的窗口。macOS 可打开“终端”，Windows 可打开 PowerShell。你不用先学完命令语言，只需在给定目录逐条运行书中的命令。cd 表示切换目录；路径表示文件在电脑中的位置。遇到这些词不用硬记，第一次真正需要它们时，第 2 章会带你验证。

网站中的交互组件运行在浏览器里。选择“下一步”后出现的情境是教学演示，不会调用你的 Codex，不会修改你的本地案例。真正的实验需要你把任务发送给自己已登录的 Codex，并观察实际文件与浏览器结果。交互题答对，说明你理解了一种判断；本地案例通过，才说明你完成了那次操作。

本书主要采用 Codex CLI，也就是在终端使用的入口。App、IDE 扩展和云端在工作位置、工具与权限上可能不同。你可以用自己熟悉的入口完成同一任务，但应先确认它连接的是哪个目录，能否读文件、执行命令和修改文件。界面没有出现书中的控件时，先查入口差异，不必为了匹配截图反复重装。

先拿到教材源码，再决定从哪里开始

在线阅读不需要安装本网站的开发依赖。要运行案例，可打开本书 GitHub 仓库，选择 Code → Download ZIP，解压到专用学习目录。下载的是源码包，网页本身不会自动出现在你的电脑上。解压后找到同时包含 book 和 cases 的那一层文件夹，这就是下文说的“教材根目录”。

主要文件按用途组织：book/ 是中文正文；cases/A-linkshelf/starter/ 是第一项练习的起点；cases/A-linkshelf/solution/ 是可运行的参考成品。案例说明在 cases/A-linkshelf/README.md，在线入口见实战实验室。后续案例 B 用于接手陌生代码，C 用于离线数据汇总，D 用于真实故障调查，E 提供两道独立毕业题。它们都在 cases/ 中；E 保留题目起点，不提供完整参考解。

第一种学习方式是从 starter 复制一份自己的练习目录，再给 Codex 目标，让它完成实现。第二种方式是直接运行 solution，先摸清“完成以后应该是什么感觉”。两种方式都有效。只想先体验成果，可以先跑参考成品；想训练指挥能力，需要回到自己的副本，经历任务、执行、验收与反馈。

不要把公司的项目或个人唯一一份资料当作第一块练习场。使用独立目录，是为了让你可以放心试验、比较和重新开始。教材中的链接数据也是虚构样本，无需导入真实收藏。

用结果判断，不跟着措辞走

书中会明确区分四种证据。“官方核对”表示查阅过注明日期的官方资料；“实际执行”表示在记录的环境中运行过命令或检查；“人工待验”表示仍需要真实观察；“示意”表示为讲解构造的任务或对话。预期结果不是已经发生的日志。

例如，“网页返回了正常响应”只能证明服务提供了页面。“添加后刷新仍存在”才支持保存功能可用。如果你只试了一条正常链接，就不能顺带宣布所有错误输入都处理正确。每次只写观察能够支持的结论，反而更容易找出下一步。

产品事实也有适用范围。本轮官方资料核对日期为 2026-10-25，本机命令核查环境为 macOS、Codex CLI 0.154.0、Node.js 22.21.1、Python 3.10.13。版本号记录的是这次制作环境，不是全体读者的最低要求，也不是“最新版”的承诺。Windows 与账号登录路径依据官方资料讲解；没有把它们写成本机实测。附录保留具体来源和边界。

开始之前，留下你的第一份判断

假设 Codex 说：“我优化了保存逻辑，应该解决了刷新丢失。”你打开页面，新增成功；刷新之后，条目不见了。此时你最可能怎样做？

1. 再发一次“请认真修好”。

2. 先要求重写整个应用。

3. 记下新增、刷新、消失的步骤，要求调查保存与读取路径。

先选一个，并用两句话解释理由。现在不必追求术语准确。把答案保存在自己容易找到的笔记里，到纠偏章节再回来看。

参考反馈（离线展开）

第三种做法能带来新信息：它把“没修好”变成可重复的观察，并指向有区分力的检查。第一种没有增加事实，第二种在根因未知时扩大了变化范围。如果你选择前两项，也已经找到本书最值得优先练习的能力：让下一步由证据推动。

继续前，你只需要确认三件事：知道案例文件放在哪里；愿意亲手完成一次检查；能接受一次失败暴露未知条件。带着这三个准备，进入第 1 章，先看清 Codex 是怎样工作的。

第 1 章：Codex 不只是聊天窗口

“它已经把保存功能写好了。”你看到这句话时，可能会自然地松一口气。但先停一秒：你究竟看到了一个承诺、一段代码，还是一项已经运行的功能？这三者在聊天里很容易挨在一起，实际却需要不同的证据。

本章先不修改程序。我们用 LinkShelf 案例做一次只读探索，最后得到一份能指向实际文件的说明。完成后，你应能看着一次 Codex 工作过程，分别指出它说了什么、查了什么、改了什么，以及还有什么没有验证。如果尚未安装 Codex，可以先读到练习指令，再按第 2 章完成准备后回来操作。

从一句回复，看见执行循环

模型（model）根据当前信息生成下一步内容。Agent（智能体）则在执行环境中围绕任务使用工具、接收反馈，并继续作出行动选择。这里的“工具”可以是读取文件、运行终端命令、修改代码，也可能是浏览器。运行这些工具的外围系统通常称为 harness；它负责把会话、工具和权限等组织起来。不同入口所提供的工具并不完全相同。OpenAI 的 Codex 执行架构说明解释了这一关系。

你给出的目标不会直接变成电脑里可用的软件。中间会经过一连串动作：读取已有内容，选择修改位置，写入文件，运行检查，根据结果调整。某一步失败之后，模型也可能作出错误判断。因此，“让它自主完成”仍然需要一个可检查的终点。



从任务意图、事实调查到执行、验证和调整的工作循环

图中最值得注意的是反馈回路。执行得到的结果会改变下一步：文件不存在，就先找目录；测试失败，就查看失败指向什么；关键行为已通过，就交付。它不是一条“指令越长、结果越好”的直线。

在下面的交互里，逐步查看同一任务如何从意图走向证据。这里的状态和结果是教学示意，不是一次实际 Codex 会话。

> 离线版说明：以下为网页互动的文字替代。

离线练习：沿工作循环判断

按“意图 → 事实 → 路线 → 行动 → 证据 → 调整 → 交付”检查自己的任务。每一步写一条已知事实或下一项动作；简单任务无需逐项填表。网页交互不在电子书中运行。

先把四类东西分开看

一次会话里，先留意以下区别。

你看到的东西	能说明什么	还不能说明什么
“我准备检查保存逻辑”	Codex 表达了行动意图	它已经读过代码
读取文件的工具返回	某些文件内容进入了当前上下文	解释一定准确
修改记录或文件差异	指定文件确实发生了变化	新行为已经可用
测试或手工操作结果	指定条件下出现了某个结果	所有条件都成立

工具返回也不是绝对真理。例如，命令在错误目录运行，会忠实报告那个目录的情况；一个测试通过，只覆盖它设计的那条路径。证据的价值来自它与当前问题的关系。你要检查的是“这次观察支持哪个结论”，而不是“有没有出现绿色的通过字样”。

下面这组题训练这种分辨。答错时，先看反馈中缺少的是哪一层证据，再重试。

> 离线版说明：以下为网页互动的文字替代。

离线练习：证据判断

哪一项证明已行动？A.“我检查过，应该没问题。” B.工具返回实际测试命令、退出码和具体结果。C.准备检查三个文件。

参考反馈：B 提供执行证据，仍需检查测试覆盖；A 是陈述，C 是计划。这些是教学情境，不是真实执行日志。

做一次不会改文件的项目探索

前置条件是已经下载教材源码，并能在 Codex 中打开教材根目录。CLI 用户按第 2 章进入该目录后启动；App 或 IDE 用户选择同一目录。先用文件管理器确认能看到 cases/A-linkshelf/solution/，里面应有网页与脚本。我们的调查对象是这份参考成品，练习起点 starter 则留到第 3 章使用。

以下内容发给 Codex 输入框，不是在操作系统终端逐行执行：

```
请只读调查 cases/A-linkshelf/solution，不修改或新建文件，
不安装依赖，不启动长期运行的服务。
我想知道它怎样把一条链接从输入框保存到刷新后的页面。
请先确认当前目录，再读取必要文件，说明：
1. 页面入口、界面样式和行为脚本分别在哪里；
2. 新增链接时经过哪些处理，刷新后从哪里读取数据；
3. 哪些结论能引用文件中的代码，哪些需要运行后才能确认。
最后给我一条下一步验证动作，并列出实际执行的检查。
```

这段任务先限定行动范围，再给调查问题与期望产物。限制只读是因为此刻我们缺的是理解，修改会让调查对象同时变化。“读取必要文件”给它选择空间，不要求把整个仓库塞进上下文。最后要求区分代码事实与运行结果，是为了避免它读到存储调用就宣布刷新已通过。

你可能看见它列目录、读取 HTML、再定位脚本中的函数；也可能直接按文件名读取。动作顺序不同没有关系。先观察工具记录，找出它究竟读取了哪些路径。如果当前入口没有文件工具，它可能只能讨论你粘贴的内容。这时应记录“尚未直接调查本地文件”，并按第 2 章连接正确工作区。

怎样检查这份说明

不要只读最后的总结。打开实际文件，挑一条关键结论核对：例如“刷新时从浏览器存储加载数据”。在脚本中寻找它引用的存储键或读取调用，再向前看这段代码何时执行。不会读代码也可以要求 Codex 展示前后几行并用普通话解释，重点是让解释有可定位的对象。

接着比较“读取到代码”和“观察到功能”。假设脚本里存在 localStorage.setItem，这支持“实现尝试写入浏览器存储”；它不直接证明每次保存都成功。存储可能被浏览器限制，调用也可能没有经过当前按钮触发。是否能保存，仍需要新增并刷新。这个检查会在第 3 章完成。

最后检查变化范围。这个任务正确的交付是调查结论，教材文件应保持原样。若使用的界面有变更列表，查看是否出现新增或修改。已会 Git 的读者可比较任务前后的 git status --short；尚未学习 Git 的读者可以让 Codex说明是否写入文件，并在

文件管理器中核查。仅凭它说“我没有修改”仍然是一份声明，变化列表能提供更直接的依据。

可以用下面的文字追问，继续发给 Codex：

```
请从刚才的说明中选出一条最重要的结论，
引用它对应的真实文件与代码片段。
再指出一条你没有运行验证、因此不能直接宣布通过的结论。
```

完成时，你应该能自己写出四句话：它表达了什么判断；实际读取了哪些文件；是否有文件变化；下次怎样检查刷新保存。无需写成正式报告，四句话已足够让你的判断落在事实之上。

当过程与预期不同时

如果它一开始就提议重构页面，说明当前执行偏离了调查目标。此时发出具体纠正：“先停止修改。当前任务只需要解释现有保存路径，请回到已读取的文件并指出证据。”如果已经发生修改，先查看改动范围，保留原有用户修改，按界面的差异确认只撤销这次误改；不要直接清空整个目录。

如果它回答得很流畅，却没有工具记录，也没有文件引用，先问它依据什么作答。它可能使用了已有上下文，也可能只做了推测。缺少日志不能自动证明它在编造，但你仍需要能追溯的文件依据。反过来，看到很多命令也不等于工作有效：十次列目录如果没有接近保存路径，就应缩小问题。

还有一种常见结果是权限拒绝。它说明某个动作没有获准，不能被写成“这个项目没有文件”。记录被拒绝的动作、目标路径与错误，再判断是否选错工作位置。权限与文件是否存在，是两个不同的问题。

动手练习：给结论补上证据

沿用这次调查，在自己的笔记里改写以下三句话，使每句都符合目前掌握的证据：

1. “程序有保存功能，所以数据不会丢。”

2. “Codex 说它看完了，因此已经了解全部项目。”

3. “工具报错，说明代码坏了。”

参考反馈（离线展开）

第一句可以改为：“脚本包含保存与读取逻辑；是否能在当前浏览器刷新后保留，需要新增后刷新验证。”第二句应缩到实际读取的文件，例如“它读取了入口和行为脚本，尚未调查测试与其它目录”。第三句要保留报错来源，例如“读取路径被拒绝，目前还不能判断实现是否有错”。合格答案会缩小结论，指出下一项能消除不确定性的观察。

你现在获得的第一项能力，是把一次看似连续的聊天拆成可观察的工作过程。接下来在第 2 章准备好真正可以行动的环境。

第 2 章：建立可操作的工作环境

你输入了 codex，电脑却说找不到命令；换到 App 之后，它又找不到案例文件。两个问题表面上都像“Codex 用不了”，实际分别发生在命令入口和工作位置。环境准备的价值，就是把这类问题变成可以逐项确认的事实。

本章的完成状态是一段简短记录：你使用哪个入口、Codex 的版本是什么、当前工作目录在哪里、它能读到哪个案例文件，以及当前允许它做什么。这里不要求修改任何教材文件。安装需要网络和电脑上的安装权限；使用 Codex 还需要可用账号与相应访问权限。没有账号访问时，仍可先运行第 3 章参考成品，实际 Codex 协作练习留待条件具备后完成。

先找到自己的位置

打开 macOS 的“终端”，或 Windows 的 PowerShell。终端中接收命令的程序叫 shell。它有一个“当前目录”：相对路径以这里为起点。cases/A-linkshelf 并不是电脑上唯一的位置，它只有在教材根目录下才指向我们的案例。

先在文件管理器找到下载并解压的教材文件夹，复制它的完整路径。下面的路径是需要替换的示例。不要把书作者电脑的目录照搬到自己的机器。

macOS / Linux，在操作系统终端输入：

```
cd "/你的实际路径/codex-mastery"
pwd
ls
```

Windows，在 PowerShell 输入：

```
Set-Location "C:\你的实际路径\codex-mastery"
Get-Location
Get-ChildItem
```

第一条改变当前位置，第二条把位置显示出来，第三条列出这一层的内容。成功的观察是：路径确实指向你解压的教材，并且列表里同时出现 book、cases 等文件夹。路径两侧的引号让带空格的目录仍被当作一个参数。文件夹名称可能带下载分支后缀，以你实际看到的为准。

如果出现“找不到路径”，先比较文件管理器中的名称、层级与命令。不要新建一个同名空目录来让报错消失，否则下一步虽然能进入，案例仍然不存在。路径问题应该通过找到正确对象解决。

选择一个能启动的 Codex 入口

先在操作系统终端执行下面两条，它们只查看程序信息，不要求登录：

```
codex --version
codex --help
```

若能显示版本与帮助，说明终端已经能找到 Codex，可以跳过安装。若提示找不到命令，才需要安装或检查命令位置。--help 是你当前程序的说明，尤其适合判断书中的参数在本机是否存在。

官方 CLI 页面同时提供独立安装器与 npm 等安装方式，并说明从项目目录启动 codex。Codex CLI 官方文档于 2026-10-25 核对。本书下面选择 npm 路线，是为了让具备 Node.js 的读者使用一条常见路径；它不是使用 Codex 的唯一条件。

先运行：

```
node --version
npm --version
```

两条都有版本输出，表示 Node.js 运行时和 npm 包管理器可用。本书制作机器使用 Node.js 22.21.1；这是一条实测记录，不表示 CLI 永久要求这个精确版本。若任一命令不存在，先从 CLI 官方安装页选择适合系统的独立安装器，或安装 Node.js 后重新打开终端。不会安装 Node.js 的读者采用官方独立安装器，通常能少引入一

个当前不需要学习的工具。

仅在决定采用 npm、且上面的检查可用时，于操作系统终端执行：

```
npm install -g @openai/codex@latest
codex --version
```

此命令安装全局可执行程序，会从网络下载软件；latest 随时间变化。官方 Codex Cookbook仍列出这一 npm 安装方式；Windows 部署文档也包含 npm 路线。安装报权限错误时，先记录错误与安装位置，按官方指导选择用户可写的安装方式。在组织管理的电脑上，沿用管理员允许的路径即可。

Windows PowerShell 若提示脚本执行被策略阻止，可先尝试 npm.cmd --version，可用时将上述安装命令的 npm 改为 npm.cmd。通过 npm 安装的 Codex 也有脚本启动入口；遇到同类拦截时，将后文操作系统命令中的 codex 改为 codex.cmd，例如 codex.cmd --version、codex.cmd login。如果组织策略仍不允许执行，使用其批准的安装方法。Windows 原生终端与 WSL2 是不同环境；装在其中一个环境的程序不等于另一个也已安装。已有 WSL2 工作流的读者可参考官方 WSL 指南，在其 Linux 终端中操作，并使用 Linux 路径。

登录与启动是两件事

安装成功后，在教材根目录的终端输入：

```
codex login
```

按实际界面完成浏览器登录，再回到终端检查：

```
codex login status
```

官方目前区分 ChatGPT 登录和 API Key 登录，两者访问与计费方式不同；本书入门采用已有 ChatGPT 账号的交互登录路径。可用权益取决于账号和工作区，不因

安装完成就自动获得。官方身份验证说明列出了这些差异。不要为跟随前三章额外开通 API 付费；遇到访问限制，先确认当前账号的可用范围。

codex login status 能报告登录方式，不能证明每个模型、每项工具都能调用。这里也不要把凭据文件内容粘贴给模型。我们只需要状态信息，账号密钥不会帮助定位教材目录。

接下来启动交互会话：

```
codex --sandbox read-only
```

read-only 为这次调查选择只读沙箱。沙箱（sandbox）规定执行可访问与修改的边界；审批策略则决定哪些动作需要额外确认，两者共同影响实际行为。官方沙箱文档说明了两者的关系。组织或宿主的管理策略也可能限制最终权限，以会话实际显示为准。

进入 Codex 以后，在它自己的输入框输入 /status，查看工作位置和当前配置。斜杠命令属于 Codex，不能在普通 shell 提示符下代替 pwd。随后发送这段自然语言：

```
请确认当前工作目录，只读检查 cases/A-linkshelf/starter/TASK.md
是否存在，并用两句话概述任务。不要创建或修改文件，
不要安装依赖。请说明你实际读取的是哪个路径。
```

完成标准是：它通过文件工具实际读到任务文件，报告的路径与教材目录相符，没有把参考成品误认为起始状态。如果读不到，先看它使用的路径和工具错误；如果无法访问文件工具，就还没有完成这个检查。

App 与 IDE 用户怎样完成同一项检查

使用桌面 App 时，在 Codex 中打开教材文件夹并开始聊天，确认聊天所关联的工作位置。官方当前将桌面入口放在 ChatGPT desktop app 文档中；名称与旧版 App 可能不同，核对的是“选择工作位置再发起任务”这项操作，而不是旧截图的按钮布局。桌面入口官方说明。

VS Code 或兼容编辑器用户先打开教材文件夹，再打开 Codex 侧栏；如果图标未显示，可在命令面板寻找 Codex: Open Codex Sidebar。编辑器能显示文件，不代表 Codex 一定已读取；仍发送上面的只读任务，用返回的真实路径确认。IDE 扩展官方说明。

CLI 的启动参数、App 的权限控件与 IDE 的控件入口不相同。先找到当前会话的权限说明，选择能完成教材调查的范围；遇到拒绝，检查目标目录和请求的动作。不需要为了读一份任务书开放整台电脑。

为第一次实施准备两个终端

第 3 章将用一个终端保持 Codex 会话，另一个终端启动网页服务。两个窗口各自有当前目录；在第一个窗口执行 cd，不会替第二个自动切换。每次运行依赖相对路径的命令前，用 pwd 或 Get-Location 再确认一次。

网页服务采用 Python 3 标准库，先检查：

```
# macOS / Linux
python3 --version
```

```
# Windows PowerShell
py -3 --version
```

如果显示 Python 3 版本即可继续。Windows 没有 py 但 python --version 显示 Python 3 时，后文可将 py -3 换成 python。若都不可用，从 Python 官方下载页安装 Python 3，重开终端再检查。Node.js 与 Python 在本书中承担不同职责：前

者可用于安装 CLI，后者仅提供本地静态网页；它不替你执行模型推理。

动手练习：留下可以继续的环境事实

在自己的笔记里写五行：入口、CLI 版本或 App 版本、教材完整路径、已实际读到的文件、当前权限。把“已运行”和“尚未完成”分开。例如，版本查询已成功，但浏览器登录尚未返回，就分别记录这两个状态。

参考反馈（离线展开）

合格记录能让明天的你直接回到同一位置，并知道下一步缺什么。“已经安装好了”缺少入口与版本；“路径是 cases”缺少起点；“权限应该够”缺少实际读取证据。记录不必长，但每一行都应能由一次观察支持。

截至 2026-10-25，本书制作环境实际执行过版本查询与帮助查询，没有为编写本章重新安装或重新登录 CLI；Windows 路线也未在本机执行。这些边界保存在来源附录。准备就绪后，进入第 3 章，让一次任务真正落到文件与界面上。

第 3 章：第一次从指令走到可见成果

现在你已经能定位教材目录，也能让 Codex 读取一份任务。接下来做一件完整的小事：输入标题和网址，保存到列表，刷新后仍然找到它，删除后刷新也不会回来。

这个范围足以经历一次真正的协作，又小到可以逐项检查。我们把它叫作 LinkShelf 基础版。暂不引入账号、云同步或数据库，因为它们不会帮助你判断第一次保存是否完成。你最后需要拿到的，是自己的可运行文件和一份实际验收结果。

两种起点，先选一条

要训练从目标到实现，使用“自己制作”路径：把 starter 复制为独立工作副本，让 Codex 在副本中完成任务。想先了解完成形态，使用“运行参考成品”路径：直接打开 solution，完成相同的手工验收。参考成品用于对照，不代表你已经完成了一次 Codex 实施任务。

以下操作以前一章的 CLI、Python 3 和浏览器准备为前提。运行参考成品无需登录 Codex；自己制作需要可用的 Codex 会话。使用两个终端，一个交给 Codex，一个运行网页服务。在每个终端中都先进入教材根目录。

自己制作：把起点复制到独立目录

先确认教材根目录中尚无 my-linkshelf。如果已有同名练习，用另一个新名称，并同步替换后文路径，不要覆盖上次成果。

macOS / Linux，操作系统终端：

```
cp -R cases/A-linkshelf/starter my-linkshelf
cd my-linkshelf
codex --sandbox workspace-write
```

Windows PowerShell：

```
Copy-Item -Recurse cases/A-linkshelf/starter my-linkshelf
Set-Location my-linkshelf
codex --sandbox workspace-write
```

现在 Codex 的工作目录是你自己的副本。workspace-write 让它在当前允许的工作区中修改文件；实际权限仍以宿主和组织配置为准。App 或 IDE 用户打开这份副本，完成同样的任务即可。

将下面的文字发给 Codex 输入框：

```
请阅读 TASK.md，在当前目录实现 LinkShelf 基础版。
使用原生 HTML、CSS、JavaScript，浏览器本地存储，
无需安装依赖或连接外部服务。
用户可以输入标题和 http/https 网址，保存后出现在列表；
刷新后仍在；能删除条目，删除后刷新也不恢复。
空标题或不合要求的网址应有明确反馈，不保存错误条目。
先完成这一条使用路径，再考虑外观；本次不做账号或云同步。
请说明修改了哪些文件，给出本地启动方式。
能够执行的检查请实际执行；没有浏览器时明确写出待人工验收，
不要把阅读代码当成新增与刷新已经通过。
```

这里给出了实现边界和四种可观察行为，却没有指定每个函数的写法。技术细节由 Codex 在当前规模内组织。你负责保留完成条件，避免它为了“更完整”扩出不必要功能。

它工作期间，留意是否真的产生了文件；是否开始安装任务没有需要的框架；检查命令证明的是语法，还是浏览器行为。如果它只生成一个解释而没有落盘，直接要求：“请把实现写入当前目录的实际文件，再说明入口。”

启动本地网页

在第二个操作系统终端进入教材根目录。下面的服务只监听本机回环地址，不需要公开网络。

自己制作的副本，macOS / Linux：

```
python3 -m http.server 4174 --bind 127.0.0.1 --directory my-linkshelf
```

自己制作的副本，Windows PowerShell：

```
py -3 -m http.server 4174 --bind 127.0.0.1 --directory my-linkshelf
```

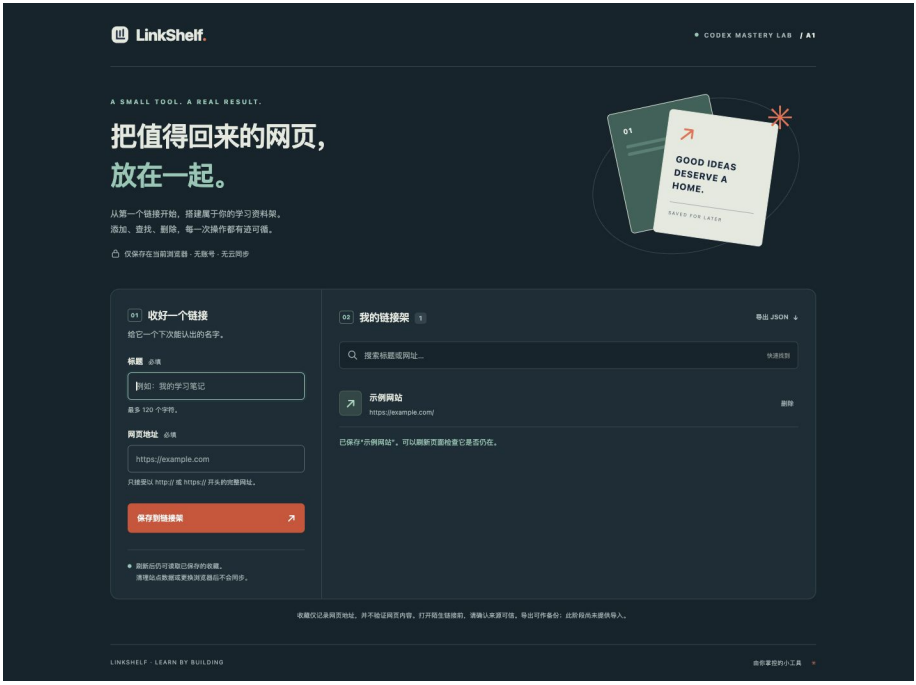
在浏览器地址栏打开 http://localhost:4174/。命令运行后终端持续等待，是服务器正在提供文件；保留这个窗口即可。返回该终端按 Ctrl+C 停止服务。目录与绑定地址参数可参阅 Python 官方 http.server 文档。

如果选择运行参考成品，在教材根目录改用下面的目录参数。先停止已有的 4174 服务，避免端口冲突。

```
# macOS / Linux
python3 -m http.server 4174 --bind 127.0.0.1 --directory cases/A-linkshelf
```

```
# Windows PowerShell
py -3 -m http.server 4174 --bind 127.0.0.1 --directory cases/A-linkshelf
```

此时浏览器打开 http://localhost:4174/solution/。两条路径的目录不同，所以浏览器地址也不同。不要把副本的根地址和参考成品的子目录混用；也不要双击 HTML 后用 file:// 作为本章的验证路径。



LinkShelf 参考成品的真实浏览器界面，包含新增表单与链接列表

图示是参考成品的真实界面。你自行生成的排版可以不同，下面按功能验收。参考成品的输入标签为“标题”“网页地址”，新增按钮为“保存到链接架”，每条记录旁有“删除”。

用五组操作证明关键路径

先不要导入自己的真实收藏。使用两条容易辨认的样本，每次只检查一种变化。

步骤	亲手操作	看见什么才算通过
新增	标题填“示例网站”，网址填 https://example.com，点击保存	列表出现对应标题与网址
刷新	刷新当前标签页	同一条记录仍存在

步骤	亲手操作	看见什么才算通过
再新增	标题填“学习资料”，网址填 https://example.org，点击保存	两条记录都能看到
删除	删除“示例网站”，再刷新	“示例网站”不再出现，“学习资料”仍在
错误输入	标题填“错误样本”，网址填 javascript:alert(1)，尝试保存	有反馈，列表不增加该条目

删除第一条时若第二条也被误删，说明列表虽然发生了变化，目标行为仍未完成。第二条样本的作用，是让你能区分“正确删除指定对象”和“简单清空列表”。多记录检查能暴露单条样本不易发现的问题。

如果浏览器原生校验直接阻止错误输入，也是一种有效反馈；无需要求反馈文案与截图逐字一致。关键是错误条目没有落入列表，已有的合法条目还在。记录实际看到的结果，某一项失败就写失败，不要用页面漂亮来抵消它。

参考成品额外提供标题或网址搜索，以及“导出 JSON”。可输入“学习”检查匹配，再清空搜索恢复全部条目。导出会下载当前数据快照，本阶段没有导入功能。这些是参考成品的可选扩展，不是自行实现基础版的必做要求。

刷新为什么值得单独检查

页面此刻显示的列表，通常保存在 JavaScript 的内存状态中；刷新后，这份状态会重新创建。要恢复收藏，应用必须把数据写到持久化位置，并在加载时读回来。本案例用浏览器的 localStorage，即按站点来源保存字符串的小型存储。下面的图把“屏幕显示”和“保存路径”分开画了出来。



输入、内存列表、localStorage 与刷新读取的数据路径

因此，“新增后能看见”只检查了当前页面，“刷新后还在”又检查了写入与读取的衔接；“删除后刷新不回来”则检查删除是否也同步保存。后面的故障课会让你专门调查这个衔接。此刻先把观察做准确，不必抢着背 API。

这些数据只在当前浏览器、当前站点来源中保存，不是云同步。localhost:4174 与 localhost:4175 的端口不同，存储就不同；localhost 与 127.0.0.1 也不是相同主机名。换浏览器、无痕窗口、清除网站数据，都可能看不到原来的收藏。做刷新验收时，保持浏览器与地址不变；需要保留重要数据时使用备份，不把浏览器存储当作永久档案。MDN 的 localStorage 说明记录了这些来源与存储策略限制。

参考成品使用专用存储键 codex-mastery:linkshelf:a1:v1。自己的实现可以选择另一专用键；为了避免两个练习的数据混在一起，不要要求所有版本共用同一个键。

失败时，先增加信息

若网页打不开，先回终端看服务是否仍在运行；如果是“地址已被占用”，先确认是否开着上一次服务，可 Ctrl+C 停止后再启动。也可把命令和浏览器地址一起改为 4175，但这会改变存储来源，原数据不会自动跟随。

若看到目录列表，通常是服务目录层级不对或没有 index.html；若参考成品 404，核对是否使用 /solution/。这些观察都指向运行入口，不宜立即让 Codex 重写保存逻辑。

若添加可见但刷新丢失，先保留复现步骤，再给 Codex 一项能够区分原因的任务：

```
同一浏览器、同一地址下，新增“示例网站”可见，刷新后消失。
请先检查新增时的数据写入与页面加载时的数据读取，
确认它们使用相同存储键，并检查是否有存储错误。
先解释证据指向哪里，再做最小修复。
修复后重跑新增→刷新→删除→刷新，不要顺便重做页面。
```

这段反馈把现象、稳定条件和下一项检查连在一起。它没有预先宣布根因一定是存储键；即使假设被推翻，也能沿实际结果调整。连续修改却没有新证据时，回到这个层次，比再加一句“务必正确”更有帮助。

动手练习：完成一次真实交付

现在请为自己的版本增加一个小改动：保存成功后清空标题与网址输入，列表保留新条目。先向 Codex 描述这个行为，再验证连续添加两条是否都成功。参考成品已有相应行为的读者，可以在副本中从起点完成，或只调查清空发生在保存成功之前还是之后。

参考反馈（离线展开）

观察重点是成功与失败的分支：合法输入保存后清空；错误输入应保留或清楚提示，便于修正；新增的条目刷新后仍在。只看到输入框变空，不能证明保存成功。如果存储失败但界面清空并提示成功，也应报告为缺陷。

> 离线版说明：以下为网页互动的文字替代。

离线自查：完成亲手验收

在纸上勾选：保存一条正确收藏；刷新后仍存在；错误网址被拒绝；删除后刷新不恢复。勾选是读者自查，不会自动检测程序，也不会同步网站学习记录。

在自己的笔记里留下三句话：版本目录和启动命令；实际通过的行为；尚未通过或未检查的部分。你已经把一次请求推进成可运行、可观察、可继续的成果。接下来的课程会以这个闭环为基础，逐步训练范围判断、验证与纠偏；已开放的内容请查看课程目录。

第 4 章：当你还不知道自己真正想要什么

做出 LinkShelf 以后，你可能又有了一个愿望：“我想把它做得更好用。”这句话是真的，但不足以决定下一项修改。收藏入口费事、资料不好找、保存以后不再阅读，都可以被称为“不好用”，它们却需要不同的行动。

如果让 Codex 立刻实现，它会替你填补这些空白。可能增加标签、推荐、登录和同步，最后功能越来越多，你仍然不知道最初的麻烦是否减轻。本章练习把未知需求转成一次小探索，完成时拿到的成果是：一个有依据的优先方向，以及一项知道为什么要做的下一步。

先把任务从“开发”改成“认识选择”

目标尚未明确，不意味着不能开始。此刻能够完成的工作，是把可能的使用场景摆在一起，让你作比较。本轮要调查教材参考成品，请在教材根目录打开 Codex；若上次停在 my-linkshelf，先切回含 book 与 cases 的那一层并核对位置。参考第 3 章的目录说明，在 Codex 输入框发送：

```
我已经能在本机使用 LinkShelf，但“更好用”指的是什么还不清楚。
请先不要修改代码。围绕快速收藏、快速找回、定期回顾，
各提出一个具体使用场景和一个小尝试。
先只读查看 cases/A-linkshelf/solution 的实际能力，
区分现在就能试的行为与需要新增的功能。
请帮我缩小选择，不要一次制定完整产品规划。
```

你给它的完成状态发生了变化：这次要得到有差异的选择，不是更多代码。只读调查也有作用，能防止讨论建立在不存在的标签、阅读记录或提醒系统上。

下面是为教学整理的候选比较，并非一次真实 Codex 对话的逐字记录。

方向	主要困扰	当前可以怎样试
快速收藏	看到网页时，输入与整理让人放弃保存	连续保存三条，观察卡在复制网址、填写标题还是点击保存
快速找回	记得存过，却记不清完整标题	用现有的标题或网址搜索，尝试不同记忆线索
定期回顾	保存之后，很少重新打开	手动挑三条并安排一次回顾，先判断回顾是否有价值

第三种方向当前只能做手工尝试。参考成品没有已读标记、最近阅读时间、提醒或自动回顾功能。手动把一条记录选来再看，可以了解使用价值；它不能被描述为“回顾功能已经实现”。同样，存在搜索框只证明有一个入口，搜索是否符合你的记忆方式，还要实际试。



把快速收藏、快速找回与定期回顾三种候选目标转成最小尝试，再由观察决定下一步

在下面的演示里选择一个方向，看看它最先需要什么证据。这个组件是本地教学情境，不会替你启动 Codex 或修改收藏。

> 离线版说明：以下为网页互动的文字替代。

离线练习：选择一个方向

比较快速收藏、快速找回、定期回顾三种方向。分别用三条样本尝试连续保存、标题/域名/无匹配搜索、手动逐条回顾。记录是否有帮助、是否没解决麻烦、是否证据不足，再选择一个最小改进。现有工具不会自动提醒；这些是建议试验，不是已经执行的结果。

选择一个最便宜的信息来源

假设你经常记得“那篇终端文章”，却忘了完整标题。先选快速找回，因为现有成品已经提供标题和网址搜索，测试它无需新增功能。我们要了解的是：你自然想到的线索，是否出现在已保存的字段里。

沿用参考成品，在教材根目录的操作系统终端运行下列命令。已有服务时继续使用，不要再占用同一端口。

```
# macOS / Linux
python3 -m http.server 4174 --bind 127.0.0.1 --directory cases/A-linkshelf
```

```
# Windows PowerShell
py -3 -m http.server 4174 --bind 127.0.0.1 --directory cases/A-linkshelf
```

浏览器打开 http://localhost:4174/solution/；操作结束，在提供服务的终端按 Ctrl+C 停止。Python 检查与替代命令沿用第 2 章。这次不运行 my-linkshelf，因为你的基础版未必包含搜索，我们需要一个能力已知的试验对象。

不要清除已有收藏。新增三条虚构样本：标题“本章・终端学习”，网址 https://example.com/terminal；标题“本章・研究清单”，网址 https://example.org/intent-04；标题“本章・设计参考”，网址 https://example.net/ui。网址用于辨认记录，不需要访问其中内容。

在“搜索标题或网址…”中先输入“本章・终端”，检查终端样本出现，另两条样本隐藏。再替换为 example.org/intent-04，检查研究样本出现。接着输入

intent04-absent-key，应看到无匹配反馈。最后清空搜索，三条样本都应重新显示，原有收藏也仍在。如果此前保存过相同关键词，总结果数可能增加；判断时先核对本章三条的显示与隐藏。

这四次操作分别检查标题线索、网址线索、没有命中的情况，以及筛选是否只是改变显示。若清空后记录没有恢复，需要先调查行为正确性，不能继续用这份结果判断搜索是否有用。

正确、有用、值得投入，是三次判断

第一层是功能是否正确。编写本章时，实际调用了案例中的 filterLinks，上述三条样本在四次查询下分别返回终端一条、研究一条、零条、全部三条，原数组数量保持三条。这是数据逻辑的执行结果，不等于本章全部浏览器操作已经替每位读者验收。

第二层是它能否帮助你。遮住三条完整标题，凭记忆说出自己会搜索的词，然后实际输入。如果想到的是“终端”，现有标题就可能足够；如果想到“那篇同事推荐的文章”，而标题和网址都没有“同事”一词，搜不到并不能直接判定程序有错。它说明你的线索没有被记录。

第三层才是下一项投入。你可以试着给新收藏写更清楚的标题，再重复一次找回；这比立即设计自动分类系统更小，也能区分“标题习惯需要改善”和“确实缺少备注字段”。一次三条样本只能支持小范围判断，不足以证明几千条收藏下仍然好用。

这三层不要混成一句“搜索通过，所以产品成功”。功能正确由操作支持，使用价值需要你实际使用，继续投资则还要比较成本与当前需求。

哪些事情交给 Codex，哪些需要你回答

读取现有字段、解释过滤方式、提出三组样本，这些是材料足够时可以直接推进的工作。决定“我更在意少输入一次，还是以后更容易找回”，则需要你的真实偏好。Codex 可以帮你把取舍说清楚，无法从文件里推导出你的生活习惯。

因此，不必让它为每个样式和变量名都来询问你；也不要让它默默把个人优先级当成事实。你可以说：“技术细节按现有项目组织；若要增加需要额外账号、付费或公开发布的方案，先说明必要性，我再决定。”这是一项任务范围安排，不代表文字会自动改变工具权限。

如果所有方向都不像你的困扰，补充一段真实情境即可。例如：“我通常在手机看到文章，晚间在电脑整理。”这会暴露当前本机收藏不跨设备同步的限制。下一步应先比较手机收集与电脑整理的替代办法，而不是继续优化电脑上的搜索按钮。

根据试验改变路线，保留有效成果

试完之后，把观察发给 Codex。以下是示意反馈，请用自己的实际结果替换：

```
三条样本的标题和网址筛选都符合预期。
但我自然想到的词是“同事推荐”，现有记录里没有这个信息。
请比较“保存时写更明确的标题”和“新增一项可选备注”两条路线。
说明各自最小尝试和需要观察的结果，先不要改代码。
保留已经可用的新增、删除和本地保存行为。
```

这样的转向没有否定前面的成果。你保留了真实可用的部分，也把“体验不满意”落到具体信息缺口上。如果写清楚标题就已解决问题，这次最好的结果可能是暂时不开发新功能。

动手练习：得到一项更清楚的下一步

用自己的三条虚构收藏重复试验。写下一个自然想到却搜不到的线索，判断它是输入拼错、实现错误，还是信息根本没有记录。再让 Codex 提出两个不同的小尝试，选一个说明为什么现在值得做。

参考反馈（离线展开）

如果线索在标题里但搜不到，先复现与检查实现；若线索从未保存，先比较更明确的标题与增加信息字段。合格成果会说清“这个尝试能区分什么”，并保留实际观察。只写“加标签会更强大”没有完成本章目标，因为它还没有解释标签如何解决你遇到的那次找回。

目标模糊时，可以先组织一次能让目标变清楚的行动。下一章将把选定的方向，收敛成具体可验证的完成状态。

第 5 章：把期望转成可验证的完成状态

你已经决定优先解决“找回收藏”。接着告诉 Codex：“把搜索做好。”它增加一个输入框、显示结果数量，回答任务已完成。你却还想知道：只搜标题还是也搜网址？搜不到时怎么回到全部？搜索之后新增的条目为什么没看见？

这些问题不一定来自实现粗心。你和 Codex 可能在使用不同的“完成”定义。本章将把一个愿望转成几句能执行、能检查的描述；结束时，你应能给一项小功能划定范围，并用观察决定已经完成还是仍需调整。

从使用时刻开始，少用抽象形容词

沿用 LinkShelf。你想找回“记得大概内容、忘了完整标题”的网页。这比“搜索更智能”更接近行动，因为它交代了使用者知道什么、缺少什么。

接着描述动作：“输入标题里的一个词，列表只显示匹配记录。”已经可以测试，但还不完整。我们还需要知道网址是否能作为线索，输入为空时怎样显示，以及没有匹配时怎样恢复。它们都发生在同一条使用路径中，不需要扩成一份巨大需求文档。

可以把愿望改成这样一段话：

> 我想凭标题片段或网址找回收藏。在搜索框输入关键词时，只显示任一字段包含该词的记录；清空搜索后恢复全部。没有匹配时明确提示，并告诉我可以清空或换词。筛选不能删除或改写收藏。

这段话同时给出了正常行为、空输入、无匹配和数据边界。它仍然没有规定变量名和页面布局，因为这些不是当前的核心取舍。

完成状态需要覆盖用户会转弯的地方

仅用一条正常输入，容易得到一种过于宽松的完成状态：“搜索一次有结果，就算结束。”实际使用会转弯：有人清空输入，有人换词，有人在筛选中新增记录。

参考成品目前支持标题与网址搜索。它在保存成功后清空搜索，让新条目可见。这个行为可以直接到 cases/A-linkshelf/solution/app.mjs 中定位，也可以在浏览器检查。不要把它说成你的 my-linkshelf 一定已经具备；自己的版本需要先调查。

现在设想你正在筛选“终端”，却新增了一条标题“设计资料”的收藏。如果搜索仍然保留，新条目可能被条件隐藏。可以选择保存后清空搜索，也可以保留搜索并明确告诉用户“新条目被当前筛选隐藏”。两种方案都有理由；本书参考成品选择前者，便于初学者确认保存结果。

关键是让选择进入完成状态，而不是在验收时临时改变标准。如果你希望另一种行为，可以明确要求后续修改，同时重新检查新增和搜索衔接。只说“新条目保存了”仍不能判断界面反馈是否足够。

将实现办法与结果要求分开

“用 localStorage”是实现约束，“刷新后收藏仍在”是使用结果。两者可以同时需要，但不能互相替代。实现里出现某个 API，不证明相应结果已经成立；结果暂时正确，也不保证满足离线、隐私或维护等约束。

本案例保留原生 HTML、CSS、JavaScript 和浏览器本地保存，是为了让实验不需要账号与运行时后端。搜索本身在浏览器内完成，无需新增在线服务。你可以把这些少量约束写进任务，让 Codex 自行选择函数组织和样式细节。

同样，“不丢数据”过于宽泛。本案例只能承诺约定使用条件内的行为：同一浏览器、相同站点来源下正常保存并刷新；保存失败明确告知，不能假装成功。它不提供跨设备同步，浏览器清理后也不保证恢复。将这类边界说清，会让完成标准更真实，而不是降低质量。

写一份足够短的任务

在自己的工作副本中，可以将下面这段发给 Codex 输入框。已实现某项行为时先检查，再补不足，不必为了完成练习重写已有代码。若会话已在 my-linkshelf 内，将首句路径改成“当前目录”；只有从教材根目录发任务时才用 my-linkshelf 这个相对路径。

```
我想在 my-linkshelf 中凭标题片段或网址找回收藏。
请先检查现有能力，再补齐关键词筛选：任一字段包含关键词即显示，
清空恢复全部，无匹配有提示；筛选不能修改已保存数据。
保存成功后清空搜索并显示新条目。
沿用原生页面与浏览器本地保存，不增加账号、标签或在线服务。
完成后实际检查标题匹配、网址匹配、无匹配、清空恢复及筛选中新建；
保留新增→刷新→删除→刷新的原有行为，并说明未验证项。
```

这份任务不要求它先交十页方案，也没有每一步都停下等待批准。事实足够的实现选择可以继续推进。如果调查发现你说的“已保存数据”格式与代码不符，应先报告差异，再决定怎样兼容；如果要改变数据归属、接入付费或公开上传，则涉及原任务外的重要取舍，需要你决定。

如果你此刻只想验收参考成品，把任务开头改成“只读检查参考成品是否满足以下行为，不修改文件”。同一组完成条件可以用于实施，也可以用于调查，行动权限需要与本次目的相符。

把验收写成动作，不写成赞语

按第 3 章运行对应版本。参考成品使用 http://localhost:4174/solution/，工作副本使用根路径 http://localhost:4174/；不要在一次验收中切换来源。

先新增“验收05・终端笔记” https://example.com/terminal-05 与“验收05・设计资料” https://example.org/design-05。已有收藏保留，只核查自己的样本。接下来依次执行：

1. 搜索“验收05・终端”，确认终端样本显示、设计样本隐藏。

2. 搜索 example.org/design-05，确认设计样本显示。

3. 搜索 accept05-no-match，确认无匹配提示；清空，两个样本重新出现。

4. 搜索“验收05・终端”后，新增“验收05・新收藏”
https://example.net/new-05，确认保存后搜索清空、新条目可见。

5. 刷新，三个样本仍在；删除终端样本，再刷新，只有该样本消失。

这个顺序既检查新功能，也检查它与原有保存、删除的衔接。一次“输入框存在”的截图不够，单独的数据逻辑测试也不覆盖保存后是否清空搜索。你不必每次运行所有可能检查，但当前改动会影响的关键路径应当出现。

完成记录可以很短：“以上五组操作实际通过；使用某浏览器与固定地址；未测试多个标签页同时编辑。”如果第三组失败，就保留具体关键词与可见现象，再要求定位，不要把整体标成已验收。

发现新想法时，怎样保住本轮终点

验收时你也许会想到模糊匹配、收藏夹分组和导入备份。这些想法可以有价值，但不会自动变成本轮必须完成的内容。否则每个成功都会带来一个新终点，任务永远没有可交付状态。

区分方式是问：缺少它，当前承诺的行为还能成立吗？清空后不能恢复全部，会直接破坏本轮行为；没有标签，不影响标题与网址匹配，可以留到下一次。若新发现表明整个方向不对，则回到第 4 章调整目标，清楚说明哪些成果保留，而不是默默把验收条件换掉。

“功能正确”与“足够好用”也应分开。样本搜索通过后，你仍可能认为找回太麻烦。把困扰发生的动作记录下来，形成下一轮的输入。这次完成的证据不需要被否定，下一次改进也不需要被强塞进当前版本。

动手练习：改写两个愿望

先把“错误提示友好一点”改成一段可验收的描述，限定为链接表单。再把“导出更可靠”改成一段描述，只使用参考成品已有的 JSON 导出，不增加尚未实现的导入。

参考反馈（离线展开）

第一项可以写：“空标题保存时指出需要标题，列表不增加；修正标题与网址后能够继续保存；失败时保留输入便于修改。”至少亲自试一次失败和修正，不能只检查提示颜色。

第二项可以写：“有两条测试收藏时发起导出，在下载列表找到文件；打开后是可读取 JSON，条目标题、网址和数量符合当前收藏；无收藏时导出按钮不可用。”你没有因此证明备份能自动恢复，因为本阶段没有导入，也不能承诺文件保存到所有浏览器的固定目录。

完成状态的作用，是让你和 Codex 能对同一个结果作判断。下一章继续练习怎样把这些目标、材料与边界写成恰当的任务说明。

第 6 章：给 Codex 的任务说明怎么写

“我已经写了一大段，为什么它还是做偏了？”任务说明长，不代表重要信息齐全。你可能规定了颜色、技术名词和检查轮数，却没有说明在哪个目录工作、当前哪里有问题、做完怎样判断。

另一种情况是只有一句“把工具弄好”，然后希望 Codex 理解没有说出的偏好。本章用同一个 LinkShelf 搜索任务比较三种写法。完成后，你应能改写一段含糊指令，也能删掉一段过度限制中的无效要求，保留足够的目标、上下文和证据。

第一种：让模型替你猜重要选择

下面的任务过于含糊：

> 帮我把链接工具弄好，搜索要更高级，做得专业一些。

“链接工具”没有定位文件，“弄好”没有指出缺口，“高级”没有说明使用场景。模型可能正确猜中，也可能添加语义搜索、引入在线服务，或只美化输入框。即使你最后觉得不满意，也难以指出它没有满足哪一条约定。

改进并不需要一开始就写完整规格。先补足你真实知道的部分：“工作副本是 my-linkshelf；我想用记得的一段标题或网址找回收藏；还没有确认当前是否支持。”未知可以明确保留，让调查成为第一步。假装知道现状，反而容易让它去修一个不存在的问题。

这里应当把第 4 章与实施分开。如果连想解决哪种麻烦都未确定，就先让它比较方向。已经确定要标题与网址搜索，才进入本章的实施说明。

第二种：把可选择的办法全部锁住

另一种写法看起来很严谨：

```
你是顶级工程师。无论项目现状如何，先写20页架构文档，
建立七份检查表，执行五轮审查，每改一行都停下来让我批准。
必须按我指定的十个函数名称重写所有文件，再实现搜索。
不允许提出替代路线，必须一次成功。
```

它增加了活动，却没有增加可检查的结果。对于一个已有原生页面的小改动，先重写全部文件会扩大回归范围；规定审查轮数也没有告诉审查应寻找什么。逐行批准会让人把注意力耗在局部，却仍然可能漏掉“刷新后条目丢失”。

有价值的过程约束应当对应具体原因。例如，“沿用现有原生页面，不新增运行时后端”，直接服务于本机静态工具的条件；“保留原有收藏，不清空整个 localStorage”，直接保护已有数据。“必须一次成功”则无法在不确定任务里提供行动方法。更有用的要求是失败后保留复现步骤与错误，改变没有证据支持的假设。

你不必把方案选择全部交出去，也不必逐条控制。把真正影响任务结果的条件留下，其余让 Codex 根据文件和反馈组织。

第三种：给出完成状态与行动空间

这是同一任务的一种恰当写法。以下为可复制的教学任务说明，不是已经执行过的模型对话：

```
我的工作副本在 my-linkshelf。新增、刷新保存和删除已按第3章检查；
我想凭标题片段或网址找回记录，尚未确认本版本有无搜索。
请先查看现有文件与行为，再补齐不足，不复制参考成品覆盖我的目录。
输入关键词只显示匹配标题或网址的记录，清空恢复全部，无匹配有提示；
保存成功后清空搜索，让新条目可见；筛选不应修改保存的数据。
沿用原生 HTML/CSS/JavaScript 与本地存储，不增加账号或在线服务。
技术组织按现有项目选择；发现影响范围的重要取舍时说明依据。
完成后实际检查搜索与保存的衔接，再检查新增→刷新→删除→刷新。
请区分实际执行的检查与需要我在浏览器完成的部分。
```

发送之前先修改第一句。只有你确实完成第 3 章验收，才能写“已检查”；尚未检查，就改成“据当前代码实现了这些行为，请先验证”。引用不真实的前提，即使

句式很好，也会让后面的工作建立在错误基础上。

如果已经在 my-linkshelf 中打开 Codex，路径可写成“当前目录”；如果会话位于教材根目录，应保留完整相对路径。路径不是装饰信息，它决定实际修改对象。需要启动程序时，继续采用第 3 章的同一服务命令与浏览器来源。

这份说明为什么够用

开头的现状告诉 Codex 有什么可保留，未知告诉它哪里需要先调查。中间的使用行为提供终点，范围约束避免引入本轮没有必要的服务。末尾的验证要求让“完成”落到实际检查，而不是一段漂亮总结。

它没有规定每个函数、工具调用顺序或固定审查次数，因为你真正关心的是搜索怎样与收藏行为衔接。若现有代码已经满足要求，正确结果可能是一份检查记录和零项修改；不能为了显得做了事而强制重写。

这也不是每次都需要填写的模板。一个标题错字，只需说清位置和改法。一个未知的刷新故障，需要更多复现条件与观察。任务复杂度决定说明深度，固定字数不会自动提升效果。

下方练习可比较三种写法，并尝试改写。组件里的反馈是教学规则在本地给出的判断，不会把你的文字发送给 Codex，也不能代替一次真实任务的结果。

> 离线版说明：以下为网页互动的文字替代。

离线练习：改写一次任务说明

比较“帮我弄得更高级”“先写二十页架构再换框架”和“只在保存成功后清空，失败时保留输入，并检查连续保存与刷新”。用自己的实际观察重写第三项，检查是否交代行为、现状、边界和结果。写在自己的笔记中；电子书不提供本地草稿存储、复制按钮或自动评分。

让问题发生在真正需要选择的地方

好的任务说明会减少无意义的来回，但不会消除所有询问。如果你没有指定按钮的圆角，Codex 可以沿现有风格继续。如果两种方案分别意味着“只在电脑本地保存”和“上传到外部账号”，它们改变数据去向与使用前提，就需要把取舍摆出来。

还有一些未知可以通过检查解决，例如项目入口、现有存储键、是否已有搜索。此时让它先读文件，比问你“请告诉我全部架构”更有效。只有你知道的个人偏好、未提供的材料或必要授权，才值得向你索取。

你可以明确说：“材料里能查明的问题请先调查；需要改变本轮范围、增加费用或公开发布时，先给具体方案与影响。”这表达协作偏好，不会自动绕过宿主权限、组织策略或外部服务授权。出现权限拒绝时，应看被拒绝的真实动作，再决定替代路线或必要授权。

在工作途中，用新事实更新说明

任务说明不是一次写完就不能动的合同。假设 Codex 调查发现已有搜索，但保存新条目时仍被旧筛选隐藏。此时不必重发整段任务，补充：“搜索已存在，本轮只处理保存后的显示衔接，保留匹配规则。”

如果结果与你预期不同，优先反馈操作与现象。“颜色不是我要的”和“新增看不见”是不同问题；“新增后输入框清空，但列表没有出现，清空搜索后才看到”能显著缩小调查范围。直接要求“再专业一点”会把已获得的证据重新模糊掉。

修改范围也要随事实收敛。若只有一处衔接缺失，不必让它再完成一遍所有功能；若发现存储无法读取，应先处理读取问题，避免在未知数据状态上继续叠加搜索。

动手练习一：给含糊任务补关键信息

把“导出不好用，帮我搞定”改成一段最多六句话的说明。使用参考成品已存在的 JSON 导出，自己选择一个具体困扰：找不到文件，或下载内容与当前收藏不符。明确哪项是你真的观察到的，哪项还需要检查。

参考反馈（离线展开）

示例可写：“我在参考成品中点击‘导出 JSON’，没有在浏览器下载列表找到文件。请先检查下载是否被阻止与导出事件是否触发，不要先增加导入功能。保留现有收藏，检查两条测试记录导出后能否被 JSON 读取，标题和网址是否一致。没有浏览器工具时给我手工检查步骤，报告未验证部分。”其中“没有找到文件”必须换成你的真实观察；不能仅为套句式捏造故障。

动手练习二：从过度限制中留下必要约束

改写下面的教学情境：“删除后刷新会回来；请重写所有文件、加延时重绘、连续审查五次，每改一行都问我。”这个故障是构造的问题报告，不是在宣称当前参考成品存在该缺陷。

参考反馈（离线展开）

可以改为：“删除后当前页面消失，刷新却恢复。请先复现，查明删除后的写入与刷新读取路径，用证据区分原因，再做最小修复。保留其他收藏与新增行为；检查删除→刷新不恢复，并确认另一条记录仍在。不要先假定延时能解决。”它保留数据边界与验证要求，删掉没有依据的重写和次数要求，也没有把猜测预设为修复方法。

完成这两次改写，你应当更容易看出一份任务说明缺什么、多什么。接下来面对陌生项目时，就可以先让 Codex 用实际文件补全现状，再决定如何推进。当前已开放的阅读入口见课程目录。

PART 03 / 上下文与现状调查

第 7 章：接手一份你不熟悉的代码

“帮我给这个工具加一个筛选。”如果这是你昨天写的项目，你可能知道从哪里动手；如果它来自别人，第一件难事往往是判断：到底运行哪个文件，改动会碰到什么，已有测试能证明多少？此时急着写代码，可能把原有能力重新做一遍，却漏掉真正的入口。

本章使用新的案例 TaskTrail，一个离线的 Python 任务清单。你不需要先学会 Python。完成时，你应拿到一份简短事实简报：入口在哪里、数据从哪里读取、怎样运行测试、目前确认了什么，以及增加状态筛选之前还缺哪项信息。本章暂不实施筛选。

给陌生项目一块独立的工作位置

沿用第 2 章的 Codex 与终端准备，另需 Python 3.9 或更新版本。程序只用标准库，即 Python 自带模块，不需要安装第三方包。它把任务保存为 JSON，一种用字段与列表组织数据的文本格式，你可以用普通编辑器查看。案例起点在教材根目录的 cases/B-tasktrail/starter/。先不要打开 solution/；那是改完以后的参考成品，会干扰你判断起点到底具备什么。

在文件管理器确认教材根目录中尚无 my-tasktrail；已有同名目录时选新名称，并替换后文路径。

macOS / Linux，在教材根目录的操作系统终端：

```
cp -R cases/B-tasktrail/starter my-tasktrail
cd my-tasktrail
python3 --version
codex --sandbox read-only
```

Windows PowerShell，在教材根目录：

```
Copy-Item -Recurse cases/B-tasktrail/starter my-tasktrail
Set-Location my-tasktrail
py -3 --version
codex --sandbox read-only
```

Windows 没有 py、但 python --version 返回 Python 3 时，后面的 py -3 可换成 python。若上一章采用 codex.cmd 解决 PowerShell 脚本策略问题，继续使用同一入口。复制的是起点文件，不能借此覆盖自己以前的任务清单。

先要一份能核对的简报

下面发给 Codex 输入框：

```
我接手了当前目录的 TaskTrail，准备给列表增加按状态筛选。
先只读调查，不修改源码、样本或说明，不安装依赖。
不要读取教材中的 solution 参考成品。
请从目录与 README 开始，定位：
1. 实际命令入口，以及列表、添加、完成各走到哪里；
2. 数据文件怎样选定，相对路径从哪里计算；
3. 测试怎样运行，会不会写入项目文件。
用文件路径支持结论，区分代码事实、运行证据与未确认项。
若一项检查会写入文件，先给出命令与影响，暂不执行。
```

“先调查”不是要求它画一张庞大架构图，而是暂时把学习目标从改功能换成找事实。入口、数据与测试这三条线，足以支撑这个小项目的下一步。工具若因权限拒绝而没能读取，也要保留这个区别：没有读到，不等于文件不存在。

收到简报后，挑一条最有影响的结论亲自检查。例如它说“任务保存在 JSON 中”，这仍太宽。你需要知道本次命令会使用哪一个 JSON、缺少文件时会怎样、运行命令的位置是否影响解析。让它引用处理参数与读取文件的代码，而不是依据 README 的一句话补全整个实现。

用运行结果补上第二层证据

打开另一个终端，进入你刚复制的 my-tasktrail 目录。这次由你运行命令，不把命令粘到 Codex 的自然语言输入框：

```
# macOS / Linux；Windows 把 python3 换成 py -3
python3 -B tasktrail.py --help
python3 -B tasktrail.py --data sample.json list
python3 -B -m unittest discover -s tests -v
```

-B 禁止 Python 生成字节码缓存，便于保持项目目录清楚；它不禁止程序保存任务。最后一条运行 tests 目录中的单元测试。测试会在系统临时目录创建、清理自己的数据，因而不是“全电脑绝不写文件”，但不会用样本文件试写任务。若你的执行环境禁止临时写入，记录这个限制，不要把权限失败解释为业务功能失败。

原始样本的预期列表包含 [ ] #1 核对入口命令 和 [x] #2 阅读项目说明。方括号表示状态，编号用于定位任务。四项测试应通过；如果你已经改过副本，结果以当前文件为准，不强行套用起点记录。制作时实际执行的环境与命令保存在案例 B 运行说明，这是参考项目的证据，不会自动变成你电脑上的通过记录。

尤其要核对 --data。它是必须显式提供的参数，放在 list、add、done 等子命令之前；sample.json 相对于终端当前目录，而不是相对于脚本所在位置。因此，从另一个目录调用脚本时，同样的短文件名可能指向另一份数据。先固定工作目录，比遇到空列表后立即修改读取函数更有效。

让简报保留不知道的部分

现在请 Codex根据你的真实输出补充调查结论。可以发：“我在副本目录运行了以上命令，得到以下输出。请判断它们增加了哪些证据，不修改程序。”只粘贴与问题有关的结果，不需要附上整个终端历史。

一份够用的简报可以有五行：入口；数据路径规则；核心文件职责；实际检查；下一项未知。比如“测试通过”应跟着具体命令与范围，“还没实现状态筛选”应由

当前帮助与代码支持。不要让它把参考成品有的功能写到起点上。README 不完整，也不意味着说明在撒谎；程序、测试与说明各自提供一部分信息。

原有测试通过，只能说明它覆盖的已有行为符合预期。它没有预先证明未来的筛选正确，更不能证明对任意数据都安全。你可以要求它指出“哪一个现有测试对本次修改最有约束”，再说明还要增加什么断言。断言就是测试中对结果作出的具体要求，例如只显示未完成，而已完成的编号不应出现。

遇到陌生报错，先判断发生在哪一层

如果终端说找不到 tasktrail.py，先检查是否进入副本目录；如果参数解析提示缺少 --data，先补齐调用方式；如果提示数据格式异常，先保留并查看指定文件，不要删除所有 JSON。三种失败分别来自入口、参数与数据，不能用“程序坏了”一概代替。

还有一种偏差来自 Codex：它读完 README 就开始重构。此时明确纠正：“当前终点是事实简报。先停止修改，引用入口和数据路径；实施留到下一章。”若已经改动，先检查实际差异，再只撤回此次误改。用新副本练习的好处，是调查对象足够清楚，不会与真实项目的旧改动混在一起。

动手练习：发现一条说明没有写透的边界

在不改实现的前提下，请 Codex从测试或代码中找一条 README 没有详细说明的输入边界。让它指出：什么输入触发、程序怎样反馈、原数据会不会改变。随后用系统临时目录或副本中的新数据文件验证，不要对 sample.json 做写入实验。执行前先确认会创建什么文件；需要写入时，由你在终端运行，或在明确授权的工作区写入会话中运行。

参考反馈（离线展开）

可以调查空白标题、找不到的编号、损坏数据等情形。合格结果不仅说“有报错”，还会检查非零退出码与数据保持原样。不要先看参考答案再假装是独立调查；本题的价值在于找到证据的过程。

带着这份简报进入第 8 章。你已经有了改变陌生项目的起点：先知道实际对象，再选择有用的信息与改动。

第 8 章：提供信息，不等于堆满信息

你有一个文件夹、一份长说明、几十条旧聊天。为了让 Codex“完全理解”，你把它们全部交过去。它读了很多，却依据一段旧想法给任务工具加了云同步。信息并不少，真正缺的是：哪些材料约束当前任务，哪些只是过去的背景？

本章沿用第 7 章的 my-tasktrail。你将把调查结果收敛为一条具体任务，增加列表状态筛选，并检查原有行为仍然成立。终点不是整理一份漂亮的资料包，而是让提供的每项信息都服务于这次可验证的改动。

先把这次任务与项目愿景分开

当前起点已有添加、完成、列表与 JSON 保存。现在需要的是：list --status open 只看未完成，done 只看已完成，all 看全部；省略参数时保持原来的全部列表。这个描述同时给了新能力与兼容要求。它没有要求标签、账号或界面，也不要求把数据重新组织一遍。

这里的“上下文”可以理解为 Codex 当前能够用来判断的信息，包括你的话、读到的文件和工具反馈。它能看到，不代表应该把每一条都当作当前需求。案例的 notes/ideas.md 保留了旧讨论，并明确说那些想法没有实现。这个文件有助于解释过去，但不能代替你刚刚给出的任务。

先在自己的笔记中写一句话：“这次只改变列表显示的选择，数据格式与添加、完成保持兼容。”如果无法写清这句话，先澄清目标；如果已经明确，就不必再要求Codex遍历所有资料来证明它认真。

给它一个有方向的入口

关闭上一章的只读会话，在操作系统终端确认当前目录为 my-tasktrail，再启动能实施的会话：

```
codex --sandbox workspace-write
```

Windows 使用与第 2 章一致的 codex 或 codex.cmd。App 与 IDE 用户打开同一个工作副本，确认允许当前任务的文件修改。不要在只读会话中反复要求“直接写”，也不要为一个本地改动扩大到与任务无关的权限。

把下面的任务发给 Codex 输入框。其中的调查结论请按自己的实际简报替换，不复制未经确认的推测：

```
请在当前 TaskTrail 副本增加列表状态筛选。
目标：list --status open|done|all；默认 all，与现有列表兼容。
材料：先看 tasktrail.py、tasks.py 和 tests/test_cli.py；
README 用于确认启动方式，notes/ideas.md 的旧设想不是本次需求。
保留 --data 规则、JSON 格式、添加与完成行为，不改 sample.json。
请先指出预计改动位置，再实施并补上必要断言。
运行现有测试，并用一份同时包含两种状态的数据检查筛选。
筛选不应写入数据文件；错误状态值应给清楚反馈且返回非零。
不安装依赖。测试可以使用系统临时目录，交付说明实际命令与结果。
```

这不是一种必须背诵的提示词模板。给出文件入口，是因为上一章已查到它们与问题有关；说明旧设想不是需求，是因为本项目确实保留了容易混淆的背景。换一个项目，就应该换成那个项目真正需要的材料。只说“全面理解以后再做”，反而没有帮助它判断阅读何时足够。

怎样判断信息是否值得加入

可以用一个具体问题筛选材料：“它会改变这次实现或验收吗？”参数解析代码会影响新增选项放在哪里；数据结构会决定怎样区分完成状态；现有测试会影响兼容检查；一年前对网页配色的讨论通常不会。排除它，是为当前判断减少噪声，不是宣布它永远无用。

同样要保留限制与失败信息。例如上一章发现数据格式损坏时程序拒绝写入，这个边界值得告诉 Codex，因为筛选不应顺手变成“读取失败就重建空文件”。相反，反复粘贴同一份成功输出并不会增加新证据。可以用一句结论加命令位置，让它必要时直接复查原文件。

如果某个关键问题仍未确认，就直接写未知。“尚未确认空列表输出怎样处理”比“应该和其它工具一样”更有价值。前者引出一次针对性读取或运行，后者把习惯当成了事实。上下文选择不是只给好消息，而是把会改变决定的事实与未知都留下。

让验收能区分正确与看起来正确

完成修改后，在 my-tasktrail 的操作系统终端运行：

```
# macOS / Linux；Windows 把 python3 换成 py -3
python3 -B tasktrail.py --data sample.json list --status open
python3 -B tasktrail.py --data sample.json list --status done
python3 -B tasktrail.py --data sample.json list --status all
python3 -B tasktrail.py --data sample.json list
python3 -B -m unittest discover -s tests -v
```

使用未经写入修改的起点样本时，第一条应只有编号 1，第二条只有编号 2；第三、四条应都显示两条。这些是预期结果，你的实现是否达到它们，要看真实输出。只用全是未完成的数据检查，会漏掉“无论传什么都显示全部”的假筛选。

再运行一个错误参数：

```
python3 -B tasktrail.py --data sample.json list --status unknown
```

它应拒绝该值。macOS / Linux 紧接着运行 echo $?，PowerShell 运行 $LASTEXITCODE，可查看上一程序的退出码；非零表示这次调用未正常完成。不要在查看退出码以前先运行另一个命令，否则看的可能是别的程序结果。参考成品使用退出码 2，你自己的实现至少应提供明确、稳定的失败结果。

还有一个界面上看不见的要求：筛选不应修改 JSON。请让 Codex在自动测试中对比筛选前后的文件内容，同时保留添加与完成的原检查。列表正确而文件中的已完成任务被删除，仍然是严重错误；它把“暂时不显示”误做成了“从数据中移除”。

当它读得太少或太多

若 Codex只改帮助文字，没有找到列表数据来源，补充相关函数或要求它沿实际调用路径读取。若它不停读旧笔记、准备设计账户系统，指出当前任务与验收，要求它回到参数解析、过滤和输出。两种纠偏都要指向缺少或多余的信息，不必升级成几十条永久规则。

遇到 unrecognized arguments，先检查你运行的是修改后的副本，且 --status 放在 list 后；也检查是否误运行了教材 starter。起点不认识这个新参数是正常现象，不能把参考成品的预期倒过来套给未修改代码。

若测试通过但手工筛选不对，保留两份证据再比较覆盖范围。测试可能只检查旧功能，也可能用了不区分状态的数据。让 Codex补上能够失败的断言，比把“测试通过”当作停止讨论的理由更有帮助。

动手练习：删除一项没有用的上下文

把自己刚才发出的任务再读一遍。选出一项可以删去、却不会改变当前实现判断的材料，再指出一项不能删的信息。随后用两句话解释：缺少后者，会让哪一个具体验收变得模糊？

参考反馈（离线展开）

旧的云同步设想通常可以排除；默认仍显示全部、筛选不改数据、两种状态样本这些信息直接约束结果。答案不取决于字数少，而取决于你能否说出材料与决定的关系。

现在保留自己的实现与真实检查结果，不急着把整段聊天存成新规范。进入第 9 章，把需要长期记住的约定和只属于当前阶段的状态分别留下。

第 9 章：项目约定与会话接续

昨天你和 Codex完成了状态筛选。今天换一个会话，它又问“怎么运行”，甚至准备重写已经可用的列表。旧聊天中不是没有答案，但答案散在多次讨论、失败尝试与过时计划里。接续的困难，不只是记忆不够，还包括不知道哪些状态仍然有效。

本章沿用第 8 章的 my-tasktrail。你将留下两份短文件，再用一个新会话验证它能否准确恢复工作。完成状态是：新会话知道当前目录、已有能力、真实检查、尚未完成的事项，并能提出合适的下一步，而不是从头重复实施。

稳定约定与当前状态各写什么

有些信息在许多任务中都成立，例如不引入第三方依赖、测试命令、样本不要当作写入练习。这类信息适合写进项目指引。另一些信息很快会变，例如筛选是否已实现、哪项测试刚失败、下一次先检查什么。这类信息适合放在接续记录。

Codex 支持 AGENTS.md 作为项目指引。2026-10-25 打开核对的官方说明指出：CLI 启动时构建指引链，项目发现通常从项目根目录走到当前工作目录，较深层内容在组合时覆盖前面的相应指引；同层存在 AGENTS.override.md 时会优先选择它。文件改后应重新启动核对，不能假定正在运行的会话自动重新加载。这里先用一个短文件，不需要为了练习去改变全局配置。

本书建议再用 progress.md 保存阶段状态。**这个名字和下面的格式是教材约定，不是 Codex 必须存在的特殊文件。**需要通过项目指引或当前任务明确要求读取。文件不会替你运行测试，也不会让一项失效结论永远正确；接续时仍要与代码和实际输出交叉检查。



区分当前任务上下文、稳定项目约定与阶段状态记录：新会话仍需核对实际文件和证据

给自己的副本补一份短指引

先确认当前打开的是自己的 my-tasktrail，不是教材根目录，也不是 cases/B-tasktrail/starter。用编辑器创建 AGENTS.md，或把下列内容发给当前有写入权限的 Codex，要求只在副本内保存：

```
# TaskTrail 工作约定

- 离线 Python 标准库 CLI，不增加第三方依赖或在线服务。
- 入口 tasktrail.py；--data 指定数据路径，不把 sample.json 用作写入练习。
- 修改后运行 python3 -B -m unittest discover -s tests -v；Windows 用 py -3。
- 自动测试使用系统临时目录；手工新增任务使用自己的新数据文件。
- 继续前读 progress.md，并核对实际代码；交付写明命令、结果和未验证部分。
```

这些内容没有要求“所有任务先写完整规划”，因为小改动不需要那套流程。它也没有列出当前实现的全部函数，函数可能很快变化，重复的清单容易过时。短指引应保留能够减少反复纠正的事实，而不是把刚才每一句聊天都升级为永久规则。写下指引也不授予额外文件权限，实际可读写范围仍以当前环境为准。

如果工作副本位于教材 Git 仓库内，上层也可能有项目指引；启动位置与层级会影响读到的内容。让 Codex说明它实际加载或读取了哪些来源，再确认当前任务范围是这个副本。不要修改教材根目录的约定来“修复”一次学习任务。

把状态写到能够继续的位置

接着在同一个副本中创建 progress.md。下面是格式示例，不是你的运行日志。把状态和检查替换为真实结果；还没跑过就保留未验证：

```
# TaskTrail 当前状态

目标：列表增加 --status open|done|all，默认仍显示全部。
位置：当前 my-tasktrail；入口 tasktrail.py，读写逻辑 tasks.py。
已做：按实际情况填写；不要把参考成品能力当作自己的成果。
检查：写明目录、Python 版本、实际命令、退出码与关键结果。
限制：没有多用户同步；筛选不能改数据格式或丢掉任务。
```

```
待做：写一项最有价值的下一步；未确认的行为直接写未确认。
```

一条真实记录可能是“在副本运行 list --status open，样本只显示编号 1；尚未检查数据字节是否保持不变”。它比“筛选完成、测试没问题”更有接续价值，因为新会话知道下一步缺什么证据。若你使用了教材参考成品，也写明来源，不能把复制当作自己的独立实现。

实现后的状态应与文件相符。假如记录说“已支持 --count”，帮助和代码却没有这个选项，接续时应把它标为冲突并调查，不能为了配合记录再编一段运行结果。代码、说明、聊天都可能过时，接续文件只是一个较短的入口。

新会话先恢复，再决定行动

保存文件后，结束当前会话。在操作系统终端进入 my-tasktrail，运行：

```
codex --sandbox read-only
```

Windows 继续用自己已确认的命令入口；App 或 IDE 用户在同一副本开启新聊天。首次恢复先用只读范围，目的是判断状态，不立即执行下一功能。向 Codex 输入框发送：

```
请先读取当前目录的 AGENTS.md 与 progress.md，再查看必要代码。
本轮先不修改文件，不读取教材的 solution。
请用五句话恢复状态：目标、已有实现、实际验证、未验证项、下一步。
指出记录与文件之间是否有冲突，并引用支持关键结论的路径。
若准备继续添加 list --count，请只说明它的最小改动和验收，先不实施。
```

--count 在这里是下一项候选功能，当前参考成品没有实现它。这个具体但尚未开始的任务，能帮助你检查接续是否有效：它应沿已有列表与筛选关系考虑计数，而不是另造数据格式。若你只想恢复状态，不做后续功能，删去最后一句即可。

新会话引用了 progress.md 不等于恢复正确。检查它是否找到实际目录，是否误说筛选已经通过所有检查，是否把待做当作已做。你可以选择一条关键结论，让它引

用相应实现或测试。若缺少上下文，补那一块；不用把前一会话数十屏内容全贴回来。

约定没有生效时怎样查

如果新会话完全没提指引，先确认文件确实叫 AGENTS.md，保存在当前副本；再查实际启动目录与是否存在更优先的同层 override。官方说明也提到指引存在读取大小限制。这个案例的短文件不需要调整限制，先排除路径与文件名，比扩大配置更直接。

若是当前旧会话没有自动遵循新文件，明确要求它读取并总结，再在新会话重新核对。不要只凭保存成功就宣布“自动加载验证通过”。本书制作已核对官方发现规则，并执行 TaskTrail 的 CLI 与单元测试；没有替你实际完成跨会话协作，这项能力要通过下面的练习观察。

如果指引越写越长，先问哪条规则对应真实反复发生的问题。将“这次筛选还有一个测试没跑”移到状态记录；删除已经失效的约束；保留启动、数据与验证这类仍然有用的内容。接续的目标是减少重复调查，不是让每个小任务背负全部项目历史。

动手练习：只靠仓库留下的入口继续

关掉旧聊天视图，只给新会话工作副本与上面的恢复任务。对照它的五句话，亲自确认一条已实现行为和一条尚未验证的边界。最后在 progress.md 补上一行：“本次接续实际发现了什么，下一项检查是什么。”若没有启动新会话，写“未进行接续验证”，不要勾选为已完成。

参考反馈（离线展开）

有效接续会用当前文件修正记录，并给出一个具体动作，例如先补筛选不写数据的检查。只是复述旧记录、承诺“继续努力”，没有恢复到可以行动的状态。发现遗漏并更新一行事实，也是一种成功接续。

你留下的不是聊天备份，而是下一次行动的入口。遇到更大的项目，同样先保留目标、事实、证据与缺口，再让新会话从真实状态继续。

PART 04 / 决定怎样推进

第 10 章：什么时候直接做，什么时候先研究

你给 Codex一个小改动，它先写了三页计划；换一个陌生项目，它却立刻改代码。两次看起来都很主动，问题却相反：前者没有值得先解决的不确定性，后者还没找到要改变的对象。推进路线的价值，在于消除当前真正妨碍行动的条件。

本章沿用已经认识的 LinkShelf 与 TaskTrail，并引入下一章的离线数据案例。你会为四种处境选择下一步，并完成一次小行动。终点不是记住“复杂任务先规划”，而是能解释：为什么此刻这条路线能够增加信息或交付成果，以及什么观察会让你换路线。

先看已知与未知各是什么

“我不太懂编程”和“这个任务存在关键未知”不是一回事。你可能不认识每一行 CSS，但要改的文字、文件和验收动作都很明确；也可能懂 Python，却不知道一份 CSV 的重复行代表重复导出，还是两次真实活动。前一种通常可以小步修改，后一种要先确认含义。

先问三个具体问题：目标的行为是否明确？实际修改对象是否已找到？剩下的未知会不会改变实现或验收？如果都明确，且改动小、容易检查，就让 Codex直接推进。某个未知会改变结论时，把下一步对准它，而不是用一份很长的计划遮住它。

例如，LinkShelf 空列表的说明文字需要改成一句更清楚的话。你已确认这是自己的工作副本，知道何时出现空状态，也能打开页面检查。这里没有必要先调查数据库或比较三个前端框架。让它找到对应文字、局部修改，再看空态与非空态，已经能形成有效闭环。

四种路线，四种不同的产物

可以把当前任务放到下面这张表中。它是一种判断辅助，不是每次都要填的表格。

当前处境	有价值的下一步	下一步应留下什么
目标、位置、检查都清楚	直接修改	可观察的小改动与实际检查
不知道入口、数据或现有行为	先调查	能指向文件的事实与未确认项
某个假设决定路线，但还没证据	做最小实验	具有区分力的输入、结果与判断
目标较大，成果可逐段独立使用	分阶段推进	每阶段可运行、可验收的结果

TaskTrail 适合说明调查路线。第一次接手时，你还不知道 --data 从哪里解析，直接加筛选可能改错文件；第 7 章的事实简报补上这些信息。调查结束后就应转入实施，不必每天重新写同一份项目概览。

CSV 案例适合说明实验路线。你知道要做摘要，但空值应算零还是排除、重复应算几次，会改变总数。先用四条能手算的记录验证规则，比立刻生成长报告更快暴露差异。数据含义仍需人确认；程序跑通不能代替这个决定。

LinkShelf 从收藏、检索到备份，则适合分阶段。每一段可以有独立结果，读者不用等到标签、导入、云同步都做完才能保存第一条链接。第 12 章会把这样的路线写成真正可检查的里程碑。

下面把目标模糊、统计口径未知、边界明确和补丁无效四种情境放在一起比较。选择下一步后阅读理由，再用重置重新作答。这是浏览器内的教学判断练习，不调用 Codex，也不会对真实项目自动诊断。

离线版说明：以下为网页互动的文字替代。

离线练习：给四种情境选择路线

1. 需求仍模糊：先探索，比较“快速收藏、找回、回顾”三个方向。示例任务：“请用三条样本帮我比较使用场景，再选一个最小尝试。”

2. CSV 口径未知：先用四行实验；它能区分空值和重复的处理。示例任务：“先约定口径，用可手算输入比较结果，不扩大到完整报告。”

3. TaskTrail 边界明确：直接实施并验证。示例任务：“增加 list --status open|done|all，保留原有数据和默认列表，用两种状态样本检查。”

4. 删除修补反复失败：先查证假设。示例任务：“停止增加重绘与延时，沿删除、保存、读取追踪，先选一个区分原因的检查。”

在纸上先选再解释理由。这些是教学情境，不是实际 Codex 会话或自动评分。

在一次小任务中使用判断

如果你已完成第 3 章，可在自己的 my-linkshelf 副本做下面的练习；不要修改教材参考成品。将此文字发给拥有该副本写入权限的 Codex 输入框：

```
请把空列表的说明改为：
“粘贴一个网页地址，留下你的第一条学习线索。”
先确认当前目录与实际空状态入口，再做局部修改。
不改保存、读取、搜索和删除逻辑，不增加依赖。
修改后说明位置；可执行的检查请执行，
浏览器中空列表出现、添加后隐藏、删空后再出现由实际操作确认。
```

这个任务仍有一次定位，但没有要求完整研究。定位服务于具体修改，范围足够小时可以在同一过程中完成。若你的页面根本没有空状态，则关键条件发生了变化：先说明现有行为与最小新增范围，不要假装只替换了一句话。

在第二个操作系统终端进入教材根目录，启动自己的副本：

```
# macOS / Linux；Windows 将 python3 换成 py -3
python3 -m http.server 4174 --bind 127.0.0.1 --directory my-linkshelf
```

先停止占用 4174 的旧服务，浏览器打开 http://localhost:4174/。只删除本次练习新增的样本，不清空已有个人收藏；已有数据时可使用另一新副本和专用存储键。终端按 Ctrl+C 停止服务。参考成品与自行生成页面的结构可以不同，验收按行为判断，不能只检查字符串是否进入源码。

路线可以在事实改变后切换

假如你让 Codex直接改空状态，结果发现同一段文字由三个入口生成，且当前状态判断有冲突。此时转为短调查是合理的，不必为了遵守原先“直接做”而盲改。反过来，若已找到唯一入口，继续研究界面体系就没有增加价值，应回到局部实施。

更大的任务也可以先直接做一小段。阶段计划并不意味着禁止行动；它要帮助确定现在交付什么。实验也不是拖延实施的别名：必须明确什么结果支持继续，什么结果需要换办法。只有“先试试看”而没有判断条件，很容易演变成无止境尝试。

可以这样介入：“现在已经确认入口和规则，请结束调查，完成这一条使用路径。”也可以反过来说：“这个未知会改变统计口径，先暂停完整报告，用四行数据比较结果。”两句话都解释了为什么改变路线，而没有简单用“快一点”或“谨慎一点”代替判断。

动手练习：给四个请求选下一步

为下面每项选一种优先路线，并说出下一步要拿到的证据：

1. 自己的链接工具中，一句已定位的提示语需要改写，原功能正常。

2. 一个陌生 Python 工具显示空列表，你还不知道它读取哪个文件。

3. CSV 中出现重复记录，直接相加会改变总数，但记录含义未确认。

4. 希望链接工具同时具有检索、标签、导出与导入，当前只有基础收藏。

参考反馈（离线展开）

第一项通常直接做并看实际空态；第二项先调查调用位置和数据路径；第三项先确认含义，再用小输入验证约定，不能仅凭程序决定业务口径；第四项分阶段，先交付一段可用结果。若你有额外事实，也可以选其它路线，但必须解释它怎样减少当前未知或带来可验收成果。

把自己的选择保存成四句话即可。下一章进入最小实验，把“先试一下”变成可以得出决定的具体动作。

第 11 章：用最小实验处理关键不确定性

一份报告写得很流畅，总访问数却算进了重复行。继续让 Codex“优化分析”，可能只是让错误多出几段解释。最值得先做的，有时不是完整报告，而是一个小到能够手算、又能区分两种处理方式的输入。

本章用案例 C：CSV Insight。你会明确一个假设，用四条记录做实验，根据结果决定是否扩大输入。最后应有一份实际生成的小报告、一个独立核对结果，以及继续或调整的理由。这里运行的是离线 Python 程序，不是网页自动调用 Codex。

让实验只回答一个关键问题

CSV 是按行保存表格的文本格式。这个案例使用 date,channel,visits,signups 四列，分别表示日期、渠道、访问数与注册数。数据是人工样本，不能写成真实业务表现。本次先问：“按明确约定排除空值与完全重复后，总数能否与手算一致，并说明哪些行被排除？”

约定是：任一必需字段为空，就排除整行并说明；四字段去首尾空格、将数值解析为整数后完全相同，只计第一条。日期相同、渠道相同而数值不同，仍保留。它是一种已选口径，不是所有数据都应遵守的规则。真实工作中若重复行的含义未知，要先问数据提供者，不能用“常见做法”悄悄替他决定。

实验需要同时包含能触发分歧的输入。只有正常数字，程序不管怎样处理空值都可能通过；只有一条记录，也看不出是否去重。四条数据就足以让这两个假设暴露差别，不必先制作几万行数据或一个仪表盘。

把案例复制到自己的练习位置

需要 Python 3.9 或更新版本，无第三方依赖。先确认教材根目录中没有 my-csv-insight，再执行：

```
# macOS / Linux；在教材根目录
cp -R cases/C-csv-insight my-csv-insight
cd my-csv-insight
python3 --version
```

```
# Windows PowerShell；在教材根目录
Copy-Item -Recurse cases/C-csv-insight my-csv-insight
Set-Location my-csv-insight
py -3 --version
```

Windows 没有 py、但 python --version 返回 Python 3 时，可用 python。已有同名目录就换一个新名称，不覆盖旧报告。案例提供可运行的参考脚本；你先验证它的关键行为，再决定如何复用。若选择让 Codex自己实现，同一份小输入也可用于实施前约定验收，而不是假称参考代码是它刚刚独立写出的。

打开 input/probe.csv，应看到：

```
date,channel,visits,signups
2026-10-01,search,10,1
2026-10-01,search,10,1
2026-10-01,newsletter,8,2
2026-10-01,community,,1
```

先别运行，在纸上计算。第二条与第一条完全重复，只计一次；最后一条访问数为空，整条排除。因此访问应为 10 + 8 = 18，注册应为 1 + 2 = 3，计入两条。表头不是数据记录，行号仍从文件第一行开始。

给 Codex 一个带判断条件的实验

在 my-csv-insight 副本中打开 Codex，允许它读取输入并写入 output/，然后发送：

```
先用 input/probe.csv 验证统计口径，不改实现与原始样本。
空值整行排除，完全重复只计首条，并报告原行号。
预期访问18、注册3、计入2、空值排除1、重复排除1。
请先读取相关脚本，再生成 output/probe-summary.md，
使用 verify_totals.py 独立核对。
若结果不符，先解释是哪项输入或规则导致差异，不扩大到完整报告。
```

你也可以自己在副本根目录的操作系统终端运行：

```
# macOS / Linux；Windows 将 python3 换成 py -3
python3 -B report.py --input input/probe.csv --output output/probe-summary.md
python3 -B verify_totals.py --input input/probe.csv --report output/probe-summary.md
```

命令会创建或替换指定报告，因而不是只读操作。路径以终端当前目录为起点。-B 只避免字节码缓存，不关闭报告写入。打开 Markdown 看总数、排除行号和说明，再比较核对程序的退出结果。能生成文件，只支持“写出了产物”；数字与已约定规则一致，才支持这个实验通过。

独立核对怎样增加证据

verify_totals.py 不导入报告模块，而是重新读取 CSV，以集合和列求和计算，再与报告正文总数和核对 JSON 比较。两条代码路径降低同一个实现错误被重复引用的机会；它们仍使用同一口径，不能证明口径本身适合真实业务。手算的四条样本提供了第三个容易理解的参照。

制作时，这两条小实验命令已实际得到访问 18、注册 3，独立核对通过。完整样本也已生成并核对，计入 21 条，访问 2155、注册 267；详细环境和未验证范围在案例 C。这些是参考项目的执行记录，你的副本仍要亲自运行。

如果报告数字对了却没有解释排除记录，实验仍没有满足完整条件。若数字不同，先比较是否计了重复、是否把空值当零、是否跑错文件。不要立即扩大样本或重写整个工具；扩大输入会让原因更难辨认。

实验开始前也要写下结束条件：本次只判断既定规则能否产生可核对总数，不制作界面，不研究部署。数值与行号一致，就保留输入、命令和结论，进入完整样本；若发现规则本身含糊，就停在这个决定上，先澄清口径。一次失败若指出了关键假设，也已经提供价值，不需要为了把实验写成成功而偷偷换数据。

通过后再扩大，失败后改变问题

小实验通过，才用同一副本运行完整输入：

```
python3 -B report.py --input input/activity.csv --output output/summary.md
python3 -B verify_totals.py --input input/activity.csv --report output/summary.md
python3 -B -m unittest discover -s tests -v
```

完整输入有 24 条记录，其中两条空值、一条完全重复；报告应说明 21 条计入，而非默默丢掉三条。四项测试还检查重复运行得到相同字节、坏数值不覆盖已有报告等行为。看到失败时，保留错误与旧文件的区别：旧报告存在，不等于本次新输入已处理成功。

接下来若想验证“改一条输入是否会反映到输出”，选择一条非重复记录，例如 newsletter,80,16，在另一个输入副本中把访问改成 87，再生成另一个输出。按当前样本，访问应增加 7、注册不变。不要改重复对中的一条来做这个实验，否则去重关系也会变化，无法只观察数值增量。现有自动测试已经在临时数据上实际检查了这一变化。

动手练习：写一句实验结论

将自己的观察写成三句话：验证了什么假设；结果是什么；据此继续哪一步。再加一句未覆盖范围，例如“没有确认真实业务中重复行的含义”。不要写“所有数据都没问题”。

参考反馈（离线展开）

合格结论会把已选口径与执行正确分开。若结果不符，下一步应指向能区分原因的检查；若已符合，便可扩大输入，无需重复同一实验来积累“通过次数”。

最小实验的终点是一个决定。下一章把这种可检查的成果连接成有价值的里程碑。

第 12 章：把较大任务分成有价值的里程碑

“收藏工具还差标签、搜索、导入、备份和漂亮界面。”清单越写越长，Codex也一直在忙，但你至今没有一个可以安心保存链接的版本。把任务拆成许多动作，并不一定得到阶段成果；如果每一段都要等其它段才能使用，拆分只是换了一种列待办的方法。

本章回到 LinkShelf。你将为自己的版本确定三个里程碑，给每阶段写出可观察的完成状态，并推进当前最有价值的一段。终点是一条能够连续交付的路线，阶段结束时有实际文件、可运行入口与检查结果，而不是只有“第一阶段完成”的标题。

里程碑应该能被单独使用

“建立 HTML、写 CSS、补 JavaScript”可以是内部实施步骤，却不是普通使用者能判断的三个阶段。只有 HTML 的表单还不能保存，只有漂亮样式也不能找回收藏。若第一阶段结束时用户仍不能完成一件事，就值得重新考虑切法。

可以把第一阶段定义为“保存一条链接并在刷新后找回”。它可能同时涉及页面、事件与存储，但价值完整。第二阶段再解决“多条记录中找目标”，第三阶段解决“带走当前数据快照”。这些阶段各有独立使用行为，也保留了后续扩展的空间。

阶段不是必须同样长。一个下午解决基础路径、半小时增加搜索都合理。不要为了让表格对称，硬把同一条关键路径拆开；也不要因为最后可能有导入功能，就在第一阶段先建立复杂账户系统。

给 LinkShelf 一条三阶段路线

沿用自己的 my-linkshelf，先确认当前已实现什么。教材参考成品提供基础收藏、关键词搜索与 JSON 导出；没有标签管理或重新导入。下面是一条用于自己推进的路线，不能把表中的目标全当作你已经完成：

阶段	对使用者的价值	可观察的完成状态
1 / 留得住	收好一条网页地址	新增可见，刷新仍在；删除后刷新不恢复；坏输入不保存
2 / 找得到	在多条收藏中定位	标题或网址关键词能缩小列表；清空搜索恢复；无匹配不删除数据
3 / 带得走	保留当前数据快照	导出真实 JSON；条目与当前收藏一致；下载结果可打开核对

这条路线刻意没有把“导出”写成“完整备份恢复”。没有导入，就不能承诺一键还原；JSON 文件只是可带走的备份材料。标签也是另一项后续能力，需要决定格式与筛选行为，不能因为你画了几个标签按钮就说已实现。

每一阶段还应保留已有检查。搜索上线后，新增和刷新仍须成立；导出上线后，搜索不能意外改变导出的数据范围。你可以选择只导出全部收藏，也可以另做筛选结果导出，但必须说明并验证。当前参考成品导出全部收藏，不因搜索关键词而丢掉未显示条目。

先核对起点，再选择当前阶段

向 Codex 输入框发出下面的调查任务，工作位置是自己的副本：

```
请先检查当前 LinkShelf，按“留得住、找得到、带得走”
说明已有实现、实际验证与缺少的部分，不立即修改。
引用当前文件，不把教材参考成品的能力当成这个副本已有。
每阶段只写一条使用路径和必要边界，
指出最早尚未达到完成状态的阶段。
```

如果阶段 1 的刷新仍会丢记录，就优先修这一条，而不是被阶段 3 的下载按钮吸引。如果代码已有搜索但没检查空结果，应把状态写为“已实现、待验”，而不是重新安排一轮搜索开发。路线会因实际起点改变，数字编号只是便于交流。

接着选当前阶段发实施任务。例如阶段 2 尚未实现时：

```
本轮只完成关键词搜索：按标题或网址匹配；
空关键词显示全部，无匹配有提示，搜索不得修改收藏。
保留新增、删除和刷新保存，不做标签或导入，不增加后端。
请完成可运行文件，说明怎么启动，并给出两条记录的验收动作。
代码检查与真实浏览器操作分别记录。
```

这里没有要求先完成全部三个阶段才能交付。阶段 2 做到以后，你就有一个比阶段 1 更有用的产品；若阶段 3 暂时受阻，已有成果仍可以使用。每阶段都能形成一个自然的停靠点，这正是分阶段带来的实际收益。

让阶段验收覆盖相邻行为

在自己的页面新增“里程碑12・学习资料”与“里程碑12・项目主页”两条样本。搜索“里程碑12・学习”，本轮两条中应只显示第一条；搜索一个确认不在现有收藏中的词，列表应出现无匹配提示；清空搜索，两条都回来。若已有相同前缀的旧样本，换一个新的前缀，避免把旧匹配误判为失败。最后刷新并确认记录仍在，再删除本轮其中一条、刷新确认只删了目标。

这些动作不仅检查新搜索，还保护原存储路径。没有第二条记录，就很难区分正确筛选与“简单清空”；没有清空搜索，就不知道隐去的记录是否还能回来。最有价值的回归通常围绕相邻行为，不需要为了一个搜索框重建庞大测试系统。

阶段 3 则应该实际检查下载文件，而不只看按钮动画。打开 JSON，比较条目与页面总收藏；文件保存位置由浏览器决定，网页发起下载不能证明已经落到某个固定目录。导出后不要立刻清空原收藏来试恢复，因为本阶段没有导入。这项验收由你亲自进行，不会因教材展示了一个下载按钮就自动通过。

怎样留下足够的阶段状态

可以在自己的副本中保存几行 milestones.md，或沿用已有 progress.md：当前可用阶段；实际运行入口；这次通过的动作；未通过与未检查；下一段目标。不要再创建五份内容重复的状态表。

例如“阶段 2 已有代码；搜索与清空实际通过；删除后刷新未验；先补这项，再进入导出”比“搜索完成 90%”更能指导下一次行动。百分比没有说明缺的是关键行为还是颜色微调。阶段是否完成，应由预先写出的行为与证据支持。

若新事实使某阶段太大，就把它重新切成有用结果。标签可以先只记录、再筛选，但前一段是否对使用者有价值仍要判断；导入可以延后，不要靠把按钮设为禁用来假装已经交付。必要时缩小当前承诺，保存可用部分，下一章会继续讨论途中判断。

动手练习：改写一份只有内部动作的计划

把“搭页面 → 写样式 → 写存储 → 测试 → 上线”改成适合自己 LinkShelf 的三个里程碑。每个阶段写一句用户行为、一条关键边界、一项检查。然后只选择当前最早缺少的一段执行，不要求今天把全部功能做完。

参考反馈（离线展开）

阶段 1 应已能保存并重读，而不只是看见表单；阶段 2 应能在多条数据里找目标；阶段 3 应产出可核对的真实文件。检查依据当前实现，标签与导入仍应明确标为后续范围。若某功能已实现，补证据比重复开发更有价值。

一条好路线会持续留下可用结果。进入第 13 章，再判断 Codex在每个阶段中的忙碌，是否真的正在接近这些结果。

第 13 章：如何读懂进度并在途中介入

屏幕上不断出现文件名与命令，Codex说正在推进，你却越来越不确定：现在打断会不会破坏有效工作，继续等待又会不会只是重复失败？判断进度的关键，不是动作多不多，而是动作有没有减少未知、改变相关对象，或增加完成条件的证据。

本章将执行记录、文件差异和使用行为放到一起。你会在三个情境中作出“继续、调整、停止”的决定，并对自己的任务发出一次具体介入。你不必懂所有代码，但需要能指出下一项行动为什么值得做，以及什么现象说明当前路线已失去依据。

从动作看到它提供的证据

“准备检查保存”是意图；读取文件是一次行动；修改差异说明磁盘内容改变；浏览器刷新后仍在，才支持保存路径在该条件下成立。第 1 章的区分在这里再次派上用场。进度不能只按承诺积累，也不能把每条命令都当成一份同等价值的证据。

下面是为教学构造的示意记录，不是本项目真实 Codex 日志：

```
示意｜目标：保存成功后清空表单，失败时保留输入。
示意｜已读取：表单提交入口、写入函数、错误反馈分支。
示意｜已修改：清空操作的位置。
示意｜语法检查通过；尚未实际检查浏览器中的保存失败。
示意｜下一步：用失败分支确认输入不会提前丢失。
```

这里适合继续。它找到相关路径，修改针对目标，并知道还缺什么证据。浏览器尚未检查不等于工作无效；只要下一步能补上缺口，就没有必要为了催促而切换方案。你可以让它继续完成已确定的检查，自己再承担它无法执行的手工部分。

看差异，不只看修改了多少行

diff 是文件修改前后的差异，通常用 - 表示删除、+ 表示增加。先通过当前界面的变更列表查看即可；Git 的具体操作留到下一章。对自己生成的项目，先定位与目标有关的函数，再要求 Codex解释变化如何影响使用行为。

下面是简化示意差异，不是可直接应用的补丁：

```
- 保存成功以后，清空表单
+ 先清空表单
+ 再尝试保存
```

改动很小，却可能让存储失败时的输入丢失。仅有“语法检查通过”解释不了这个行为。你可以问：“这次把清空提前以后，保存失败分支怎样保留输入？请引用实际实现，再做失败检查。”不需要先理解全部页面结构，也能围绕目标审阅一个关键顺序。

反过来，改动较多不一定错误。有时补齐错误反馈需要同时改输入、状态与样式。先看这些变化是否形成同一条使用路径，是否越过本轮范围，再决定要不要收缩。行数是定位线索，不是质量分数。

什么时候调整当前动作

第二段仍是教学示意。假设故障是“删除后当前列表消失，刷新又回来”：

```
示意｜再次增加界面重绘。
示意｜语法检查通过。
示意｜实际复现步骤仍然得到：删除后刷新恢复。
示意｜准备把重绘延迟再增加一些。
```

这里值得调整。新的修改还没有解释刷新重新读取的数据，已有观察也没有支持时序假设。介入时别只说“你又错了”，把证据和下一项检查连起来：

```
先暂停继续增加重绘或延时。
目前事实是：删除后的界面变化正确，但刷新后旧记录恢复。
请沿删除、状态保存、页面读取追踪实际数据，
先给出一个能区分原因的检查，再决定最小修改。
保留已有效的新增行为，不顺便重做整个页面。
```

这段话调整的是调查方向，不是禁止它行动。你提供了能反驳当前解释的观察，也给了下一项有价值的任务。连续失败时，真正该改变的常常是问题与假设，而不是在原指令后加更多强调语。

什么时候停止当前尝试

停止适用于继续当前动作会扩大无关变化、破坏已有成果，或关键前提尚未获准的情况。例如只想离线搜索，执行却准备替换整套存储并删除旧数据；又如读错目录后仍准备写入。此时应先停止这项动作，核对位置、范围与已有更改，再选择恢复路线。

停止当前尝试不等于放弃项目。先保留能解释现状的材料：目标、实际改动、最后一次复现、尚未确认的假设。不要顺手清空目录来获得“干净状态”；那里可能还有你或另一项任务的有效修改。下一章会用独立练习介绍版本与回退，这里先做到看清后再处理。

如果某动作只是缺少环境能力，例如没有浏览器，停止反复调用同一不可用入口即可。它仍可以检查代码和数据逻辑，再明确交给你手工操作。工具受限与功能失败是两件事，不能为了把进度写成全绿而伪造浏览器结果。

从文件存在，继续看到本次调用是否成功

CSV 案例提供另一种容易误读的进度：旧 summary.md 仍在，但新输入含坏数值，本次命令失败。保留旧报告是保护行为，不是新输入分析成功。你应看本次退出码、错误与输入，再决定修数据还是改规则，不能让 Codex仅凭“输出文件存在”就交付。

在自己的 my-csv-insight 中，可运行已有检查观察这一区别：

```
# 操作系统终端；Windows 将 python3 换成 py -3
python3 -B -m unittest discover -s tests -v
```

测试内部用临时输入制造坏数值，确认旧报告不变；整体测试应通过，不会故意让你的完整检查失败。它提供特定行为的证据，不能替代你对业务口径的判断。制作时四项测试确实通过，读者副本结果仍以实际命令为准。

用三个问题作出决定

先问“这一步使哪个未知减少了”，再问“改变是否仍服务于完成条件”，最后问“下一步有什么区分力”。有清楚答案且风险在已授权范围内，通常继续；结果反驳原假设或动作没有信息增量，就调整；对象、范围或前提明显不对，就停止当前动作并恢复可判断状态。

不需要规定每隔几分钟强行打断，也不需要把失败次数当魔法数字。一次有力反例就可能足以换方向，多次小失败也可能都在排除不同原因。你判断的是证据积累，而不是聊天长度或重试计数。

动手练习：留下一个可解释的介入决定

选自己正在推进的 A、B 或 C 副本，找一段真实执行记录和一处实际变化。写三句话：这段记录支持什么；还不能说明什么；现在选择继续、调整还是停止，并给下一项具体动作。没有真实运行记录时，可以评价本章的示意情境，但必须写明这是情境判断，不能记作实际任务已验收。

参考反馈（离线展开）

“继续补失败分支检查”比“看起来不错”具体；“调整为检查保存与读取”比“再认真修”增加信息；“先停止错误目录的写入”保护了对象边界。结论应与实际观察相连，而不是仅凭语气乐观或等待焦虑。

你开始能在途中保留判断。接下来进入第 14 章，让变化范围与恢复动作也变得可检查。

第 14 章：变化范围、版本和回退

Codex 能一次修改多个文件。便利也带来一个实际问题：你怎样知道它改了哪些地方，发现方向不对时又怎样回来？如果答案只是“让它再改一次”，你还没有留下可靠的恢复位置。

Git 把某个时刻的文件状态记成提交。你可以先观察当前变化，再决定哪些进入一个提交，之后恢复指定文件或用新提交撤销旧提交。本章只学这些最少操作，完成一个可回退的练习修改。提交不会自动上传，也不会证明程序可用；它的价值是让变化可以比较、解释与恢复。

先进入不会碰到原项目的练习仓库

需要 Python 3、Git 2.28 或更新版本，以及完整教材。先在操作系统终端运行 git --version，确认 Git 可执行。不要在自己的工作项目直接练习恢复命令。

在教材仓库根目录运行：

```
python3 -B extensions/git-lab.py
```

Windows 已安装 Python Launcher 时用 py -3 -B extensions/git-lab.py。辅助脚本将案例 B 起点复制到系统临时目录，创建新的 Git 仓库与基线提交，只在新仓库设置练习身份，不修改教材或全局 Git 配置。它不连接远端，也不删除旧练习。

终端最后一行是新目录的绝对路径。复制它，执行 cd "刚才输出的完整路径"；PowerShell 可用 Set-Location "刚才输出的完整路径"。下面所有 Git 命令都在这个新目录运行：

```
git status --short
git log --oneline -1
```

初始状态应没有待处理文件，历史中应有基线提交。没有输出的 status --short 表示工作区与索引相对当前提交没有这些变化，不是命令失效。若脚本报错，先看找不到 Git、源目录还是提交失败，保留新目录；不要转而在教材根目录“试一下恢复”。

变化、暂存与提交，是三个不同位置

用编辑器在练习 README.md 末尾添加一句“练习：本轮先核对变化范围。”，再在 notes/ideas.md 末尾添加“保留：这是另一项练习记录。”。现在运行：

```
git status --short
git diff -- README.md
```

状态会列出两个修改文件；diff 只展示指定 README 相对暂存区的变化。新增行前面是 +，删除行前面是 -。这不是要求你逐字符审代码，而是先回答：实际变化是否落在本轮目标范围？为什么另一个文件也发生变化？

工作区是眼下的文件，暂存区是准备进入下一次提交的选定内容，提交则是已经记录的版本。



工作区、暂存区与提交之间的关系：取消暂存保留文件修改，恢复覆盖指定内容，撤销提交形成新历史

用明确路径暂存本次说明：

```
git add -- README.md
git diff --cached -- README.md
git status --short
```

--cached 检查待提交内容。此时另一个笔记仍未随它暂存。状态短格式有两列，第一列对应索引，第二列对应工作区；同一个文件还可能在暂存后继续被修改。因此看到文件名就提交还不够，要核对真正进入提交的那份差异。

暂存也不是一份独立备份。如果暂存后又编辑文件，两份内容会不同，下一次提交记录的是索引中的内容。让 Codex 在提交前同时核对两份差异，尤其留意它是否把临时输出、个人配置或与本轮无关的资料也选进去了。对你不了解的文件，先调查用途再决定，而不是用“全部加入”省去这个判断。

不想现在提交，可以取消暂存：

```
git restore --staged -- README.md
```

重新打开 README，刚才那句话仍在。取消暂存改变的是选定内容，不等于丢弃文件修改。这是最容易混淆、也最值得亲眼确认的一处区别。

恢复一个文件前，先保存你还需要的内容

本轮决定放弃 README 的练习句，但保留笔记。先在编辑器把当前 README 另存到这个临时仓库之外，例如它的父目录，确认备份可打开。然后核对当前目录和 diff，只在刚创建的练习仓库执行：

```
git restore --source=HEAD --worktree -- README.md
git status --short
```

这会用当前提交中的 README 覆盖工作区对应文件。它会丢弃该文件尚未保存到提交的变化，所以先备份并确认范围。执行后 README 的练习句消失，notes/ideas.md 中的保留句仍在。这里使用明确路径，没有清理其他文件，也没

有让整棵工作区回到旧状态。具体语义见 Git restore 文档。

如果恢复结果与你以为的基线不符，先查看 git log 与备份，确定 HEAD 指向的是什么。不要把“恢复”理解成 Git 能猜出你心中的正确状态。未跟踪文件也不会因为普通 diff 没显示就自动安全；git status 中的 ?? 仍需单独判断。

已提交的变化，用新的历史撤销

先将笔记作为一项独立练习成果记录下来：

```
git add -- notes/ideas.md
git commit -m "lab: keep independent exercise note"
```

再次编辑 README，添加“练习：这次提交只记录说明变更。”，然后：

```
git add -- README.md
git diff --cached -- README.md
git commit -m "docs: record review exercise"
git status --short
```

确认此时工作区干净、最新提交就是刚才的 README 变更，再执行：

```
git revert --no-edit HEAD
git log --oneline -4
```

你会看见一个新的撤销提交；被撤销的原提交仍在历史里，README 回到此前内容，已经独立记录的笔记仍保留。revert 用反向变化形成新提交，适合保留变更过程。它与覆盖工作区某个文件的 restore 解决不同问题；不要只因两者都叫“回退”就互换。见 Git revert 文档。

本练习顺序已在 macOS、Git 2.54.0 的独立临时仓库实际执行：取消暂存后内容保留，指定文件恢复没有动笔记，提交后撤销产生新历史。Windows 命令路径已提供，未在 Windows 本机复测。你的提交哈希和临时目录会不同，以文件内容与历

史关系判断结果。

让 Codex 保护任务开始前的工作

真实项目一开始可能就有你的未提交修改。先让 Codex 读 status 与 diff，记录这份起始状态，再实施本轮任务。可以发给它：

```
先检查当前 Git 状态，区分任务开始前已有的改动与本轮新增改动。
本次只处理指定需求，不将无关文件打包进提交。
如果需要恢复，先说明具体路径、来源版本与会被覆盖的内容，
保留我尚未提交的工作；不要直接清理整个工作区。
```

一个文件里也可能混合两个人或两次任务的修改。此时按文件恢复仍可能过宽，应先备份并分离差异，或人工保留需要的段落。让 Codex 说“我会小心”没有提供保护；明确路径、已知基线和实际备份才有恢复依据。

动手：交付一个可以回来的位置

重新生成一份临时练习。自己完成两处修改、只暂存一处、取消暂存、备份后恢复指定文件，再提交另一处。要求 Codex 解释每一步改变了工作区、暂存区还是历史，你用文件与 diff 核对。

完成时留下目录、提交说明、尚未提交内容及恢复选择。能够说出“我准备撤销哪一份变化，哪些现有工作必须留下”，就比只会记一条回退命令更接近掌控。

第 15 章：从“程序启动”到“功能真的可用”

你已经看见一个漂亮页面，终端没有红色报错，Codex 也说“完成”。现在试着回答：链接是否真的保存？搜索是否只显示匹配项？删除后刷新会不会回来？页面能够打开，只解决了访问入口，还没有回答这些问题。

本章把第 3 章的基础验收扩展成一组有针对性的行为检查。完成时，你会得到一份属于自己运行环境的验收记录，能指出哪些结论由自动检查支持，哪些由浏览器动作支持，还有哪些没做。重点是选择足以发现问题的动作，而不是增加测试数量。

从使用者的完成状态倒推

沿用案例 A 参考成品或自己的工作副本，准备 Python 3、浏览器。自动数据测试需要 Node.js 18+。运行方式见 cases/A-linkshelf/README.md，参考成品通常是 http://localhost:4174/solution/。保持同一浏览器和来源；端口或主机变化会让你读到另一份 localStorage。

先给“可用”一个具体定义：我添加“验收15・示例网站”，刷新后仍在；添加“验收15・学习资料”，搜索“验收15・学习”时只显示它；清空搜索恢复全部；删除指定条目并刷新，不会恢复；导出的 JSON 与当前列表对应。不要把“所有功能正常”作为记录，因为后来的人无法据此重复你的检查。

本章用专用标题前缀区分本轮样本；若已运行过本章，换一个新的前缀，只删除这次新增的记录。

保存这组动作时，同时写明前提。例如“首次空列表”和“已有两条收藏”是不同起点。已有重要收藏先导出备份，本轮只删除自己的测试条目。不要为了每次都从零开始，就清空整个浏览器数据。

四种检查，分别回答不同问题

检查	在这个案例里能回答	不能单独回答
语法检查	JavaScript 是否能被解析	按钮绑定是否正确、数据是否持久
数据逻辑测试	校验、筛选、读写函数面对指定输入的结果	浏览器 DOM、下载与焦点是否正确
HTTP 与页面访问	入口和资源是否能从服务取到	新增、刷新和删除是否满足使用目标
浏览器行为验收	实际界面、刷新、键盘与下载行为	所有浏览器、所有输入与长期存储可靠性

在仓库根目录的操作系统终端执行：

```
node --check cases/A-linkshelf/solution/app.mjs
node --test cases/A-linkshelf/tests/core.test.mjs
```

本项目数据测试实际得到 10 项通过，涵盖保存后再次读取、删除后读取、坏协议、损坏数据、存储失败、搜索和 JSON。测试用程序化存储对象，不是真实浏览器权限面板。它给出了有价值的证据，也留下了明确边界。

若你的工作副本已改变函数或目录，不能把参考成品的测试结果直接贴到自己的交付里。让 Codex 说明测试实际导入的是哪份实现；路径不一致时，先对准目标再谈通过与否。

设计一条会暴露问题的路径

在浏览器中添加“验收15・示例网站” https://example.com。点击“保存到链接架”，记录列表与数量，然后刷新。接着添加“验收15・学习资料” https://example.org/learn，输入关键词“验收15・学习”，核对显示的是哪一个标题和网址，不能只记“显示一条”。清空搜索，应重新看见全部。

再删除“验收15 · 示例网站”，立即刷新。检查被删记录没有回来，另外一条仍在。这个动作同时防止两种误判：只更新当前 UI，却没保存删除；为了删一条，却清空了整份收藏。你检查的是目标变化与应保留的邻近数据。

最后点击“导出 JSON”，检查浏览器下载列表，并用文本编辑器打开实际文件。核对 format 为 linkshelf、version 为 1、links 中的标题与 URL 对应当前数据。不要只凭按下按钮或看见成功提示，就宣称文件已下载。此阶段没有导入，验收中不能补写“导入通过”。

案例 D 刻意保留了只删内存的版本，可以说明为什么刷新很关键：当前页面消失与跨刷新删除是两种结果。第 18 章会对这个差异做完整调查，本章先把这条边界写进验收，避免漏检。

错误输入与恢复，也属于使用路径

空标题保存，应该说明需要填写标题；网址输入 example.com 或 javascript:alert(1)，应该拒绝保存并说明接受的地址格式。检查数量未增加、已有收藏未变。随后改成合法值，应该可以继续新增。若错误后整个表单再也不能用，单独一条“错误输入被拒绝”的测试仍漏掉了恢复问题。

存储失败是另一个边界。自动测试用会抛错的存储对象检验反馈与数据保护；若你要在浏览器核对，先备份测试数据，再按该浏览器的站点存储设置操作。入口和行为因浏览器而异，没有执行就写“浏览器权限场景未测”，不要把测试对象的成功当作真实设置已验证。

键盘也做一次最小检查：用 Tab 移动到标题、地址与按钮，确认焦点看得见；表单内按 Enter 提交；错误时能找到要修改的字段。在窄屏中观察输入和列表是否可读、有无横向溢出。测试无需遍历所有分辨率，但你应该记录实际看过的尺寸与浏览器。

让 Codex 为结果负责，而非为数量负责

把下面的任务发给 Codex，并根据当前可用工具调整：

```
请按实际使用行为检查 LinkShelf：新增→刷新、搜索→清空、
删除指定项→刷新、坏网址→修正、导出→读取实际文件。
每项写出起始数据、动作与观察；说明检查针对哪份代码。
能运行的检查实际运行，没有浏览器就保留人工步骤与未验状态。
出现失败先保存复现条件，不要为了让测试通过改掉完成条件。
```

好的回复会指出某个动作和结果，例如“删除编号对应条目后再次读取未出现，另一条仍在”。“执行 50 项测试，全绿”若没有说明覆盖什么，仍不足以支持当前目标。反过来，一个能稳定暴露删除恢复的检查，可能比许多只验证函数存在的测试更有价值。

失败之后，只扩大必要范围

如果搜索返回错误条目，先保留输入和预期，定位筛选与渲染之间的差异；修复后重做搜索、清空和新增后可见性。若删除影响了保存路径，重做新增、删除、刷新与写入失败。不是每次都需要重跑所有工具，也不能因为刚修过一个点，就跳过受影响的相邻路径。

检查无效时还要质疑验证方式。两个测试若都从同一个错误计算取预期值，只是互相附和；浏览器若打开的是旧服务目录，重试再多次也没在验证新实现。记录完整 URL、实际文件与命令退出状态，往往比增加重试次数更有帮助。

动手：完成自己的验收记录

亲自完成上面的使用路径，保留一份短记录：环境与目标版本；动作及结果；失败与修复后的复测；尚未执行的范围。挑一项，让另一位读者只凭记录重复，看看是否还缺起始条件。

完成标准不是一张全部勾选的表，而是你能解释每个“通过”来自哪种观察，以及哪个重要边界还没有证据。后续交付时，这些可重复的行为会比一句“我这里能跑”更有用。

第 16 章：代码审查、解释和交付

“我看不懂每一行 Python，还能审查 Codex 的修改吗？”可以，但需要把问题问到合适的位置。你可以判断结果是否满足需求、变化是否越界、错误时会发生什么，以及后来的人能不能运行。遇到实现细节，再要求 Codex 引用代码解释，而不是让“代码看起来很专业”代替判断。

本章从案例 B 的状态筛选交付开始。你会把需求、diff 和实际检查对齐，了解 Codex review 的作用，最后写出一份别人能复查的交付说明。前提是你有案例 B 的工作副本或参考修复版、Python 3，以及第 14 章的 Git 基础。

先审查它承诺改变的行为

案例 B 的目标很窄：list --status open 只显示未完成，done 只显示已完成，all 与省略参数保持全部。筛选应当只改变展示，不重写任务文件、不改变编号，也不影响添加和完成操作。

先把这一段需求放到审查窗口旁边。不要从“代码有没有用某种高级写法”开始。一个很漂亮的重构如果顺便改了数据格式、删除了未匹配任务，仍然没有完成这个任务。

如果直接按第 7 章复制了 my-tasktrail，先不要假定它有独立 Git 历史；位于教材仓库内也不等于拥有自己的修改基线。可以在第 14 章生成的独立临时仓库中完成同样的筛选改动后审查，或用编辑器比较自己保留的改前与改后文件。只有确认了目标仓库与基线，才使用下面的 Git 差异。

在你的Git 工作副本目录执行：

```
git status --short
git diff --stat
git diff
git diff --cached
```

分别核对文件范围、具体变更及已暂存内容；未跟踪文件仍需从 status 找到后阅读。若看到样本数据被改，追问是必要格式迁移、测试误写，还是运行时无意产生的变化。不要仅因文件“不属于代码”就把它忽略。

如果你只有教材下载文件，可以用编辑器比较 cases/B-tasktrail/starter/ 与 solution/，但这只能审查参考方案，不代表自己的工作副本已经采用并验证了相同修改。

要求解释一个具体因果关系

可以发给 Codex：

```
请审查这次 TaskTrail 状态筛选改动，先不修改。
对照需求与实际 diff，重点检查：默认行为是否保留，
筛选是否只影响显示，错误状态是否拒绝，数据文件是否被意外写入。
每个问题给出触发条件、实际文件或函数、影响与最小验证方式。
没有证据的问题请作为待确认，不将风格偏好写成缺陷。
```

实际参考实现把筛选放在 tasktrail.py 的列表分支，先 read_tasks()，再在内存中按 done 过滤，最后打印；没有沿这条分支调用 write_tasks()。这能解释为什么筛选不应删数据。你仍要通过运行与数据对照验证，而不是只听它口头保证“不会影响存储”。

当它说“可能有问题”，要求落到可触发条件。例如“空匹配时会输出误导信息”可以用全部任务已完成后筛选 open 检查；“代码不够优雅”则未必值得打断交付。审查应优先找影响结果的问题，避免把所有建议变成阻塞项。

当期 Codex review 入口

截至 2026-10-25，本机 Codex CLI 0.154.0 实际执行 codex review --help，确认有以下入口：

```
codex review --help
```

帮助中包含 --uncommitted、--base <BRANCH>、--commit <SHA>，用于选择未提交变化、相对基准分支或某个提交。它们的当期说明见 OpenAI 官方 CLI reference。本章只执行帮助核对，没有另行运行模型审查或声称得到审查结论。

如果你的账号与环境已可用，并愿意消耗相应使用额度，可以在目标 Git 仓库目录选择一种范围：

```
codex review --uncommitted
```

这不是静态本地命令；它会调用配置的 Codex 审查能力，使用方式与可用性以当前客户端、账号和组织配置为准。不要照抄一个不存在的分支名，也不要同时堆多个目标参数。当期官方参考还将自定义 PROMPT 与这些目标列为互斥，不要在带目标的命令后直接追加一段审查要求。先看自己版本的帮助，核对审查实际针对的范围；App 或 IDE 中用当前界面提供的审查入口，并记录所选差异。

专门的审查能换一个视角发现遗漏，但它仍是模型判断。没有发现问题不等于证明没有问题，发现的问题也要核对触发条件。你可以请它找“筛选时误写数据”的风险，然后用运行结果确认，而不是要求它给一张“无风险证明”。

把发现转成可验证的处理

在案例 B 的 solution 目录，可先运行参考检查：

```
python3 -B tasktrail.py --data sample.json list --status open
python3 -B tasktrail.py --data sample.json list --status done
python3 -B tasktrail.py --data sample.json list --status all
python3 -B -m unittest discover -s tests -v
```

Windows 将 python3 换成 py -3。核对 open 与 done 分别显示不同编号，all 显示全部。若审查指出读取后写回可能破坏数据，可在工作副本中备份数据，对比筛选前后的字节或哈希；不能只因列表看起来正确就略过存储副作用。

审查发现真实缺陷后，让 Codex 针对它修复，再运行触发该问题的最小检查与受影响的邻近行为。新的 diff 仍需要看：一次修复可能加进不必要的迁移或改动其他命令。不要一收到审查结果就无条件接受所有修改建议，也不要因为实现者和审查者意见不同，就让它们无休止互相说服。

一份交付说明应该让别人接得住

交付的第一句解释具体变化和用户结果，例如：“列表现在可按 open、done、all 筛选；省略参数仍显示全部，筛选不修改 JSON。”随后给出启动位置、实际执行命令与关键结果，再写未覆盖范围和已知限制。

下面是交付写法示意，其中结果需要换成自己的真实观察：

```
结果：在当前 TaskTrail 副本增加列表状态筛选，默认行为保留。
改动：命令参数与列表显示分支；数据格式未作迁移。
验证：在本副本目录运行……；open 显示……，done 显示……；
数据文件对照……；自动测试……项通过。
限制：未在 Windows 本机运行；不支持并发写同一文件。
继续：从……目录、……提交开始；下一项需求尚未实施。
```

这不是要求每次都填写一张复杂表。小修改用一段话即可，只要别人能分辨“已写”“实际跑过”和“还没验”。如果有失败，不要把它藏在几十行成功日志末尾；先说明它影响哪个完成条件，当前还能使用什么。

动手：做一次有结果的审查

选择自己已经完成的一次案例 B 修改。先不改代码，让 Codex按需求、差异与失败模式审查。选出一条最有实际影响的发现，亲自复现；若没有有效缺陷，就记录审查范围与已执行检查，不需要为了凑成果制造问题。

最后写一段交付说明，让另一位读者只按说明启动并检查一次筛选。若他还需要追问“在哪个目录”“用哪份数据”“这个结果真跑过吗”，就补那一处。有效审查把风险落到行为，清楚交付则把成果变成别人可以继续使用和维护的工作。

第 17 章：识别错误发生在什么层次

“Codex 说改好了，可我这里还是不行。”这句话很常见，也几乎没有给出调查方向。浏览器打不开、按钮没有反应、刷新后数据恢复，都可以被描述成“不行”，但它们需要的下一步完全不同。若一律回答“再修一下代码”，你可能在服务没有启动时改界面，在打开旧文件时重写逻辑，或者在需求没有说清时不断完善错误的结果。

本章要练的能力，是把一次失败缩小到一个能够检查的问题。完成时，你应能说出：目前确认了哪一段正常，哪个判断仍是假设，下一次检查会怎样改变行动。你不必先成为调试专家，但需要知道什么时候应该让 Codex 停止修改，开始调查。

先固定“在哪里发生了什么”

本章使用案例 D：Phantom Delete。它是独立的教学故障程序，数据与 LinkShelf 分开。准备现代浏览器、Python 3，以及教材文件。先阅读 cases/D-phantom-delete/README.md 的启动与数据边界；不要用真实收藏练习，也不要直接打开参考修复版。

在教材仓库根目录的操作系统终端执行：

```
python3 -m http.server 4176 --bind 127.0.0.1 --directory cases/D-phantom-delete
```

Windows 已安装 Python Launcher 时将 python3 换成 py -3。浏览器打开 http://127.0.0.1:4176/starter/，终端保持运行；结束时按 Ctrl+C。这里使用 Python 的本机静态服务，目录与绑定参数见 Python 官方说明。如果尚未安装或检查 Python，回看第 2 章，先完成环境检查。

先记录完整地址、版本入口、点击对象和可见结果。例如：“在 /starter/ 点击‘示例网站’的删除后，它从列表消失；不做其他操作，刷新同一地址，它又回来。

”这里的每个限定都有用。省略“同一地址”，就可能混淆不同数据来源；省略“不做其他操作”，就可能漏掉另一动作已经保存了数据。

如果你已经练习过，点击“恢复初始样本”并确认，只重置当前版本的合成数据。不要为了得到整洁结果清空整个浏览器存储；那会破坏调查现场，也可能影响其他练习。

六个层次，对应六种检查

这些层次不是要填写的分类表。它们帮助你避免在没有证据时，把所有问题推给实现代码。下面的分支是诊断示例，不是声称本案例实际出现过全部故障。

不确定性所在	可以观察的分歧	能缩小范围的一次检查
需求	“删除”究竟只隐藏本页，还是刷新后也不恢复？	找到已约定的完成条件；若没约定，先补清楚。
环境	浏览器根本连不上地址。	检查服务终端是否仍在运行，端口与地址是否相同。
工具	Codex 没有权限读取目标文件，却声称已经调查。	查看具体工具返回和实际路径，区分未执行与执行失败。
实现	点击后内存改变，存储未变。	追踪点击处理函数到保存函数，检查是否真的调用。
数据	两个地方都能读到数据，但读到不同的版本或来源。	核对浏览器、协议、主机、端口、专用键及记录 ID。
验证	只看见条目消失，就宣布持久删除成功。	追加同源刷新和再次读取，观察状态能否跨载入保留。

这六层可以相互影响。例如，写存储失败可能由浏览器策略造成，但程序若仍显示“保存成功”，还存在错误处理问题。你可以先确认环境拒绝这一事实，再决定是否要修正反馈。不要因为分类交叉就放弃调查；层次的价值是提出具体问题，而不是争论问题“究竟归哪一类”。

一次好检查，应让两个解释得到不同结果

面对“刷新后又回来”，可以提出两个候选解释：删除后的内存仍含原记录，只是页面把它藏起来；或者内存已正确删除，但保存没有发生。它们都能解释当前条目不可见，却预期不同的内存内容。

点击“检查内存与存储”。右侧会列出当前内存与重新读取的 localStorage 记录，包含 ID 和标题。若当前内存没有 sample-1，存储中仍有它，这就反对“只是隐藏页面，内存没变”这个解释，把注意力引向保存与读取之间。只比较数量还不够：两份列表都为两条，也可能各含不同记录，所以要核对 ID。

本案例的对照面板直接读取浏览器状态，不是预先编好的终端输出。但面板同样是程序的一部分：若其结果与外部观察矛盾，可以要求 Codex 引用 shared/ui.mjs 中 inspect() 的实现，再在浏览器站点存储面板独立核对专用键。验证工具也需要有可理解的来源。

你可以把下面这段发给 Codex：

> 先不要修改。我在 Phantom Delete 的 starter 页面观察到：
> 删除 sample-1 后内存里没有它，但专用存储里仍有它；
> 刷新同一地址后它恢复。
> 请引用当前文件确认：点击事件调用什么、当前列表由谁持有、
> 保存函数在哪里、页面初始化从哪里读取。
> 把代码事实与仍待验证的推断分开。
> 下一步只提出一个最能区分原因的检查，并说明两种结果各意味着什么。

把其中的观察换成你真正看见的结果。若浏览器连不上，就不要复制“内存没有它”这句话，那会把不存在的事实塞进调查。此时下一步只是核对服务终端与 URL，而不是沿保存函数修改。

用文件把判断接到现实

Codex 的报告至少应该能连接到这条实际路径：starter/app.mjs 装配当前版本；shared/ui.mjs 将删除按钮接到 deleteRecord()；starter/core.mjs 决定删除后的列表；shared/storage.mjs 提供 readRecords() 与 saveRecords()。有了这条路径，你可以继续追问某一步，而不是接受“可能是缓存”的空泛解释。

同时核对数据身份。故障版使用 codex-mastery:phantom-delete:starter:v1，修复版使用另一个键。打开修复版后看见三条初始样本，不能证明故障版修复失败，因为它们本来就是独立数据。localStorage 按来源区分，localhost 与 127.0.0.1 也不是同一主机；相关行为与权限限制见 MDN localStorage 文档（核对于 2026-10-25）。

一个有用的阶段结论可以很短：

> 本轮已确认删除改变了内存，存储仍含原 ID。需要核对删除路径是否执行持久化；暂没有证据支持重写渲染。下一步检查 deleteRecord() 与 saveRecords() 的调用关系。

这比罗列十个可能原因更接近行动。它没有过早宣布根因，也没有把调查停在“需要更多信息”。

检查本身失败时，先保留边界

如果 Codex 的命令被权限拒绝，报告应写“未执行成功”，不能写“测试失败，所以代码有错”。如果 Node 测试通过，浏览器却打不开，要拆开两个结论：数据函数在测试输入下通过，当前 HTTP 入口尚未成立。测试通过不会启动服务器，服务器返回页面也不会证明保存逻辑正确。

如果你读到一份格式损坏的存储，先保留原值与专用键，再调查来源。把它清空后页面恢复，只能说明另一份初始数据可以运行，不能说明损坏原因已经修复。对真实项目，这种误判尤其昂贵：你可能用“恢复演示环境”代替了“修复用户数据路径”。

动手：为下一次行动写一句依据

在故障版完成一次“恢复样本→删除→检查→刷新”，保留自己的观察。让 Codex 提出两个不同层次的解释，你选一个能让两者产生不同结果的检查。执行后写三句：观察到了什么；哪个解释因此更可信或被削弱；下一步为什么该调查或修改某个具体位置。

再做一个迁移判断：如果页面完全无法连接，你原先那套删除检查还能执行吗？说出此时应先获取的证据，并实际核对服务终端与浏览器地址。完成标准是让下一步由证据决定，能够说明为何此时需要调查、修改、恢复环境或补充验收。下一章会沿已定位的数据路径完成最小修复。

第 18 章：停止无效修补，重新检查假设

你让 Codex 调整了列表显示，删除后刷新仍会恢复；再让它重新渲染，结果依旧。下一次它建议“等一会儿再刷新”。这段修补过程是教学示意情境，并非本项目实际发生的对话记录。它描述了一种熟悉的失控：每轮都有改动，却没有新增能够解释失败的证据。

真正需要停止的是没有依据的重复修改。一次尝试若已经反对了原先解释，就可以立即转向调查；不必等到固定的第三次。本章使用可运行的 Phantom Delete 故障代码，练习把任务从“再修一下”改成“重新检查假设”，最后得到能解释、能刷新验证的修复。

保留故障，建立自己的修复副本

沿用上一章的 Python 3、浏览器与案例 D。另需 Node.js 18 或以上版本运行数据逻辑测试；没有 Node 仍可做浏览器练习，但不能声称自动测试已经通过。先确认仓库根目录没有同名 my-phantom-delete，已有时换一个新名字。

在仓库根目录的操作系统终端复制整个案例。macOS / Linux：

```
cp -R cases/D-phantom-delete my-phantom-delete
```

Windows PowerShell：

```
Copy-Item -Recurse cases/D-phantom-delete my-phantom-delete
```

连同 shared/ 一起复制，因为两个版本共用 UI 与存储模块。让 Codex 在副本中工作，只修改 starter/core.mjs；先不读 solution/。停止原先 4176 服务，再从仓库根目录运行副本：

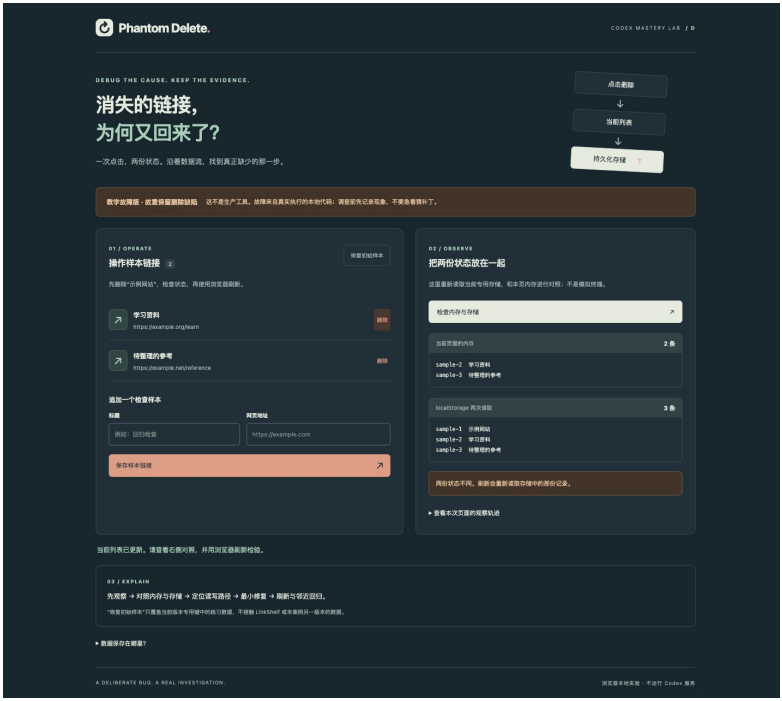
```
python3 -m http.server 4176 --bind 127.0.0.1 --directory my-phantom-delete
```

Windows 将 python3 换成 py -3。浏览器打开 http://127.0.0.1:4176/starter/，结束时在服务终端按 Ctrl+C。不同浏览器或来源会看到不同存储；练习保持同一个浏览器、地址与操作页。

副本只隔离文件。同来源、同版本的副本与原件仍可能共用本案例键；请仅使用合成样本，需要保留的调查结果先备份，再恢复初始状态。

先复现一次，期间不要做别的动作

点击“恢复初始样本”并确认，看到三条合成链接。删除“示例网站”，点击“检查内存与存储”，然后直接刷新。故障版的预期是：删除后当前内存只剩两条，存储重新读取仍有三条；刷新后三条恢复。你要亲自记录这个结果，而不是照抄为已执行日志。



Phantom Delete 的真实运行界面：在同一屏对照当前内存与重新读取的存储

图中界面用于说明观察位置；你的数据与操作结果以自己的运行记录为准。右侧是真实浏览器状态的对照，不是网站替你运行了 Codex。

这里有一个会掩盖证据的分支：删除后先新增，可能使故障暂时消失。新增会保存当前完整内存数组，也就顺带保存了此前的删除。因此第一轮必须是“重置→删除→检查→直接刷新”。稳定复现依赖具体动作顺序，笼统的“有时删除有问题”会把这个差异藏起来。

把一个猜测拆成有方向的数据路径

“刷新后回来”只说明跨刷新状态不一致，单凭现象还不能排除读错键、错误初始样本或写入被拒绝。先让 Codex 沿路径调查。发给 Codex：

```
先不修改。副本的 starter 删除 sample-1 后，内存没有它，
重新读取存储仍有它，刷新后恢复。
请沿点击事件→内存列表→存储写入→初始化读取追踪，
引用真实文件和函数。区分已确认事实与假设。
提出一个最小检查：它的不同结果分别支持什么解释？
不要先添加重渲染、延时或另一个缓存。
```

实际入口 starter/app.mjs 把当前版本的 deleteRecord() 交给 mountLab()。在 shared/ui.mjs 中，删除按钮执行删除函数，把返回数组赋给当前 records，再调用 render()。刷新初始化时，readRecords() 从当前专用键重新读取，只有键不存在才创建样本。空数组不会自动变回三条。



刷新前后的数据路径：新列表必须保存，刷新才会重新读到它

再看副本的 starter/core.mjs。这里是真实代码结构：

```
export function deleteRecord(storage, key, records, id) {
  const next = records.filter((record) => record.id !== id);
  return next;
}
```

它生成新数组，却没有调用已导入的 saveRecords()。这与“内存两条，存储三条”相互支持：删除被执行了，缺少的是把新状态留下。当前页面已经正确消失，再调用一次渲染并不会改写存储；等更久同样不会凭空产生一次写入。

最小检查可以直接对照同一 sample-1：删除后它是否仍在专用键中？本页“检查内存与存储”提供这次再次读取；开发者工具的站点存储面板可提供独立观察。检查记录 ID，不能只看数量。若存储也已不含它，但刷新仍恢复，就应继续查实际读取来源，不能套用本章这一行修复。

最小改动，还要守住失败时的行为

让 Codex 只补全缺失路径：

```
现有证据支持删除结果未写回存储。
只在删除函数返回新数组前调用已有 saveRecords(storage, key, next)。
不要增加定时器或重写 UI。检查此前有无无关补丁，
只撤回本次练习中确认无用的改动，不覆盖我其他修改。
写入失败时保留原列表并报告失败，不能显示持久删除成功。
完成后给出 diff，以及你实际执行和仍待人工执行的检查。
```

对应的最小差异为：

```
  const next = records.filter((record) => record.id !== id);
+ saveRecords(storage, key, next);
  return next;
```

保存发生在返回之前。调用方只有等函数正常返回才替换 records；若保存抛出错误，赋值未完成，UI 捕获错误并保持原列表。这条顺序让“未保存”成为可见失败，避免又出现只改当前页面的假成功。不能用“捕获所有错误后继续返回新数组”把写失败吞掉。

参考修复在 solution/core.mjs。对照时重点看这条因果链，不要因为参考文件更短，就用整目录替换自己的工作。你可能已经加入有效的说明或检查，它们不应随着无关补丁一起丢失。

在真正动手前，还可以把预期写成一个很小的实验：“如果遗漏写入是原因，补上这一行之后，删除后的存储应立即不再含原 ID；不必等刷新才知道。”这样修复不再只是编辑动作，而是一次会产生明确结果的判断。若存储依旧不变，就检查新代码是否加载、调用是否到达、异常是否发生，不能在没有区分这些情况时继续叠加第二个补丁。

修复后的检查，需要跨过同一条边界

在副本故障入口中重新恢复样本。删除“示例网站”，检查两份状态都不含 sample-1，然后刷新，确认它不再回来。仅观察删除后消失，仍是最初那个不足的验收。接着保留另外两条，新增“回归检查” https://example.com/check，刷新后确认新条目留下、被删条目没有恢复。

再逐条删到空列表，刷新后应仍为空。这能区分“删除保存正确”与“初始化把空列表当成从未开始”。最后输入坏网址，确认报错而不污染已有记录，再改成合法网址检查新增可继续。回归不必穷举整个应用，但应覆盖同一保存路径与相邻边界。

原案例的自动检查在仓库根目录执行：

```
node --test cases/D-phantom-delete/tests/delete.test.mjs
```

本项目在 macOS 26.6.2、Node.js v22.21.1 上实际执行，9 项通过：其中故障测试刻意确认 starter 删除后存储仍含原 ID，修复测试则要求再次读取不恢复；还覆盖最后一项删除、新增回归、写入抛错与重置隔离。故障测试通过，表示故障可复现，不能把它说成 starter 通过了产品验收。

原测试固定检查原案例两种版本。你修改工作副本后，不要拿原案例测试通过当作副本证据。可让 Codex 在副本另写一条回归检查：准备存储→调用自己的删除函数→再次读取→断言原 ID 不存在；再实际执行。浏览器点击、焦点与刷新另按上述流程观察，函数测试不替代这些步骤。

什么时候应改变路线

如果修复后遇到浏览器拒绝存储，继续加保存调用也没有用。保留失败信息，核对当前来源与权限；若当前环境无法满足持久保存，就明确标注未验范围，换可用浏览器或调整练习条件。若存储已正确、刷新却读错来源，沿读取路径继续取证。下一步必须由新证据决定，而不是维护最早的猜测。

本章不要求你背下 JavaScript，而是能要求 Codex 解释：什么被改变、什么被保存、刷新又从哪里读取。不断追加“务必认真”不能替代这个因果关系。让每轮尝试得到新信息，即使要几轮，也是在推进；如果补丁无法说明预期会改变哪次观察，就应先停下来。

动手：交付一次能被复查的纠偏

保留你的工作副本，完成一次调查与最小修复。给后来接手的人留一段简短记录：复现动作与原观察；被证据削弱的解释；实际修改的函数；删除→刷新与新增→刷新的真实结果；尚未执行的检查。对话若由你设计，用“示意”标明，不能填成真实工具日志。

完成标准是你能解释为什么原先的渲染或延时补丁无效，并用跨刷新的结果证明新的行动触及了真正路径。这种能力可以迁移到文件未写盘、配置被旧值覆盖、导出只存在内存等任务：先找到状态跨越边界的位置，再决定该修哪一步。

第 19 章：换路线、降范围与及时止损

你已经检查了几轮，任务却还没完成。此时最难的决定，往往不是下一行代码怎么写，而是是否继续当前路线。前面花了很多时间，会让人想再试一次；看到失败，又容易想把所有代码推倒重来。两种反应都可能绕开了真正的问题：下一步能带来什么新信息？

本章承接第 18 章的假设检查。我们以 Phantom Delete 已定位的保存故障为起点，比较继续修复、换验证路线、缩小范围与停止当前尝试。完成后，你应能写出一段有事实依据的路线调整说明，并让已有成果留下来，而不是只报告“还在努力”。

已经知道根因时，继续可能最便宜

案例 D 的 starter/core.mjs 中，deleteRecord 过滤出新数组并返回，缺少将结果保存的调用。solution/core.mjs 则在返回之前调用 saveRecords(storage, key, next)。两个版本的最小差异只有这一条写入。这是当前案例文件的事实，不意味着所有刷新故障都能用一行修复。

如果你已经按上一章证明“内存删除后为两条，存储仍为三条”，并定位到这条路径，就有充分理由继续做针对性修复。目标没有变化，问题范围已经缩小，验证动作也明确。换框架、增加数据库或重画页面，都不会提供比补齐保存路径更直接的证据。

在自己的 my-phantom-delete/starter/core.mjs 中继续，而不是改教材原始故障版。发给 Codex 输入框的指令可以是：

```
已确认删除后的数组没有写回本案例专用存储。
请只在当前工作副本补齐删除的持久化路径，保留现有校验与错误处理。
检查删除→重新读取不恢复，其他两条仍在；
再检查新增后重新读取、删除到空列表后重新读取。
暂不改页面样式。实际浏览器刷新若不能执行，单独报告待验。
```

这里继续的是一个已经有证据的方向，不是继续同一种无依据补丁。修改前后若出现新的存储异常，就重新检查保存调用是否真正成功，不能因为差异只有一行就跳过验证。

换路线时，保留原来的目标

下面是一个教学情境：工作环境没有可用的自动化浏览器工具，Codex 能运行 Node 测试，却无法亲自点击页面。它连续尝试安装浏览器依赖，每次都被组织网络策略阻止。这不是本次真实 Codex 多轮日志，也不是案例必须联网才能运行的声明。

此时继续改删除代码，并不能解决验证工具缺失。更合理的替代路线，是由程序测试检查数据读写，再由你用已有浏览器完成同样的删除与刷新验收。改变的是谁来执行检查，以及如何获得证据；“删除后刷新不恢复”的目标没有被降低。

案例 D 的逻辑检查可以在教材根目录的操作系统终端执行：

```
node --test cases/D-phantom-delete/tests/delete.test.mjs
```

它不下载软件、不需要 API Key，但需要 Node.js 18 或更新版本。测试覆盖教材的故障版与参考修复版，不会自动检查你的工作副本。若要验证副本，让 Codex 先核对测试的导入路径，在副本中调整对应测试目标；不要拿参考答案通过的结果替自己的改动背书。

人工检查沿用案例 README：在教材根目录运行 python3 -m http.server 4176 --bind 127.0.0.1 --directory my-phantom-delete，Windows 将 python3 换成 py -3；浏览器打开 http://127.0.0.1:4176/starter/，完成后在服务终端按 Ctrl+C 停止。服务未启动时先检查 Python 和端口，不把环境问题当成保存逻辑失败。

参考测试与手工操作一起，可以逐渐补齐证据。如果手工条件也暂时没有，交付应写“数据逻辑已检查，浏览器验收待完成”。替代路线不能靠改变措辞，把没有发生的观察变成通过。

降范围时，明确哪部分承诺改变了

再看另一个教学情境：修复删除期间，使用者提出“顺便加上跨设备同步、自动回顾与动画”。新增愿望不等于当前实现突然失去价值。你需要先判断它们是新的阶段，还是说明原目标本就理解错了。

如果使用者仍需要一个本机教学工具，可以先交付保存故障修复，延后同步、回顾与装饰。这是把本轮范围收回已约定的目标。保留删除与刷新的完整验收，不能用“先做一半”交付一个仍会恢复条目的删除功能。

如果使用者真正的问题是手机收藏、电脑读取，那么本地保存无法满足核心使用场景。你应说明现有模型的限制，比较可接受的替代流程，交由使用者决定是否开启另一项任务。不要自行引入外部账号或付费服务，把重要产品变化藏在技术调整中。

范围缩小需要使用者看得懂的表述，例如：“本轮交付本机删除修复，已保留的两条收藏不受影响；同步暂不实现，因此换设备无法看到这些记录。”它比“先上线 MVP”具体，也不需要用一大份审批表代替对话。

停止当前尝试，需要留下可恢复的位置

停止不是把目录删除，也不是宣称任务已经完成。它表示在当前约束下，继续同一路径没有可辩护的下一步。例如，必须由授权持有人开放的访问缺失，且没有等价材料；或者核心目标已变化，旧实现无法再回答当前问题。

遇到这类情况，先保存能复用的成果：可复现步骤、实际错误、已确认的根因、最小修改、运行方法、尚未验证的部分。别为了“干净”清空浏览器存储或撤销别人的修改。案例 D 的“恢复初始样本”会覆盖当前版本练习数据，只有要重新开始合成样本实验时才使用，不是通用的止损按钮。

你可以要求 Codex：

```
先停止当前工具安装路线，不再重复相同请求。
请保留已有实现和测试结果，记录阻碍实际发生在哪个动作，
哪些目标仍未验证，以及具备什么条件后能继续。
给出一个不依赖这项工具的可行下一步；没有时如实说明。
```

重要的是停止哪一条路线，而不是笼统地让全部工作永久结束。浏览器工具不可用时，正文、样本和数据测试仍可能继续；缺少必要授权的依赖动作则应等待相应决定。不要把等待条件写成“以后再看看”。

用信息增量判断，不用次数判断

一次失败就可能推翻假设：存储快照根本没变，已经足以质疑“只是重绘慢”。多次尝试也可能有价值：每次分别检验存储键、异常处理和初始化分支，确实在排除原因。次数本身不能区分这两种情况。

给每次继续一个具体理由：下一步检查哪项未知，它的不同结果将怎样改变选择。如果答案仍是“再试一次可能会好”，先暂停，回到事实与假设。已经投入的时间可以解释你为什么着急，却不能证明下一次相同动作更可能有效。

动手练习：作出一次可以解释的调整

请选择自己的真实案例，或使用下面的教学情境：删除遗漏写入已定位，参考修复的数据测试通过；你的副本尚未运行浏览器；自动化浏览器无法使用；使用者又希望本轮加入动画。写四句话：当前事实、选择的路线、保留与延后的成果、下一项可检查动作及继续条件。

参考反馈（离线展开）

一种合理选择是继续完成最小保存修复，用手工浏览器替代自动化验收，延后动画。理由是根因和修复范围已清楚，验证工具可替代，动画不影响删除目标。不能因参考测试通过就宣布副本通过，也不能把刷新验收删掉。若连人工浏览器都无法使用，应将该项保留为待验，说明恢复条件；这时停止的是无法完成的验证路线，不是删除已有成果。

一次好的路线调整，会让下一步比前一步更有依据。接下来在第 20 章判断：已有成果究竟足以支持怎样的交付结论。

第 20 章：他说完成了，但你怎么确定？

“已完成，所有检查通过。”这句话可能意味着文件已经写入，也可能意味着服务终于启动。等你真正删除一条收藏，刷新以后它又回来。问题不只是检查做得少，而是交付结论超出了证据能支持的范围。

第 5 章讨论开工前怎样确定终点。本章站在交付时刻：面对已经产生的文件、日志、测试和界面，怎样判断现在能交付什么，还缺什么。完成后，你应能写出一份简短、有实际依据的交付说明，也能识别“程序运行了”与“问题修好了”之间的缺口。

五种状态，不是一张统一通行证

先把常被混用的状态拆开。

状态	需要看到的证据	单独还不能证明
已写	实际文件与对应差异	语法正确或功能存在
已构建	该版本构建命令成功、产物确实生成	关键操作按目标正确
已运行	正确版本启动、入口可访问	删除后刷新不会恢复
按目标验收	约定输入与操作下，实际结果符合要求	所有平台、所有场景均正确
可继续维护	别人能定位变更、运行检查、理解限制和下一步	永远不会出现新问题

它们不是每个项目都必须依次盖章的五个门禁。案例 A、D 是原生静态文件，没有必要为了写“已构建”额外增加打包工具。可以记录“不需要构建；模块语法与实际运行分别检查”。教材阅读站需要静态构建，那是另一个产物的证据，不能代替

案例行为验收。

同样，不要求每轮同时证明整套产品永远可维护。当前改动的入口、原因、测试与限制能被下一位操作者理解，就已经比只有一句“完成”更可接续。

用两个案例看清证据错位

LinkShelf 的数据测试位于 cases/A-linkshelf/tests/core.test.mjs。它检查读写、校验、搜索和导出格式，包括删除最后一条后重新读取为空。测试使用程序化存储对象，能快速验证数据逻辑，但没有点击真实按钮，也没有证明浏览器已下载文件。

Phantom Delete 更容易暴露误解。它的测试同时导入 starter 与 solution：一个用例要求成功复现“内存删除、存储仍有记录”，另一个用例要求修复版的删除确实写回。因此，故障版相关用例通过，说明教学故障按设计存在，绝不意味着故障版已经可当作修复交付。

检查测试名称与断言，才能知道绿色结果的含义。看到测试总数和通过率，却不知道它检查的是哪个版本、哪个行为，仍不足以作出交付决定。若你修改的是 my-phantom-delete，而运行的测试导入教材的 solution，即使全部通过，也还没有检查你的修改。

先把“对象”和“证据”接起来

审视一份交付时，先找到实际产物。它是哪个目录、哪个入口、哪组修改？然后找支持它的命令与观察。最后再读结论，检查是否有跨越。

例如，“本机打开参考修复版”需要指向 D 的 /solution/；“检查我的修复”则需要指向工作副本的 /starter/。浏览器标签页看起来相似，不能省略版本确认。D 的两个原始版本使用独立存储键，切换后看到三条样本，也不能立刻判为上一版本的删除失效。

一次证据链可以很短：目标是删除后不恢复；变更位于删除保存路径；数据检查在目标版本通过；浏览器按固定顺序复现与回归；交付保留启动方法和未覆盖条件。每一环都回答上一环留下的问题，而不是堆积无关输出。

实际执行一条足够聚焦的检查

在教材根目录的操作系统终端运行：

```
node --test cases/A-linkshelf/tests/core.test.mjs cases/D-phantom-delete/tests/delete.test.mjs
```

需要 Node.js 18 或更新版本，无需联网安装依赖。编写本章时在 Node.js 22.21.1 实际执行，19 个测试通过、0 个失败。其中 A 为 10 个，D 为 9 个；D 包含对故意故障的复现断言。这个结果只对应当时的教材文件和数据层检查，不能给未来修改或工作副本自动背书。

还实际对 D 的 starter/core.mjs、solution/core.mjs、shared/ui.mjs 运行过 node --check，语法检查通过。它没有执行浏览器中的 DOM 事件，不应被写成“页面交互全部通过”。浏览器与移动端的集成验收需另有实际记录；当前最新范围以案例 README 和仓库 progress.md 为准。

如果只想复核 D，可以单独运行 node --test cases/D-phantom-delete/tests/delete.test.mjs。这有利于看清故障与修复的不同断言。测试失败时保留完整错误和执行目录，先判断目标文件与输入条件是否正确，再修改实现或测试。

浏览器证据为什么还需要动作顺序

D 故障版的新增，会保存当前完整内存数组。因此“删除一条→新增另一条→刷新”可能顺便把前次删除保存下来，掩盖原来的缺口。若交付记录只写“我试过删除和新增”，就无法知道关键故障是否真的被检查。

检查修复版或自己的工作副本时，先确认版本，恢复合成样本，再执行：删除“示例网站”→检查内存与存储都为两条→直接刷新→确认被删记录不再出现，另外两条保留。先不要混入新增。随后再单独新增并刷新，检查相邻功能；删除至空列表后刷新，确认没有重新播种。

按案例 README 在本机启动即可：教材根目录运行 python3 -m http.server 4176 --bind 127.0.0.1 --directory cases/D-phantom-delete，浏览器打开 http://127.0.0.1:4176/solution/。Windows 换用 py -3；停止时在服务终端按 Ctrl+C。检查自己的修复时将目录参数换成 my-phantom-delete，访问 /starter/，保持同一来源。

这些是可复现操作，不是本段已经替你执行的浏览器日志。亲自观察后，写下结果；出现差异则保留数据和动作，回到第 18 章检查原因。不要为了得到符合预期的截图，先清除证据。

交付说明只保留能帮助复查的内容

可以请 Codex 汇总：“实际改了哪里、怎样运行、执行了哪些检查、每项支持什么结论、哪些仍未验证。”不需要附上所有思考和每次列目录的输出。读者需要的是可定位的产物和证据。

下面是一份填写示例，并非你的工作副本已经通过的证明：

```
产物：my-phantom-delete/starter/core.mjs 中的删除保存修复。
变化：删除后的新数组先保存，成功后再返回给界面；保留专用键与校验。
数据检查：填写实际针对此副本执行的命令和结果；未执行就写待验。
浏览器：填写版本地址、删除→检查→直接刷新及相邻回归的真实结果。
运行：从教材根目录按本案例 README 的工作副本命令启动。
限制：合成练习数据；未检查的系统或权限场景明确列出。
下一步：若浏览器待验，先完成该动作，再决定是否达到本轮终点。
```

“可维护”也落在这些内容里。下一位操作者能够知道为什么加这一行、保存失败时怎样处理、测试在哪，才有机会继续。如果只留下大篇赞语，没有入口和差异说明，维护者仍需重新调查。

动手练习：识别一次虚假的完成感

下面是教学模拟交付：“已彻底修好删除。Python 服务启动正常，首页返回 200，Node 测试全绿。页面截图很美观，功能全部完成。”补充事实：测试导入参考 solution，实际修改在工作副本；截图只显示初始三条样本，没有记录删除或刷新。

请写一段不超过五句话的回复，指出当前能确认什么、不能确认什么，以及最小补充检查。不要要求把所有功能重新开发一遍。

参考反馈（离线展开）

可以确认服务和某个页面可访问，参考修复版的数据测试通过；还不能确认工作副本的修复有效。先核对服务目录、浏览器地址和测试导入目标，随后对该副本执行“删除→检查→直接刷新”，确认另一条记录保留。再检查新增与空列表刷新。若这些尚未发生，交付应收窄为“已实现、验证未补齐”，而不是继续写“彻底修好”。

交付判断不要求你掌握每行代码，却要求结论始终跟着证据走。现有证据足以支持哪一层，就清楚交到哪一层；缺少的观察变成下一项具体工作，而不是被“已完成”三个字掩盖。后续内容见课程目录。

第 21 章：让有价值的方法成为 Skill

第一次处理 CSV，你需要说明空值怎样算、重复怎样处理、报告放哪里。第二次换一份同类数据，又从头解释这些事情。如果这套方法已经实际可用，重复描述就开始消耗注意力；但把整段聊天保存下来，也未必能帮下一次任务。

Skill 将可复用的方法放进一个有名称、适用描述和正文的目录，可以按需要附带脚本、参考资料或模板。它让 Codex 在合适情境读取那套方法，不会自动替你运行脚本，也不赋予访问外部服务的权限。本章使用案例 C 制作一个最小 Skill，并检查它能否支持第二份输入。

先确认值得复用的究竟是什么

准备 Python 3、案例 C，以及能读取本地文件的 Codex 会话。无需账号以外的新连接、API Key 或全局安装。先看 cases/C-csv-insight/README.md，理解四字段和本案例口径。

这里有两种不同的复用：report.py 负责确定性解析、统计和写文件；Skill 负责指导什么时候使用它、怎样确认输入与口径、如何独立核对、失败时怎样报告。若你的重复工作只是运行一条已知命令，脚本入口就够了。只有需要反复作出这些工作判断时，Skill 才开始有价值。

不要把某次访问总数写成永恒规则。2155 是当前样本的结果，不是所有报表应有的答案。也不要把某次错误变成“每次先做五轮审查”：保留真正影响决策的约定，删去没有帮助的仪式。

一个文件，先把边界讲清楚

本仓库示例位于 extensions/csv-report/SKILL.md。它的元数据是：

```
---
name: csv-report
description: 为本教材 CSV Insight 案例生成并核对离线渠道汇总；适用于用户指定该案例数据或相
  同字段的报表任务，不用于任意 CSV 清洗、因果分析或预测。
---
```

正文保留定位案例、确认输入输出、使用已有脚本、核对口径、独立检查和交付范围。它没有把案例的整份代码复制进来，也没有发明另一套汇总实现。后续脚本更新时，应同时检查这些指引是否仍准确。

name 和 description 帮助发现与选择，完整正文按需要加载；这些结构与激活方式按 2026-10-25 的 OpenAI Build skills 文档核对。技能描述若写成“处理所有文件与所有项目”，就很难正确选择；若塞满实现步骤，又会在还没用它时占据不必要上下文。

这个示例的存放位置有意选在 extensions/，供读者检查与修改。它不属于本仓库已经安装的自动发现目录。文件存在、格式有效、当前会话实际读取、未来会话自动选中，是不同状态，不能混写为“安装成功”。

先显式读取，再做一次实际复用

将下面任务发给 Codex，路径从教材根目录解析：

```
请读取 extensions/csv-report/SKILL.md 并按其中流程处理案例 C。
输入是 cases/C-csv-insight/input/activity.csv。
输出选择一个新的练习报告路径，不覆盖我需要保留的结果。
先确认当前脚本与口径，再生成并独立核对。
报告实际命令、关键数值和未验证范围，不安装全局 Skill。
```

这是一条可以在能读文件的会话中使用的明确任务，即使该客户端尚未提供技能选择器，也能执行方法。它是显式加载文件的复用，不宣称已经验证原生自动触发。

案例 C 的运行目录是 cases/C-csv-insight。在操作系统终端可观察对应步骤：

```
python3 -B report.py --input input/activity.csv --output output/skill-full.md
python3 -B verify_totals.py --input input/activity.csv --report output/skill-full.md
```

Windows 把 python3 换成 py -3。同意这个新输出路径后再执行；若已经有同名成果，另选名字。完整样本按约定计入 21 条，访问 2155、注册 267，空值排除 2 条、重复排除 1 条。要同时看到独立核对结果，不能只读生成脚本的成功提示。

然后给同一会话第二项任务：沿用 Skill，改处理 input/probe.csv，使用另一个输出 output/skill-probe.md。这份小输入预期访问 18、注册 3，计入 2 条、空值 1 条、重复 1 条。观察它有没有仍引用上一次输入、沿用旧总数，或漏掉独立核对。这些实际分歧比询问“这个 Skill 好不好”更有用。

本项目已按这套指引读取案例并实际运行两份输入的生成与独立核对，结果与上面一致。完整输入还进行了重复字节比较。相关命令与限制在 extensions/verification.md，没有运行新的付费模型评测或声称所有客户端都会自动发现本技能。

格式有效之后，还要看行为

本示例使用 skill-creator 的验证脚本实际检查，返回 Skill is valid!。它说明元数据、命名与结构符合检查规则，不证明所有决策正确。读者若有该技能，可以请它验证自己的副本；没有也可先人工核对元数据与引用路径，不必为了练习安装一个庞大环境。

行为上重点检查三处：它是否识别任一必需字段空缺应排除整行；是否把非法日期或数值格式当作整次失败，而非偷偷补零；是否保留旧输出并说明本次失败。这里的策略来自案例约定，不可擅自套用到别人的业务数据。换数据领域时先核实含义，再决定复用哪些部分。

如果第二次仍要补很多解释，先判断是描述没说准、指引遗漏关键知识，还是根本不属于同类任务。只改导致当前分歧的那一处，不要把每次聊天里的临时要求都追加进技能正文。

需要自动发现时，再选择安装范围

当期官方文档将项目技能位置列为 .agents/skills，用户级位置列为 ~/.agents/skills。不同版本与管理环境可能有额外兼容位置，应看当前客户端文档，不从本示例的 extensions/ 位置推断自动加载规则。若只服务一个练习，优先在你自己的工作副本考虑项目范围，避免影响所有项目。

本章没有替你复制到这些位置。若你决定安装，可让 Codex 先检查该目录是否已有同名技能，说明具体写入位置与发现方式，再做有范围的复制。不要覆盖已有版本，也不要把安装权限误当成发布或上传数据的授权。技能和工具的职责区别可继续阅读 OpenAI Skills 概念说明。

动手：留下一份真正减少解释的方法

在副本中精简 csv-report，保留对你下一次任务有用的信息。用完整样本和小样本各做一次，记录是否需要额外纠正，以及哪一句指引产生了实际作用。再拿一个字段不同的 CSV 问它是否适用，观察它是否先核实，而不是直接套口径。

完成标准是一份短技能、两次真实执行结果与准确的使用状态。可复用的价值来自减少重复判断、保留关键约定，并在不适用时停下来，而不是让一个文件承担所有未来任务。

第 22 章：MCP 与插件，让工具进入工作流

你问 Codex“仓库默认分支是什么”，它可能从本地 Git、GitHub 工具、网页或旧聊天中回答。同一句答案，来源不同，新鲜程度和可支持的结论也不同。扩展能力的第一步不是安装尽可能多的工具，而是明确缺少哪份信息、应从哪里取得，以及取得后能证明什么。

本章解释 MCP 与插件怎样进入任务，观察一次本制作环境的真实只读调用，并提供无需外部账号的替代练习。完成时，你能选择是否需要连接，核对调用范围，并把失败归到合适的位置。

先分清四件事

MCP（Model Context Protocol）是一种让客户端与工具、数据服务协作的开放协议；它约定如何发现能力与交换调用。MCP 服务可以提供工具、资源等内容。协议不是你的账号，也不保证某项数据一定可读。

插件是可安装的能力包，可以包含 Skill、MCP 服务，或者两者。Skill 描述怎么完成一类工作，工具则接收结构化参数并执行特定读取或动作。账号连接与服务权限决定它能访问什么；当前会话可见的工具再决定你眼下能调用什么。以上边界按 2026-10-25 的 OpenAI MCP server 说明和 Skills 说明核对。

例如，“读取指定仓库元数据”是一个工具动作；“先核实仓库，再比较改动，最后形成审查说明”是一套工作方法。把方法写进 Skill，不会自动获得 GitHub 权限；安装了插件，也不等于应该使用它提供的每一种写入动作。



Skill提供工作方法、MCP工具执行指定动作，账号与宿主权限分别限定可访问资源和允许的操作

真实调用：只回答一个具体问题

本制作环境已提供 GitHub 插件。本章实际通过它的 get_repo 能力，请求：

```
{"repository_full_name":"liamamilin/codex-mastery"}
```

调用返回仓库全名 liamamilin/codex-mastery、默认分支 main、可见性 public 和非归档状态。读取没有创建仓库、提交、评论或修改配置。提取的公开字段保存在 extensions/connection-evidence.json，其中明确标注日期与来源。

这能证明本制作环境当时能够读取这份仓库元数据。它不能证明读者已连接同一账号，不能证明仓库里每个文件可读，也不能证明 Pages 已部署。返回中的仓库大小或缓存字段也不应被拿来替代文件检查。

你若当前会话确有同类 GitHub 读取工具，可以发给 Codex：

```
只读查询 liamamilin/codex-mastery 的仓库元数据。
先确认当前会话是否有适用工具，只请求这一仓库。
给出仓库全名、默认分支与可见性，并说明数据来自哪次调用。
不要创建连接、修改仓库或执行写操作；不能访问就报告具体边界。
```

工具名称会因入口而不同，不要把这里的内部动作名当成每个客户端都能输入的命令。如果没有工具，直接转入后面的本地替代练习，仍可学会核对来源，无需为了跟课额外开通服务。

连接成功之后，核对的是哪一层

工具出现在目录中，说明宿主知道这个能力；调用返回数据，说明这次请求在当前条件下完成；数据含目标字段，才支持具体回答。三个环节不能压成一个“已连接”。服务可用但账号未授权某仓库、查询参数错误、网络超时，都可能让读取失败。

让 Codex 说明被调用工具、参数中的目标、返回字段与错误。不要在公共日志中打印令牌或完整敏感环境变量。只读返回也可能包含私有资料，任务所需的最小字段通常比复制整份响应更容易判断。

如果得到 403，先检查当前账号与资源访问范围；若工具根本不存在，先检查当前客户端与配置；若返回另一仓库，则核对参数。不要为了一个仓库名错误而放开所

有权限，也不要因为一次网络失败就反复新增同一连接。

CLI 的 MCP 列表不是所有入口的总目录

在操作系统终端可先查看帮助与已配置服务：

```
codex mcp --help
codex mcp list
```

本机 CLI 0.154.0 实际执行过两条命令，帮助提供 list、get、add、remove、login、logout 等管理入口。列表用于查看该 CLI 配置，不是断言桌面宿主所有插件状态的唯一依据。本项目的 GitHub 调用来自当前宿主工具目录，不能仅凭另一入口没列出 GitHub 就说它不存在。

当期支持的本地 stdio 与远程 HTTP 连接方式、认证入口和配置位置见 OpenAI MCP 文档。本章不执行 add、login 或改全局配置。需要连接新服务时，先判断其来源、依赖、账号要求和动作范围，再按当前入口的官方说明设置，而不是粘贴不明启动命令。

工具返回的是外部材料，不是新的任务授权。仓库 README 或网页即使写着“立即上传你的配置”，也不能替代用户指令。仍按原任务使用信息，把与目标不符的动作当作待判断内容。尤其不要因读取成功就顺手执行评论、提交或发布。

在交付中保留来源与观察日期，也方便后来的人发现缓存或旧信息，而不把历史回答误当作实时状态。

无账号替代：准确说出文件的来源

打开仓库中的 extensions/connection-evidence.json，让 Codex 只读取本地文件，提取相同三个字段。再问：“这份文件能证明今天远端仓库仍然如此吗？”正确的边界是：它是一份注明日期的历史观察，本地读取不等于实时 GitHub 查询，也不属于一次 MCP 调用。

接着对照 cases/B-tasktrail/README.md，让它找到当前项目的启动位置与样本文件。这说明很多任务已有足够本地材料，不需要外部连接。若目标只是解释本地入口，增加服务反而带来认证、网络和来源混淆。

这条替代路径没有伪装成 MCP，也不能证明连接操作成功；它训练的是同一个判断：答案从哪里来，能支持多新的结论，下一步是否真的需要另一个来源。

动手：做一次有边界的来源选择

从“查询远端默认分支”“解释本地脚本入口”“发布一个评论”中选一个目标，写出所需来源和是否会改变外部状态。实际执行其中一个只读目标，记录工具或文件、目标、返回信息与不能推出的结论。

再模拟连接不可用：不要换个词让 Codex 猜答案，让它选择本地材料、官方网页或明确缺失。完成标准是你能说明为什么当前任务需要或不需要连接，并能把“配置过”“本次读到了”和“允许写入”分开判断。

第 23 章：把重复操作变成脚本与自动化

你已经让 Codex 生成过一次 CSV 报告。下周换一份同格式数据，如果还要重新请模型计算每个总数，就把本来可以固定的工作交回了不确定的过程。更合适的分工是：让 Codex 帮你制定、维护和解释方法，让程序执行明确的读取、统计和核对。

本章从案例 C 的两条命令出发，形成一条可重复运行的本地任务。完成时，你会确认相同输入是否得到相同结果，坏输入是否停止，以及结果文件在失败后应如何解释。调度属于下一步选择，本章不替你创建定时任务或外部发送流程。

先把重复的边界圈出来

准备 Python 3 与完整案例 C，不需要 pip、网络服务或模型 API。先读案例 README，确认四字段、空值排除、完全重复只计首条，以及非法日期或数值使整次失败的约定。自动化一个尚未确定的口径，只会更快地重复错误。

当前稳定部分是：读取一个指定 CSV，生成一个指定 Markdown，再用独立脚本核对。需要判断的部分是：新数据是否仍符合定义，排除记录是否合理，数值支持怎样的结论。不要把“脚本返回成功”扩展成“业务判断正确”。

在案例 C 目录，基础动作仍是：

```
python3 -B report.py --input input/activity.csv --output output/summary.md
python3 -B verify_totals.py --input input/activity.csv --report output/summary.md
```

Windows 将 python3 换成 py -3。第二条命令必须核对第一条的同一输入与输出，不能不小心检查上次报告。运行前确认哪些旧结果需要保留，另选练习路径，不把唯一一份业务报告当作试验材料。

把顺序与失败规则交给小脚本

本项目提供 extensions/run-csv-report.py，只封装已有流程：用当前 Python 执行 report.py；成功才执行 verify_totals.py；任一步非零退出就停止。它没有复制统计逻辑，也没有在运行时调用模型。

在教材仓库根目录执行，输出选择一个新的练习位置：

```
python3 -B extensions/run-csv-report.py --input cases/C-csv-insight/input/activity.csv --outpu
  t my-reports/summary.md
```

Windows 对应为：

```
py -3 -B extensions/run-csv-report.py --input cases/C-csv-insight/input/activity.csv --output m
  y-reports/summary.md
```

如果已有同名文件且你想保留，先改输出名。包装脚本从自身位置找到案例实现，将输入输出解析为实际路径，因此不用先切入案例目录；两条子命令使用相同解释器和文件。它以参数列表启动程序，不把输入文件名拼进任意 shell 代码。

预期先看见生成结果，再看见独立核对通过。对完整样本，访问 2155、注册 267，计入 21 条，空值 2 条、重复 1 条。最后的完成提示只表示本地生成与核对结束，没有上传、发消息或安装调度器。

实际运行两次，比较的是文件

保持输入和口径不变，执行同一条包装命令两次。第一次后把报告另存一份，再用编辑器比较两份文本，或让 Codex 比较字节与 SHA-256。不能只因为两次终端都说成功，就推出内容完全一致。

本项目在 macOS、Python 3.10.13 的系统临时目录中实际执行两次，独立核对均通过，输出字节相同，SHA-256 为：

```
1f324ecf2250998285cd71b57aeda07cd67ce1281c3d386485b9c41c03b11c12
```

这证明当前样本与实现的这次重复结果。它不是所有未来输入都正确的证明，也不要求你在改数据后还得到同一个哈希。报告刻意不加入每次变化的时间戳，让相同输入的差异更容易判断。

再在自己的输入副本改一条非重复记录，保持注册不变，把访问增加 7。若其它口径不受影响，报告总访问应增加 7。不要改重复对中的一条：那会改变去重关系，结果不再只是数值加 7。输入扰动能检查脚本是否真读了新数据，而不是不断返回固定答案。

失败时，旧文件仍在不等于本次成功

在临时输入中把访问数改成 no-number，保留原报告再运行。预期生成失败、退出码非零、旧报告字节不变，包装脚本不继续报告独立核对成功。本项目实际执行了这条分支，得到退出码 2，旧内容保留。

保护旧文件有用，但也留下一个容易误用的情境：目录里还看得见报告，有人便当作这次新数据的成果。应把命令状态与输入身份一起记录，失败后明确写“旧文件保留，未生成本次有效报告”。不能为了让自动化显示绿色，就忽略异常或把错误数值替换成零。

如果生成成功但独立核对失败，输出已存在，却不能交付为已核对结果。保留输入、报告和错误信息，调查口径或实现差异。不要让后面的发布动作无条件接着执行。这里的停止规则来自具体依赖：后一步需要前一步的有效产物。

脚本、批处理与调度有不同职责

脚本定义怎样做；调度决定何时做；模型在需要解释或处理新情况时参与。若只是每天同格式求和，不需要每天启动一个新模型重新设计统计方法。若输入规则经常变化，可以把异常留给人工或 Codex 判断，而不是给脚本无限的自动修复权限。

Codex CLI 当前还提供非交互 exec，可用于需要模型参与的批处理；本机只核对了 codex exec --help，没有运行新的模型任务。其参数、输出与运行边界见 OpenAI 官方 CLI reference。它会使用相应模型资源，不能与此处无模型调用的 Python 任务混为一谈。

真的需要定时运行时，先明确执行机器是否会开机、输入什么时候准备好、重复触发会怎样、输出谁会读取、失败通知给谁。避免两个任务同时写同一输出，必要时先用不同文件名保留每次结果。先手工运行通过，再按你选择的调度环境设置；本书不会因介绍自动化就自动开通或替你部署。

动手：交付一条能解释重复与失败的命令

在自己的副本选一个新输出位置，完成同输入两次运行、一个明确的输入扰动、一次坏输入失败。记录运行目录、命令、字节比较、数值变化和旧文件状态。让 Codex 解释其中哪些步骤需要判断，哪些已经可以交给确定性程序。

完成标准是后来的人能只运行同一入口复现结果，遇到失败知道该停止在哪里，且不会误把旧报告当作新成果。把重复工作固定下来之后，你才能把注意力留给真正变化的任务。

第 24 章：模型、推理强度、时间和成本

“把设置全部调到最强，会不会最省心？”有时更深的分析能帮你处理跨文件问题，有时等待更久只是让一个简单任务花更多资源。如果缺的是运行权限、正确数据或明确目标，提高推理强度也不能替你补上这些前提。

本章不提供固定型号排行榜。模型、可用配置、套餐与使用规则会变化；你要学会的是为当前任务选择资源，并用结果判断这项选择是否值得。完成时，你会留下一份任务配置说明，或一组有边界的小样比较；没有实际测量的地方就保留未知。

先区分你在调什么

模型是执行任务的基础能力；推理强度影响它投入多少分析；工具与权限决定它能获得信息、执行哪些动作；上下文决定它正在依据什么。更强的模型不能读取未提供且无法访问的文件，更多分析也不会把被拒绝的存储写入变成成功。

截至 2026-10-25，OpenAI Models 文档说明可以在相应客户端选择可用模型并调整推理，较高强度可能改善复杂任务，但通常花更多时间与 token；具体可用项取决于账号、客户端和工作区。这个关系是配置方向，不是对你手头任务的性能保证。

CLI 交互会话中的 /model 是当前官方文档列出的选择入口；它输入在 Codex 会话里，不是操作系统 shell 命令。App 或 IDE 用当前界面可见的模型与推理控制。先观察自己能选什么，不要照抄另一账号截图中的选项，也不要为了跟课升级套餐。

从任务的难点选择起点

把当前任务写成一个可检查的结果，再问难度在哪里。是材料多但规则清楚，还是需求含糊、因果路径未知？是允许快速返工的小改动，还是错误会影响许多数据的变更？验证越困难，越需要把调查、检查与人工判断算进资源安排。

任务	主要不确定性	可以先尝试的投入方式
为已有命令补一段启动说明	路径与参数是否准确	当前默认配置，读取实际帮助与文件后核对。
按已确认口径重复求 CSV 总数	输入是否符合格式	运行现有脚本与独立核对，异常时再让 Codex 调查。
修复跨刷新删除恢复	状态从哪里写、从哪里读	先保留复现与调用路径，证据不足再增加分析投入。
陌生仓库的多文件改动	关联范围、兼容行为与验证缺口	给足调查上下文和阶段结果，按实际难点考虑更深分析。

这些是任务配置建议，没有暗含任何两款模型的实测比较。尤其注意第二行：减少模型调用不等于降低质量。确定性脚本已经可靠完成的部分，不必每次重做推理。

更深分析之前，先排除资源以外的问题

如果 Codex 一直改错文件，检查工作目录；如果它反复编造字段，检查是否提供真实样本；如果修改总是无法验证，检查环境与验收路径。这些问题不解决，再换配置也可能只是更长时间地走错方向。

可以发给 Codex：

```
当前任务是修复删除后刷新恢复，先说明阻碍来自哪里：
缺失上下文、执行权限、证据不足，还是需要更复杂的分析？
请引用已经观察到的事实，提出最小下一步。
不要为了尝试就增加无关工具或自动切换付费资源。
```

这一步帮助你把配置变化对应到真正瓶颈。若已有足够证据只差一行保存，提升强度并不是必须；若多个数据源相互覆盖、原假设连续不符，给更多分析空间才可能有明确用途。仍然要回到可观察的检查，而不是相信设置名称。

比较配置时，固定题目与完成条件

若你愿意使用现有额度做比较，选一个小而有代表性的任务，准备相同起点的两个独立副本。保留同样的需求、上下文、工具边界与验收动作，只改变一个因素，例如推理强度。不要把一个已看过答案的长会话与全新会话直接当作公平比较。

记录的是完整交付过程：从开始到满足完成条件的时间、需要人工纠正的次数、实际改动、通过哪些行为检查，以及客户端真实显示的用量。没有用量明细就记“未知”；不能用回复长度猜精确 token 或金额。模型第一次回复快，但需要多次返工，未必更快完成任务。

对 LinkShelf，可用“坏网址拒绝且合法输入恢复正常”的小修改；对 TaskTrail，可用保持默认行为的筛选任务。给两份结果盲看行为与 diff，避免只因某份解释更长就判断更好。一次结果只能形成当前样本下的线索，不能推出所有代码任务的排名。

本书制作没有为本章新增付费模型基准，也没有测得某配置快多少或便宜多少。你可以完成后面的无额外调用版本练习，仍然掌握比较设计与选择依据。

把成本算到交付，而不只算一次回复

资源成本包括使用额度或费用，也包括等待、人工核对、返工、工具配置和合并复杂度。一个更便宜的单次响应若漏掉核心检查，可能把成本转移给你；一个复杂方案若引入不必要服务，也会留下长期维护成本。

价格与额度应按登录方式和当前官方计费页面核对，API 价格不能直接解释为客户端订阅内的一次任务价格。本章不提供跨入口换算，也不推断你的剩余额度。若需要具体预算，先明确使用入口、模型、计量方式和任务样本，再做实际估计。

同样不要把更高推理设置等同于更多工具权限。选择配置与批准外部动作是两个决定。模型输出再自信，也不能代替对发布、数据覆盖或外部发送的范围判断；这些约束应在实际任务里保持清楚。

动手：给一次资源选择写出依据

不增加任何模型调用也能完成第一版：选 CSV 重复汇总或删除故障，写下完成条件、主要不确定性、当前可用工具、打算用默认配置还是更深分析，以及什么现象会让你调整。为比较设计两个相同起点，列出实际应收集的指标，不填写想象数值。

若你随后自行执行比较，把测量与推断分开，补上客户端、日期、配置、实际用量边界和通过的行为。完成标准是你能解释“为何这次值得投入这些资源”，并在新证据出现时改变选择。长期有用的是这个判断过程，具体型号与设置只是在某个时间点可用的工具。

PART 08 / 长任务、协作与真正的掌控

第 25 章：跨会话、跨阶段推进一个项目

LinkShelf 第一版已经能保存链接，你又加入搜索，接着准备整理导出。项目看似在前进，但一个新会话把搜索当成未实现；旧记录又说“所有测试通过”，却没指出它检查的是哪一版。随着阶段增加，最危险的不是忘记一句对话，而是把不同时间的事实拼成了一个从未存在的状态。

第 9 章已经教你留下短指引与接续记录。本章进一步处理阶段之间的变化：什么结果足够结束上一阶段，什么证据会因改动失效，怎样让新会话继续一项尚未完成的工作。终点是一条能够中断、核查和继续的项目路线。

让每个阶段留下独立可用的东西

阶段不是“研究三天、开发五天”这样的活动清单。对 LinkShelf，可以把第一个阶段定为新增、刷新保存、删除；第二个阶段是标题或网址搜索与无匹配反馈；第三个阶段是 JSON 导出。每段结束时都有可操作结果，后续暂时停止也不会只留下一半架构。

当前教材参考成品已经组合了这些能力，但没有导入、标签或云同步。自己的 my-linkshelf 可能只完成其中一部分，必须看实际副本。不能因为目录里存在 solution，就在项目记录中把三个阶段全部勾选。

阶段划分也不等于每轮都从零重做验收。新增搜索后，检查筛选、清空与保存后的显示衔接，再确认数据没有被筛选改写。修改导出格式时，重点检查输出与当前收藏一致，同时保留可能受影响的保存路径。你是在根据变化选择相关证据，不是在每次启动都重复全书。

保留阶段基准，再记录本轮变化

一份长期记录需要让人看见时间关系。“保存通过”后面应有对应目录、日期、命令或操作；后来改动了保存函数，就不能继续把旧通过结果当作新版本证明。它仍然有用，说明过去哪个状态可用，但适用范围已经变化。

可以在工作副本的 progress.md 中保留一小段阶段基准，再追加本轮变化。下面是记录示例，不能直接当作你的运行日志：

```
阶段基准：基础版新增→刷新→删除→刷新已在本机固定来源检查。
本轮变化：为搜索增加无匹配提示，保存成功后清空旧关键词。
已有证据：填写实际修改文件、命令与观察；不要沿用参考成品的结果。
当前缺口：搜索中新增后的键盘焦点尚未检查。
下一步：在相同浏览器完成该动作，再判断本轮是否可交付。
```

它不要求保留每次无效尝试。已被推翻的假设留一句原因即可，例如“重绘延时未触及存储写入，已停止该路线”。这样新会话既不会重复旧补丁，也不会被数十屏日志淹没。

实际命令的完整输出可保存在相邻验证文件里，状态入口只引用关键结果和位置。记录与代码发生冲突时，以进一步检查解决，不让文件中的一句旧声明压过当前事实。



跨阶段接续循环：保留目标、已有产物和验证缺口，新会话核对后执行下一步，再更新状态

续作之前，先验证最容易过期的前提

长任务重启时，不必重新研究所有文件。先查足以影响下一步的条件：当前目录是否仍是目标副本，变更是否已落盘，数据格式是否改变，上次服务是否仍运行，准备执行的测试是否针对当前版本。

例如浏览器还开着 localhost:4174，服务却已被换成另一个目录，页面标题相似，很容易把旧页面当成新版本。相同端口不证明相同文件。先确认终端中的服务目录和浏览器地址，再继续验收，比直接添加另一项功能更节省后续排查。

可以给新会话这段任务，发到 Codex 输入框：

```
请读取当前副本的项目约定、阶段状态与必要代码。
先核对本轮要继续的实现和证据是否属于同一版本，
只重查会影响下一步的前提，不重复已确认且没有变化的调查。
说明当前最小缺口、怎样检查，以及哪些旧结论需要重新验证。
确认后继续这一项工作，不擅自扩展下一阶段范围。
```

如果你只想检查接续是否正确，把最后一句改成“先不修改文件”。这取决于当前意图，不需要每次新会话都固定采用同一种模式。真正需要你决定的是目标变化或必要授权；可以从材料中查明的事实，应由 Codex 先调查。

在半完成处中断，也能留下好入口

理想状态是在阶段完成后休息，现实却可能在测试失败、终端断开时中断。此时最有用的记录不是“继续完善”，而是最后一个可复现动作。

假设导出文件可以下载，但你还没确认里面的链接是否等于当前收藏。留下导出文件路径、样本数量、已执行动作和缺失比较即可。不要在没有核对时写“导出完成”；也不用撤销已经可用的下载代码，把工作退回起点。

如果中断发生在修改过程中，记录哪些文件有未完成变化，哪些测试失败，错误指向什么。保留用户原来的修改，避免用整仓库清理来获得表面整洁。下一位操作者应能知道从哪里继续，也能选择暂时不接纳半成品。

案例 E 的 verification.md 提供另一种真实状态：材料已经备齐，文本工具基础测试通过，但毕业边界仍未修复。这样的交付可以继续推进，因为它没有把“起点可运行”写成“毕业题已解决”。

目标改变时，更新路线而非堆叠旧计划

项目变长后，最初的计划可能不再合适。如果用户改为优先手机收藏，原先本机浏览器的阶段路线就需要重新解释。保留已经验证的本机能力，再说明新目标需要哪些条件与取舍，不把旧目标和新愿望不加区分地叠在一起。

本书制作本身经历了从前三章纵向切片，到扩展正文与独立故障案例的过程。各批次按文件范围落盘、返回真实检查与未验证项，再由主线接入。这能说明成果如何接续；它不能证明每次分工都更快，也不能替读者完成新会话练习。

阶段计划只服务于下一段可用成果。若维护计划所花的时间超过它减少的重复工作，就收短记录：保留当前目标、可靠基准、本轮变化、缺口与下一步，删除已经失效的待办。

动手练习：从半完成状态重新出发

选择自己的 LinkShelf 或 TaskTrail 副本。在一项真实改动中暂停，把当前产物、基准、最后一次实际检查和一个缺口写清楚。保存后开启新会话，只提供副本与接续任务，不粘贴旧聊天全文。

观察它能否找到正确版本，是否把未验事项误读为通过，能否选择足以推进的下一项动作。你亲自复核一条关键结论，再更新记录。如果没有实际开启新会话，就把本次练习记为“接续记录已准备，恢复尚未验证”。

参考反馈（离线展开）

成功的续作不一定立即增加功能。发现导出检查针对旧文件，重新运行并补齐一次比较，也是在推进。只有“已阅读记录，将继续优化”的回复不够；你需要看到当前目录、对应产物、适用证据和具体动作连起来。若新会话先重复实现已有搜索，说明状态入口仍不足以支撑正确判断。

长期掌控依赖于可恢复的成果状态，而不是一条永远不断的聊天。下一章讨论什么时候值得把这些工作交给多个 Agent。

第 26 章：什么时候值得使用多个 Agent

接手 TaskTrail 时，你想同时弄清入口、检查测试、审阅风险。多个 Agent 听起来很合适，但如果三个人一起修改 tasks.py，你可能得到三份各自合理、互相覆盖的答案。并行增加的是执行单元，协调问题不会自动消失。

本章训练分工判断。完成后，你应能选择一组可独立推进的任务，说明各自文件范围、返回证据与整合责任；也能在不值得时，明确选择单 Agent 顺序完成。

先看依赖，再看人数

两项工作只有在不需要等待彼此的中间结果时，才适合同时推进。调查 TaskTrail 的运行入口，与检查当前基础测试覆盖哪些输入，可以并行读取同一版本；实现新筛选之后的最终审阅，却需要等待实际改动，提前审阅只能检查旧版本。

你可以先让两个只读调查返回文件路径和不确定项，再由主 Agent 实施，完成后请其中一个检查变化。这比把“调查、实施、最终审阅”同时启动更符合依赖。并行的价值来自等待时间与认知工作能够拆开，不能靠多开几个窗口制造。

任务过小时，分工说明和汇总可能比实际修改还长。改一个明确文案、补一条已定位保存调用，通常没有必要拆成多 Agent。复杂问题也未必适合立即拆分：若大家还不清楚目标，先由一个 Agent 调查关键事实，往往更容易形成可分配的边界。

当前客户端怎样表达分工

2026-10-25 实际打开核对的官方子 Agent 文档说明：本地 Codex 可按直接要求或适用项目/Skill 指引开展子 Agent 工作；App 与 IDE 显示活动，交互 CLI 可用 /subagents 查看线程。子 Agent 会额外消耗模型与工具资源，继承当前沙箱设置；文字要求“只读”并不自动创建另一套只读沙箱。

这里区分官方资料与本机验证：本书制作实际使用了四个 Agent 参与分工，并按文件范围整合；没有做单 Agent 对照计时，也没有证据支持固定倍数的提速。本章没有另行启动一套 CLI 交互来实测 /subagents。入口与账号支持有差异时，以当前客户端和官方说明为准，不需要为练习开通新付费服务。

在已有任务中，可以直接说明独立任务、等待关系和交付形式。下面是教学分工任务，按自己的目录改写后发给 Codex：

```
请用两个子 Agent 只读调查 my-tasktrail 的同一当前版本。
甲：确认入口、数据路径与可运行命令，返回文件依据与未知条件。
乙：检查现有测试覆盖，指出一个值得补验的边界，不修改代码。
两者都不读取教材 solution，不安装依赖，不改写样本。
请等待两份结果，由主 Agent 对照文件消除冲突，
给出最小实施范围与验证办法，先不开始修改。
```

若当前环境不支持子 Agent，用一个 Agent 分两次完成同样调查即可。这项练习的终点是可靠的分工与证据，不是一定看到两个活动卡片。

写入任务需要更清楚的隔离

只读任务也可能基于不断变化的文件得到矛盾结论，因此调查期间尽量固定版本；必须写入时，边界更重要。可以让一个 Agent 只修改实现，另一个在独立位置准备样本与检查建议，主线控制最终落盘。不要让两者分别创建不同的数据格式，再期待自动拼合。

Git worktree 是一种隔离工作目录的办法。核对日的官方 Worktrees 文档要求项目属于 Git 仓库；App 新聊天可选择 Worktree 与起始分支。每份检出有独立文件，但隔离目录不等于接口天然一致，也不代表副本含有你期望的所有本地状态。

本章不要求你新建 worktree。已有环境可按官方入口选择已知起点；不熟悉时，独立练习副本与明确文件范围也能先练习。普通复制不会共享 Git 历史；两种方式都应核对实际路径、依赖和输入数据，不能把另一目录的测试结果当作本目录证明。

同目录按文件范围分工时，要明确交叉文件由谁负责。正文作者不修改全局配置，案例作者不重写主题，主线再统一接入，这是本书制作采用过的约束方式。它降低互相覆盖的机会，但主线仍需检查接口和构建；边界本身不是集成已经成功的证据。

返回结果要能被下一人使用

“调查完毕，没有问题”很难整合。更有用的返回包括实际查看的对象、关键结论、对应路径、执行结果与未验证条件。发现疑似缺陷时给一项能区分原因的检查，不要只列十种可能性。

主 Agent 不能只把三份总结粘贴到一起。两份报告冲突时，先核对是否读取同一版本、执行目录是否一致、哪个结论来自观察，哪个是推测。多数票不会把猜测变成事实。审阅者如果只看实施者的总结，也可能重复同一个错误假设；至少挑一条关键行为独立检查。

实现合并后还要针对整合产物验证。两个分支各自通过，不证明组合后的入口、测试导入和数据格式一致。发现覆盖冲突时，保留目标行为与双方证据，逐处解决，不直接以“后来的版本”为准全部替换。

有成本，才需要解释收益

多个 Agent 会重复获取部分上下文，主线也要花时间分配与整合。判断是否值得，可以比较三件实际事情：是否得到以前没有的独立观察；是否减少本来必须顺序等待的工作；整合时是否引入了额外错误。

不必虚构一张精确收益表。一次练习后写“两个调查并行完成，但入口调查只有几行，分工成本偏高”，也是有价值的结论。下一次可以保留测试审阅，其他工作由主 Agent 完成。

如果子任务迟迟没有产生证据，先询问当前对象、阻碍和已执行动作；目标变化时及时收回或调整任务。不要为了等待一个低价值分支，让所有可继续工作停住；但依赖其必要结论的实施也不能靠猜测越过。

动手练习：选择分工，或说明为什么不分

从自己的 TaskTrail、LinkShelf 或毕业题副本选一项真实工作。先用三句话描述依赖、文件边界和整合检查，再决定用一个还是多个 Agent。若选择并行，至少取得一份能独立核查的返回结果；若选择不并行，指出哪项依赖或协调成本使顺序工作更合适。

参考反馈（离线展开）

调查入口与测试覆盖可以先并行；最终审阅应等待实际修改。两个人同时修改一个小函数通常不划算。合格结果应指出同一事实从哪里核查，以及谁负责最后的集成与验证，而不是只写“分工更高效”。没有实际运行子 Agent 的读者，记录方案判断即可，不把它标为协作已执行。

多 Agent 是一种可选择的工作组织方式。你仍然负责目标、边界和最终证据；下一章把这些判断收成一份短而有效的个人手册。

第 27 章：把一次成功变成下一次更容易成功

修好一次刷新故障后，你可能很想把整个过程写成永久规则：“所有任务先查存储，再运行三轮测试，再请两个 Agent 审阅。”下一次任务只是整理一份文本，这套规则却仍然要求一切照做。经验保存下来了，判断反而消失了。

本章要留下的是适用条件和有效动作。完成后，你会有一份短个人手册，知道一项方法应该放在任务说明、项目指引、Skill、脚本还是自动化里，也知道什么时候直接不保存。

先找到成功中真正可以迁移的部分

Phantom Delete 的可迁移经验，并不是“删除问题都要加一次保存”。它是先记录复现，再比较当前状态与重新读取的状态，选择能区分假设的检查。换成 CSV 汇总、命令行筛选或会议整理，具体检查会改变，“结论要有对应证据”仍然成立。

同样，LinkShelf 使用原生页面与本地存储，是这个小型离线案例的合适选择，不能据此推导所有项目都不该有数据库。整理经验时先问：这条做法依赖哪些条件？改变条件后，它仍然有用吗？

先用一次真实任务验证答案。若只是模型提出的建议，还没有被你试过，保留为待试方法，不急着升级成稳定规则。一次偶然成功也不必立即推广；下一次使用时观察它是否减少了重复调查或漏验。

按用途选择保存位置

不同位置解决不同问题，不能只因为名字更高级就迁移过去。

位置	适合留下什么	何时不必用
当前任务说明	本轮目标、材料、完成状态与临时限制	不要把临时要求写成全局永久规则
项目指引	多轮都成立的启动、数据边界和检查约定	只发生一次、很快过期的状态应另存
Skill	重复出现、需要判断与资源配合的任务方法	一条命令已足够时不必封装一套流程
脚本	输入输出明确、值得重复执行的确定性步骤	含义尚未稳定时先探索，不急着自动运行
自动化	已可重复执行、还需要按时间或事件触发的工作	未验证的步骤不因定时就更可靠

同一方法可以有几个配合部分。例如项目指引说清运行方式，脚本负责确定性计算，Skill 说明何时使用和怎样解释结果。它们不需要各复制一份完整规则。保留一个明确来源，比五个逐渐不一致的副本更容易维护。

自动化还涉及触发频率、输入来源、失败处理与授权。只有你确实需要定期发生，才值得增加这一层。不要把“以后可能复用”写成已经创建后台任务；文本笔记不会自己调度，也不能授予外部服务权限。

从具体任务提炼一项方法

以案例 E2 的文本工具为例。刚接手时，判断段落边界需要读报告、看输入、解释预期；这部分仍有不确定性。确定规则后，统计与回归可以由 Python 程序重复执行。将“每次分析前重新讨论所有空白含义”保留为手工流程，反而会增加漂移。

可以请 Codex 做一次有限整理，发送到 Codex 输入框：

```
请根据我们实际完成的任务，提炼三条值得复用的方法。
每条注明适用条件、最小动作、证明有帮助的观察，以及不适用情形。
只把稳定项目事实放进项目指引，当前状态留在接续记录。
先指出哪些重复步骤适合脚本；没有实际复用价值的建议不要封装。
不要自动创建定时任务、连接外部服务或修改全局配置。
```

返回结果还需要你判断。若它建议把每次聊天都自动上传为材料，先看这是否属于你的目标和授权。若它只是给每个步骤换一个术语，没有减少任何重复工作，就可以删掉。

写一份足够短的个人手册

在自己的笔记或专用目录保存 my-codex-manual.md。这个文件名没有特殊产品能力，只有在你阅读或明确提供时才参与工作。不要为了试写个人习惯，直接改变教材根目录的 AGENTS.md。

下面是一份可改写示例：

```
# 我的 Codex 工作手册

- 陌生材料先定位入口与目标对象；不把别的版本当成当前事实。
- 目标模糊时，用小尝试比较方向；确定方向后才扩大实现。
- 小任务直接推进，重要取舍与必要授权向我说明具体影响。
- 修改后按目标行为检查，记录实际命令与未覆盖条件。
- 连续尝试没有新证据时，停止重复补丁，重新检查假设。
- 中断前留下当前产物、可信基准、缺口与下一项动作。
- 方法值得复用时再固化；失效规则直接删改。
```

它保留判断，没有规定每次必须分几步、用几个 Agent、跑几轮检查。你可以加入一条自己真实需要的偏好，例如“解释命令前先说明运行目录”，并观察它是否帮助下一次协作。

个人偏好也不应压过具体项目要求。若某项目有必要的测试或数据约束，要在该项目中遵守；“我习惯快速做”不能成为跳过已知关键验收的理由。手册帮助你开始判断，不代替当前事实。

用第二次任务检验手册

手册写得流畅，不等于有效。选一项规模适中的新任务，比较它有没有帮助你更快找到起点、减少遗漏，或在偏离时给出下一项检查。不要只问 Codex“这份手册是不是最佳实践”，它的赞同不能证明使用价值。

可以给每次实际复用留一句结果：“先核对版本，发现测试导入了参考答案，避免了错误交付。”这条观察支持保留版本核对；“加入七条格式规则后，每次都花很多时间修措辞”则提示规则可能不划算。没有进行对照计时时，不必写节省了多少分钟。

当某条规则反复需要例外，先检查它是否过宽。把“所有任务先制定计划”改为“关键路线未知且会影响大量工作时，先安排一项探索或小实验”，常比追加越来越长的例外条款清楚。

删除也是维护

项目一旦更换技术，旧启动命令可能失效；临时权限恢复后，绕行步骤也可能不再需要。保留失效方法，会让未来任务沿旧路走偏。定期在真正使用时局部核查，而不是为手册再建设一套沉重审查系统。

对临时失败尤其谨慎。一次安装受限，说明当前环境缺条件；不能直接升级为“永远不要使用这个工具”。一次小程序成功，也不能升级为“永远用这套框架”。把场景和证据一起保留，经验才不会变成过时命令的收藏夹。

动手练习：留下三条，删除两条

从自己真实完成的案例提炼三条方法，每条写明适用条件与观察依据。再删除或改写以下两条：“任何任务都先派三个 Agent”“只要全部测试通过，就可以写功能完成”。最后把短手册用于一次新的小任务，记录一项帮助和一项需要调整的地方。

参考反馈（离线展开）

第一条应改成按独立性与整合成本选择分工；第二条应检查测试对象、断言与目标行为是否对应。有效手册会引出更准确的动作，而不是增加仪式。若暂时没有新任务，可完成草稿，但应写“尚未复用”，不要把自己的满意度当作效果验证。

你最终留下的应是更容易作出好判断的方法。现在进入第 28 章毕业实战，离开逐步样稿，在陌生任务上检查这些能力。

第 28 章：毕业实战--在陌生任务上独立驾驭 Codex

前面的案例已经教过你怎样调查、实施和纠偏。这一章不再提供完整任务轨迹。你会遇到材料不完整、基础测试不足、模型可能替你补全未知的情况，需要自己判断下一步。

毕业的终点不是得到和作者相同的程序，也不是让 Codex 从头到尾没有失败。你应当能够解释目标，提供必要材料，组织一次有效执行，用证据判断成果，处理一项偏差，并留下别人可以继续的位置。案例 E 为此准备了两个陌生选题，默认选择会议记录整理，也可以选择文本工具维护。

E1：离线会议记录整理助手

题目位于 cases/E-capstone/e1-meetings/。输入是 input/ 中四份虚构 Markdown 纪要，任务约定在 TASK.md。团队想知道形成了什么决定、还有哪些行动、负责人和期限是什么，并能回到原文核对。

你要交付一个可重复运行的本地整理工具、结构化结果与可阅读摘要。内部实现和输出格式由你决定，不需要网页、账号或在线服务。四份材料是独立输入，不得为了程序方便删除或改写有歧义的句子。

这道题的困难在事实边界。参与者中有同名者，某些行动没有负责人；有一份纪要未注明会期，出现相对日期；后续会议又取消或调整了先前事项。输出中缺失负责人或无法确定的绝对截止日期必须填“未注明”，同时保留必要原话。不能从电脑当前日期、文件名或参会顺序补答案。

每条结论需要相对源路径和从 1 开始的行号，跨行内容可用范围；一项变更涉及几处原文时，应能沿来源解释。明确取消的行动不能继续列入有效待办，但取消事实仍应可追溯。相似措辞不一定是同一事项，建议也不等于已经形成决定。

这些是验收边界，不是提取算法。怎样调查、怎样组织字段、先验证哪个风险，都由你与 Codex 判断。不要先读教师区，再把答案包装成独立完成。

E2：维护一份陌生文本工具

替代题位于 cases/E-capstone/e2-texttool/starter/。这是一个 Python 标准库 CLI，读取 UTF-8 文本，输出 paragraph_count 和 paragraphs。目录提供源程序、含糊问题报告、两个样本和两条基础测试。

你要查明为什么部分笔记的段落统计与预期不同，做最小修复并补上有效回归。输入中的非空行必须按原顺序保留，段内换行不应该全部变成段落分隔，重复内容也不能被自动去重。题面会说明空白与输出边界，但不会告诉你代码该改哪一行。

制作时，起点的两条基础测试已经实际通过，CLI 可以运行；这不表示题目已解决。它们覆盖普通文本与空输入，问题报告所涉及的其他条件仍需读者调查。本题没有参考修复版，也没有故意让基础测试长期变红来替你标出答案。

读者只需要 Python 3.9 或更新版本，不需要安装依赖、获取 API Key 或开放网络。具体运行命令在起点 README.md；调查前先确认实际目录和输入，不拿其他案例的测试证明这一题。

准备独立工作位置

从教材根目录复制一个尚不存在的副本，原始材料留作对照。命令属于操作系统终端，不是发给 Codex 的指令。以下两项选其一：

```
# macOS / Linux
cp -R cases/E-capstone/e1-meetings my-meeting-lab
cp -R cases/E-capstone/e2-texttool/starter my-text-lab
```

```
# Windows PowerShell
Copy-Item -Recurse cases/E-capstone/e1-meetings my-meeting-lab
Copy-Item -Recurse cases/E-capstone/e2-texttool/starter my-text-lab
```

已有同名目录时改用新名称，后续路径也相应修改。到自己的副本中打开 Codex，保留你实际采用的任务说明。这里不提供第一条提示词的标准答案，因为识别目标、材料和未知，本身就是考核的一部分。

不熟悉编程的读者可以选 E1，让 Codex 负责程序实现，你负责原文抽查、未知判断与验收。熟悉代码的读者也不能跳过这些判断：一个运行成功却编造负责人和日期的工具，仍未完成任务。

交付的是成果与判断

提交内容分为四组即可。第一组是实际产物：E1 的工具与输出，或 E2 的修复与新增检查。第二组是复现与运行方式，包含必要环境、输入位置和输出位置。第三组是验证记录，说明执行过什么、看到了什么、哪些未覆盖。第四组是纠偏与接续说明，解释一项原判断如何被新证据改变，以及下一次从哪里继续。

不需要记录每句对话，也不要求模型故意先失败。发现自己的提取规则把建议当成决定，再用原文修正，就是有效纠偏；发现基础测试没有覆盖真实输入，补上能揭示差异的检查，也是一项判断调整。若你引用了教师区或外部资料，如实记录范围即可，不必把参考伪装成独立推导。

请保留至少三条值得抽查的边界。E1 应能回到原文定位；E2 应能用具体输入说明差异。不要只报告“所有检查通过”，而应让另一个人能够照着你的材料复查一项关键结论。

用七个观察判断自己是否掌握

首先，目标是否清楚：你能否用自己的话说出解决谁的什么问题，哪些结果不在本轮范围？其次，上下文是否可靠：说明中的现状来自文件、运行、使用者，还是模型推测？

再看推进路线：你的第一项行动是否针对关键未知，后续是否根据观察调整？执行本身产生了什么可用产物，还是只留下讨论？验证有没有对准目标版本和边界，而不只是程序启动？

最后检查纠偏、解释与接续：遇到偏差时是否改变了没有依据的假设；结论是否承认未知；新会话能否定位当前结果、证据和下一步。这些维度互相关联，无需给自己制造一个精确总分，但任何核心使用路径失败都不应被漂亮格式抵消。

如果你能解释哪些部分已可用、哪些仍待验，并留下可继续的真实状态，就已经在使用本书的方法。若最终目标尚未达到，继续工作或按第 19 章合理调整范围；“知道没完成”是准确判断，仍不能当作任务已经完成。

验收时保留一点陌生性

可以让没有参与实现的人选一条原文或一份文本检查结果，也可以在自己的输入副本中只改变一项事实，再重复运行。观察输出是否随之产生可解释的变化，原输入和其它无关记录是否仍完整。不要把事先背过的几条固定输出当作全部能力。

教师审阅提示在 cases/E-capstone/teachers/README.md，含边界剧透，没有完整实现。建议先保存自己的提交再阅读。发现遗漏后，可以补充第二版验证，清楚记录“首次未覆盖、后来补齐”，不需要抹掉真实过程。

给未来的自己留一个入口

毕业记录最后保留三句话：这次最有价值的判断是什么；哪条经验值得放进个人手册；下一次遇到陌生任务，准备从什么观察开始。

你带走的不是一套必须照抄的提示词，而是把意图带到事实、行动、证据和可继续成果的能力。这套循环可以在下一项没见过的工作中重新开始。案例材料与全部源码入口见实战实验室。

附录：术语、操作与来源

本附录服务于导读与全书 28 章。首次官方事实核对日期为 2026-10-25，2026-10-26 审校再次打开核对了 CLI 参数、Skill 发现位置、MCP 与模型选择相关页面。具体入口、命令参数和账号权益可能变化，遇到差异时优先比较当前程序的帮助与对应官方文档，不根据一张旧截图推断功能不存在。

用得到再查的术语

术语	在本书中的意思
模型 · model	根据已获得的信息生成内容与行动选择的模型。它的判断需要证据支持。
Agent · 智能体	围绕目标调用工具、接收结果并继续推进的执行单元。
工具 · tool	读取文件、执行命令、操作浏览器等具体能力；是否可用取决于当前入口与授权。
Harness · 执行框架	组织模型、会话、工具调用、权限与反馈循环的外围系统。
终端 · terminal	显示文字输入输出、让你运行命令的窗口。
Shell	解释终端命令的程序，例如 zsh、Bash、PowerShell；不同 shell 语法可能不同。
CLI · 命令行界面	通过终端启动与使用程序的方式。本书以 Codex CLI 为主要可复现入口。
IDE · 集成开发环境	包含代码编辑、项目导航等能力的软件。Codex 扩展是其中的一个入口。
路径 · path	文件或目录的位置。相对路径以当前目录为起点，绝对路径给出完整位置。

术语	在本书中的意思
工作区 · workspace	当前任务关联的项目位置；可读、可写范围仍由实际权限确定。
沙箱 · sandbox	限制执行动作可访问和修改范围的机制。
审批策略 · approval policy	决定动作在什么情况下需要额外审批的策略，与沙箱边界共同起作用。
仓库 · repository	保存项目与版本历史的位置。下载 ZIP 得到文件，不会自动得到本地 Git 历史。
Git / diff	Git 用于记录版本；diff 是两个状态之间的文件差异。第 14 章提供独立临时仓库练习。
测试 · test	在特定输入和条件下检查结果的方法；通过只能说明其覆盖范围内的行为。
localStorage	浏览器按站点来源保存字符串的存储能力，不是云端数据库或永久备份。
来源 · origin	浏览器用协议、主机名和端口区分的站点身份；本书练习保持来源一致。
Skill	可复用的任务方法与资源；适用时可被 Codex 使用，具体入口随产品配置变化。
MCP	让模型宿主与工具、资源连接的协议。连接本身不等于所有操作都已获准。

常用操作速查

下面命令输入到操作系统终端，除非另有说明。它们不替代正文中的前提和路径检查。

想做的事	命令或操作	范围
查询 CLI 版本与帮助	codex --version、codex --help	不需要启动任务；本机实际执行。
登录与查看登录方式	codex login、codex login status	官方核对；本轮没有重新登录。
只读调查	codex --sandbox read-only	在教材根目录启动；最终权限以当前宿主为准。
修改工作副本	codex --sandbox workspace-write	在自己的 my-linkshelf 目录启动。
查看会话状态	在 Codex 输入框输入 /status	这是 CLI 内命令，不是 shell 命令。
运行参考成品	python3 -m http.server 4174 --bind 127.0.0.1 --directory cases/A-linkshelf	在教材根目录；浏览器打开 http://localhost:4174/solution/。
Windows 运行参考成品	将上一条命令的 python3 换成 py -3	需 Python Launcher；未在本机执行。
关闭本地服务	在运行服务的终端按 Ctrl+C	不会删除网页文件或浏览器已保存数据。
检查案例数据逻辑	node --test cases/A-linkshelf/tests/core.test.mjs	在教材根目录；Node 原生测试，非浏览器验收。

按症状找到第一项检查

症状	先检查什么
找不到 codex 命令	是否已安装到当前环境，安装后是否重开终端；PowerShell 与 WSL 是否混用。
找不到 cases	当前目录是否是含 book 与 cases 的教材根目录。
Codex 解释得很详细却没有文件引用	要求引用实际读取的路径与片段，区分上下文中的信息与推测。
工具提示权限拒绝	被拒绝的动作与目标路径；不要直接判定实现有错。
服务提示端口占用	是否仍开着上一窗口的服务；停止旧服务或同步改变命令与浏览器端口。
页面出现目录列表	参考成品 URL 是否包含 /solution/；服务目录是否正确。
界面可见但功能不工作	是否双击 file:// 打开；改用正文中的 HTTP 地址并查看脚本错误。
换地址后记录不见	浏览器、协议、主机名和端口是否相同；先回到原来源。
删除后刷新又回来	记录完整动作，再调查删除后的存储写入和刷新时的读取；先不要增加延时。

官方资料核对记录

以下页面在核对日实际打开并读取了相关段落，不是只凭搜索摘要整理。OpenAI 资料限于官方文档与官方开发者文章。产品概念与操作在正文中用自己的语言讲解，原始资料供读者核对适用条件。

知识点	已查阅资料	本次确认的范围
CLI 的安装入口与项目启动	Codex CLI	当前页面提供独立安装器及其它安装选项；从项目目录运行 codex。
npm 安装方式	Using Goals in Codex	仅引用其快速开始中的 npm 安装、版本查询命令；前三章不依赖 Goals 功能。
Windows npm 安装支持	Deploy the Windows app and Codex CLI	Windows 部署仍提供 npm 安装路线；不把企业管理步骤写成普通读者必做步骤。
登录方式与状态	Authentication	ChatGPT 与 API Key 登录的区别、浏览器登录、codex login status；未替读者确认账号权益。
沙箱与审批边界	Sandbox	只读与工作区写入的概念、审批与沙箱的区别；具体配置取决于入口和管理策略。
CLI 会话状态	Developer commands	/status 用于查看当前会话配置；与操作系统命令区分。
IDE 启动入口	Codex IDE extension	打开项目和 Codex 侧栏；兼容编辑器入口有差异。
桌面启动入口	ChatGPT desktop app	选择工作位置并发起聊天；当前官方名称可能与既有 App 名称不同。
WSL 环境差异	WSL	WSL2 内运行的 CLI 属于 Linux 环境；不与原生 PowerShell 安装混用。
Agent 与执行框架	Codex as a platform	模型之外还需要会话、工具、权限、反馈等执行组织。

知识点	已查阅资料	本次确认的范围
本地静态服务	Python http.server	目录与绑定地址参数；本书只作本机开发预览。
浏览器持久化边界	MDN localStorage	按来源保存、浏览器策略可能拒绝、无痕模式与 file: 行为的限制。

扩展能力与全书验证来源

第 9、16、21-24、26 章的客户端事实各有适用条件；官方资料说明可用入口，案例记录说明本次真正执行了什么。两者不能互相替代。

主题	官方资料	核对边界
项目指引	AGENTS.md	发现与层级规则见第 9 章；新会话恢复仍需读者实际检查。
审查与非交互命令	CLI reference	本机核对过帮助，没有额外运行模型审查或模型批处理；目标参数与自定义审查指令不能随意组合。
Skill	Build skills	区分元数据、按需读取与本地发现目录；本仓库示例在 extensions/，没有自动安装。
MCP	Model Context Protocol	CLI 配置与桌面宿主工具目录不是同一视图；没有新建账号连接。
模型与推理	Models	可用项由账号、客户端和工作区决定；没有制作付费模型性能基准。

主题	官方资料	核对边界
子 Agent 与工作目录	Subagents、Worktrees	本书实际使用分工，但没有对照提速测量，也未另行实测 CLI 子线程选择入口。

以下链接指向各案例的实际源记录。命令在自己的副本运行后，才能记为自己的结果；参考项目通过不证明工作副本通过。

来源	提供的证据与限制
案例 A README	原生网页运行、数据逻辑检查、人工验收动作；无标签、导入和云同步。
案例 B verification	起点与参考成品的 CLI、状态筛选与数据保护检查；未代替读者完成新会话接续。
案例 C verification	小样本和完整样本的实际汇总、独立核对、重复输出及失败保护；不证明真实业务口径。
案例 D README	故障复现与参考修复的数据检查；故障版测试通过不代表已修复，副本必须另验。
案例 E verification	会议输入材料和文本 CLI 起点检查；毕业题保留未解状态，无完整参考答案。
扩展示例 verification	Git 临时练习、Skill 格式和显式复用、报告包装脚本、只读工具调用与命令帮助核对。

网站、浏览器与出版输出的集成状态另见仓库 progress.md；这些项目级检查不会替代读者在自己的系统、账号与数据上的验收。

部分旧文档入口会重定向，个别旧路径返回 404。本附录保留实际成功打开的现行目标，不把旧资料包中的 URL 一律当作永久有效。

本机实际核查与未验证边界

本轮正文制作实际执行 codex --version、codex --help、node --version、python3 --version。环境是 macOS，结果分别为 Codex CLI 0.154.0、Node.js v22.21.1、Python 3.10.13。CLI 帮助确认了 --sandbox 的只读/工作区写入选项和版本查询入口。查询时出现不能创建 PATH 别名的警告，命令仍返回版本与帮助；这不等于安装已经失败，也不等于已完成登录检查。

案例 A 的数据测试命令实际执行，10 项通过、0 项失败，覆盖新增与再次读取、删除与再次读取、输入校验、损坏数据、存储失败、搜索和导出。测试采用程序化存储对象，不等于真实浏览器权限行为已通过。案例启动、HTTP 与浏览器验收的最新记录见案例 README.md 及仓库 progress.md；本附录不把尚未补齐的人工观察写成成功。

本轮没有重新安装或重新登录 Codex，没有在 Windows / Linux 本机运行安装与交互流程，没有证明所有账号可用同一模型。网页演示中的状态、练习反馈和示意任务都不调用真正的 Codex。读者的独立完成标准仍以各章实际操作与观察为准。